Python 28 天 • 全阶段

总复习册 · Python 28 天

Python 阶段 28 天，一天一份，全都收在这一本里。

共 28 天：第1天、第2天、第3天、第4天、第5天、第6天、第7天、第8天、第9天、第10天、第11天、第12天、第13天、第14天、第15天、第16天、第17天、第18天、第19天、第20天、第21天、第22天、第23天、第24天、第25天、第26天、第27天、第28天

每份前面的深色页 = 新的一天开始 —— 面试前一晚直接按它过一遍。

配套：① 开始今天的学习.cmd 每天会自动打开当天那一份；这本合订本是给你周末连着翻、面试前一晚整个过一遍用的。

这本里都有哪几天

第1天 脚本骨架：变量、四种类型、格式化、输入 11/2

第2天 分支与循环：ifelif、whilefor、range、breakcontinue 11/3

第3天 容器四件套：list tuple dict set，切片与排序 11/4

第4天 ★Excel 一天速成：数据透视表 + XLOOKUPVLOOKUP + SUMIFS 11/5

第5天 函数：参数、返回值、作用域、lambda 11/6

第6天 面向对象 + 模块与包 + 常用标准库 11/7

第15天 pivot_table 透视表：indexcolumnsvaluesaggfunc 四件套 11/16

第16天 时间序列：to_datetime、resample、rolling、同比环比 11/17

第17天 applymaplambda + 多级索引：什么时候该用、什么时候别用 11/18

第18天 可视化：matplotlib 中文设置 + 三张有业务含义的图 11/19

第19天 ★Excel 进阶：Power Query 清洗 + 条件格式 + 图表；下午开投 11/20

第20天 把 W4–W6 的脚本收成「一个能讲的项目」 11/21

全阶段 · 第 1 天 · 11/2

脚本骨架：变量、四种类型、格式化、输入

写一个 .py 文件，收两个输入、算出一个结果、用 f-string 格式化输出，全程不查语法。

第 1 天 · 11/2 · 今日知识点回顾

脚本骨架：变量、四种类型、格式化、输入

目标：写一个 .py 文件，收两个输入、算出一个结果、用 f-string 格式化输出，全程不查语法。

对着 B 站视频看：视频讲到的每一句，这张 PPT 里都有一条；复习时只翻这份，不用再回去翻分P。

今天结束时，你应该能做到

验收标准（做到这句就算过关）

- 写一个 .py 文件，收两个输入、算出一个结果、用 f-string 格式化输出，全程不查语法。

今天的做法（顺序别颠倒）

- 本机 VSCode 里先写这几段（热手 + 验证概念）：
- 题库地址：file:///D:/DSH工作区/00-学习中心/60-练习环境/Python-Env/data
- （今天的练习数据都在这个目录；明天起真正开始读它）
- 做完记一句：今天卡在哪、错在哪 → 写进 3-每日打卡。

今天看哪几集、读哪一章

这一天的知识：5 集 · 合计约 55 分钟（边看边在本机敲一遍）

P14	字面量	约 9 分钟
P16	变量	约 12 分钟
P17	数据类型	约 10 分钟
P23	字符串格式化	约 13 分钟
P28	数据输入(input语句)	约 11 分钟

教材（已下载到 20-教材与讲义\python-data\，断网也能看） https://raw.githubusercontent.com/jackfrued/Python-100-Days/master/Day01-20/03.Python语言中的变量.md

视频地址就在下面的「今天做哪些题」网页里，点一下直达分P。

知识点 1/7 • 变量没有类型，值才有

- 一句话：Python 的变量只是「贴在对象上的名字」，换个值就换个类型，赋值时不报错。
- 代码：

```
x = 100
print(type(x), x)      # <class 'int'> 100
x = '100'
print(type(x), x)      # <class 'str'> 100   ← 同一个名字，换了类型，不报错
```

★ 这是和 SQL 最大的差别：SQL 的列类型固定，塞错类型当场报错；Python 把类型错误推迟到「你真正用它算的时候」才炸 —— 所以「跑通了」不等于「对了」。

知识点 2/7 • 四种基本类型，以及怎么转

- 一句话：int（整数）/ float（小数）/ str（文本）/ bool（真假日），转类型用同名函数包一下。
- 代码：

```
print(int('12') + 1)          # 13
print(float('12.5') * 2)      # 25.0
print(str(12) + '5')          # '125'  ← 字符串相加是拼接，不是 17
print(int(12.9))              # 12     ← 直接砍掉小数，不四舍五入
```

★ int('12.5') 会报 ValueError: invalid literal for int() with base 10: '12.5'—— 带小数点的字符串不能直接转 int，要先 float 再 int。这条坑清洗数据时天天踩。

知识点 3/7 • 三种除法，别混

- 一句话：/ 是真除法（得小数），// 是整除（向下取整），% 是取余。
- 代码：

```
print(7 / 2)     # 3.5
print(7 // 2)    # 3
print(7 % 2)     # 1
print(-7 // 2)   # -4   ← 不是 -3，// 是「向下」取整，不是「向零」
```

★ 算「第几页」「第几组」时用 //，算「能不能整除」时用 %；两个都用错会差 1。

知识点 4/7 • 字符串三种写法，f-string 是主力

- 一句话：单引号、双引号等价；三引号能跨行；格式化一律用 f'{变量}'。
- 代码：
- name, age, price = '张三', 23, 1899.0

```
print(f'{name} 今年 {age} 岁')
print(f'{price:.2f} 元，打九折 {price * 0.9:.2f} 元')   # 1899.00 元，打九折 1709.10 元
print(f'{1234567:,}')                                    # 1,234,567
```

★ f-string 的冒号后面是「格式说明」：.2f 两位小数、，千分位、>10 右对齐补空格。

- 写报表时这三个够用了，别再回去用 % 和 .format。

知识点 5/7 • input 拿到的永远是字符串

- 一句话：input() 返回 str，要算数必须先转，转不了就报 ValueError。
- 代码：

```
h = float(input('身高(m): '))     # 用户敲 1.75 → 得到 1.75
w = float(input('体重(kg): '))
print(f'BMI = {w / h ** 2:.2f}')
```

★ 直接 int(input()) 而用户敲了空回车 → ValueError: invalid literal for int() with base 10: '' 真实脚本里要用 try/except 兜住（第 7 天专门讲）。

知识点 6/7 • 变量命名与注释

- 一句话：小写字母+下划线；不能数字开头；不能是关键字；注释用 #。
- 代码：

```
order_total = 0          # 好
# 2order = 0             # 语法错：不能数字开头
# class = 'A'            # 语法错：class 是关键字
```

★ 中文变量名 Python 3 允许（订单总额 = 100 能跑），但报错信息里中文会变成一串转义，排查时更费劲 —— 一律用英文。

知识点 7/7 • 第一个「真数据」：用最笨的办法读 CSV

- 一句话：CSV 就是逗号分隔的文本，一行一条记录，第一行是表头。
- 代码：

```
import pathlib
DATA = pathlib.Path(r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
with open(DATA / 'orders_dirty.csv', encoding='utf-8-sig') as f:
for i, line in enumerate(f):
print(line.rstrip().split(',')[:6])
if i >= 3:
break
```

- （encoding='utf-8-sig' 是为了吃掉 Excel 存 CSV 时写在开头的 BOM，不写的话第一个列名会变成 '\ufefforder_id'，后面按列名取值全部 KeyError。）

代码速查：今天这几段，要在本机跑通（1/4）

```
import pathlib
```

```
DATA = pathlib.Path(r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
```

1) 变量换类型

x = 100

```
print(type(x), x)
```

x = '100'

代码速查：今天这几段，要在本机跑通（2/4）

```
print(type(x), x, int(x) + 1)
```

```
# 2) 格式化：把 1899.0 输出成「1,899.00 元，打九折 1,709.10 元」
```

price = 1899.0

```
print(f'{price:,.2f} 元，打九折 {price * 0.9:,.2f} 元')
```

3) 输入 + 计算（跑的时候手动敲 1.75 和 70）

h = float(input('身高(m)：'))

w = float(input('体重(kg)：'))

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（3/4）

```
print(f'BMI = {w / h ** 2:.2f}')
```

```
# 4) 用最笨的办法读真数据的前 4 行
with open(DATA / 'orders_dirty.csv', encoding='utf-8-sig') as f:
    for i, line in enumerate(f):
        print(line.rstrip().split(',')[:6])
        if i >= 3:
            break
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（4/4）

验收（做到才算过关）：

① 上面 4 段全部跑出结果，截图或复制输出留一份；

② 能不看资料说出 int('12.5') 为什么会报错、怎么绕过；

③ 能把 -7 // 2 的结果和原因讲清楚。

最容易错的地方

★ 这是和 SQL 最大的差别：SQL 的列类型固定，塞错类型当场报错；Python 把类型错误推迟到「你真正用它算的时候」才炸 —— 所以「跑通了」不等于「对了」。

★ 算「第几页」「第几组」时用 //，算「能不能整除」时用 %；两个都用错会差 1。

写报表时这三个够用了，别再回去用 % 和 .format。

★ 中文变量名 Python 3 允许（订单总额 = 100 能跑），但报错信息里中文会变成一串转义，排查时更费劲 —— 一律用英文。

这些句子都是「差一个字、结果差一截」的地方，考前只翻这一页也值。

合上视频，自测一遍（能打勾才算学会）

- [ ] 我能不看资料写出：算「第几页」「第几组」时用 //，算「能不能整除」时用 %；两个都用错会差 1。
- [ ] 我能不看资料写出：中文变量名 Python 3 允许（订单总额 = 100 能跑），但报错信息里中文会变成一串转义，排查时更费劲 —— 一律用英文。
- [ ] 我能不看资料说出：写报表时这三个够用了，别再回去用 % 和 .format。
- [ ] 我能不看资料说出：一句话：input() 返回 str，要算数必须先转，转不了就报 ValueError。
- [ ] 我能不看题源，把今天用到的每个函数「为什么要有它」说一遍
- [ ] 我能说出今天每个参数/返回值改了会怎样——换个写法结果差在哪

这些句子都是「差一个字、结果差一截」的地方；三个勾以上，今天就算过关。

今天收工前，先知道明天学什么

第 2 天 • 分支与循环：if/elif、while/for、range、break/continue

不用提前学。明天点「① 开始今天的学习」，会换上明天那份 PPT。

收工三件事（5 分钟）

- 今天没跑通的脚本存进 90-我的作业\，命名 2026-11-02-第01天-*.py
- 打卡表「③ 每日打卡」记一笔：今天卡在哪、错在哪（一句话就够）
- 错题明天重做一遍，练习数据在 60-练习环境\Python-Env\data

全阶段 · 第 2 天 · 11/3

分支与循环：if–elif、while–for、range、break–continue

不看资料写出「猜数字」和「九九乘法表」两个小程序，并说清 for 和 while 各该在什么场合用。

第 2 天 · 11/3 · 今日知识点回顾

分支与循环：if/elif、while/for、range、break/continue

目标：不看资料写出「猜数字」和「九九乘法表」两个小程序，并说清 for 和 while 各该在什么场合用。

对着 B 站视频看：视频讲到的每一句，这张 PPT 里都有一条；复习时只翻这份，不用再回去翻分P。

今天结束时，你应该能做到

验收标准（做到这句就算过关）

- 不看资料写出「猜数字」和「九九乘法表」两个小程序，并说清 for 和 while 各该在什么场合用。

今天的做法（顺序别颠倒）

- 本机 VSCode 里写这三段，每段都要跑出结果：
- 题库地址：file:///D:/DSH工作区/00-学习中心/60-练习环境/Python-Env/data
- （明天开始用这份数据做分支/循环的过滤统计）
- 做完记一句：今天卡在哪、错在哪 → 写进 3-每日打卡。

今天看哪几集、读哪一章

这一天的知识：6 集 · 合计约 79 分钟（边看边在本机敲一遍）

P29	布尔类型和比较运算符	约 18 分钟
P30	if语句的基本格式	约 10 分钟
P34	if_elif_else组合使用的语法	约 17 分钟
P43	for循环的基础语法	约 10 分钟
P45	range语句	约 12 分钟
P49	continue和break	约 12 分钟

教材（已下载到 20-教材与讲义\python-data\，断网也能看） https://raw.githubusercontent.com/jackfrued/Python-100-Days/master/Day01-20/05.分支结构.md

视频地址就在下面的「今天做哪些题」网页里，点一下直达分P。

知识点 1/8 • Python 靠缩进分块，不靠大括号

- 一句话：冒号后面缩进 4 个空格就是「属于这个 if / for 的代码」，缩进错了逻辑就错了。
- 代码：

```
score = 72
if score >= 60:
print('及格')
print('这句也在 if 里')
print('这句一定执行')
```

★ 混用 Tab 和空格会报 TabError: inconsistent use of tabs and spaces in indentation。

- VSCode 右下角把缩进设成「空格: 4」，一辈子不用再管这条。

知识点 2/8 • 比较与逻辑运算：and 比 or 先算

- 一句话：and 优先级高于 or，混用必须加括号。
- 代码：
- age, city = 28, '北京'

```
print(age > 25 and city == '北京')          # True
print(city == '上海' or age > 25 and age < 30)   # True — 先算 and
print((city == '上海' or age > 25) and age < 30) # True — 括号改顺序
```

★ 和 SQL 的 AND/OR 优先级规则一模一样 —— 这条你已经会了，只是换个写法。

知识点 3/8 • if / elif / else：从上往下，命中即停

- 一句话：elif 是「否则如果」，只会执行第一个成立的分支，后面的不再判断。
- 代码：

```
score = 72
if score >= 90:
grade = 'A'
elif score >= 75:
grade = 'B'
elif score >= 60:
grade = 'C'
else:
grade = 'D'
print(grade)     # C
```

知识点 3/8 • if / elif / else：从上往下，命中即停（续）

★ 顺序写反就全错：如果把 >= 60 放在最前面，100 分也会被判成 C。

- 分档判断永远「从严到宽」排。

知识点 4/8 • for + range：已知次数用它

- 一句话：for i in range(n) 循环 n 次，i 从 0 到 n-1；range(1, 11) 是 1 到 10。
- 代码：

```
for i in range(3):
print(i)              # 0 1 2
for i in range(1, 11):
print(i, end=' ')     # 1 2 3 4 5 6 7 8 9 10
print()
print(sum(range(1, 101))) # 5050
```

★ range 是「左闭右开」：range(1, 11) 到 10 为止。SQL 的 BETWEEN 是闭区间，两边的「含不含尾巴」正好相反，切换语言时最容易差 1。

知识点 5/8 • while：不知道次数、靠条件停

- 一句话：while 后面跟条件，条件为真就一直转；一定要有能让条件变假的语句。
- 代码：
- n, total = 1, 0

```
while n <= 100:
```

知识点 5/8 • while：不知道次数、靠条件停（续）

- total += nn += 1

```
print(total)      # 5050
```

★ 忘了 n += 1 就是死循环 —— 按 Ctrl+C 停。VSCode 里用「运行和调试」面板跑，比在终端里跑更好中断。

知识点 6/8 • break 与 continue：一个跳出、一个跳过

- 一句话：break 结束整个循环；continue 只跳过本轮，接着下一轮。
- 代码：

```
for i in range(1, 10):
if i == 3:
continue        # 3 被跳过
if i == 7:
break           # 到 7 就整个结束
print(i, end=' ')   # 1 2 4 5 6
print()
```

★ for-else 这个语法（循环没被 break 打断才执行 else）Python 有，但可读性差 ——面试写代码时别用，评委要反应两秒。

知识点 7/8 • 列表推导式：一行代替「建空表 + 循环 + append」

- 一句话：[表达式 for 变量 in 可迭代对象 if 条件]，这是 pandas 之前最重要的写法。
- 代码：

```
squares = [i * i for i in range(5)]
print(squares)                          # [0, 1, 4, 9, 16]
evens = [i for i in range(20) if i % 2 == 0]
print(evens)                            # [0, 2, 4, ..., 18]
names = ['张三', '李四', '王五']
print([n + '同学' for n in names])     # ['张三同学', '李四同学', '王五同学']
```

★ 推导式只能放表达式，不能放 print、不能放赋值。要写两句以上就用普通 for 循环，硬塞进推导式只会让代码更难读（也更容易写错）。

知识点 8/8 • enumerate 与 zip：两个最常用的搭档

- 一句话：enumerate 同时给下标和值；zip 把两个列表按位置配对。
- 代码：

```
cities = ['北京', '上海', '广州']
for i, c in enumerate(cities, start=1):
print(i, c)                    # 1 北京 / 2 上海 / 3 广州
amounts = [100, 200, 300]
for c, a in zip(cities, amounts):
print(c, a)                    # 北京 100 / 上海 200 / 广州 300
```

- （别用 for i in range(len(cities)) 再 cities[i] —— 能跑，但 enumerate 更短更不容易越界。）

代码速查：今天这几段，要在本机跑通（1/5）

```
# 1) 分档：把 0-100 的分数映射成 A/B/C/D，用 for 把 55 72 88 96 全部判一遍
```

```
for score in [55, 72, 88, 96]:
```

```
    if score >= 90:
        grade = 'A'
    elif score >= 75:
        grade = 'B'
    elif score >= 60:
        grade = 'C'
    else:
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（2/5）

```
    grade = 'D'
print(score, grade)
# 2) 九九乘法表（用 for 嵌套，输出要能看）
for i in range(1, 10):
    for j in range(1, i + 1):
        print(f'{j}×{i}={i*j}', end='\t')
    print()
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（3/5）

```
# 3) 猜数字（while + break）：随机数 1-100，最多猜 7 次
```

```
import random
```

target = random.randint(1, 100)

```
for attempt in range(1, 8):
```

guess = int(input('猜一个 1-100 的数：'))

if guess == target:

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（4/5）

```
        print(f'第 {attempt} 次猜中，答案 {target}')
```

break

```
    print('小了' if guess < target else '大了')
```

```
else:
```

print(f'7 次没猜中，答案是 {target}')

验收（做到才算过关）：

① 九九乘法表的对齐是对的（用 \t 或 f'{...:2}'）；

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（5/5）

② 能说清 break 和 continue 的差别，各举一个今天代码里的例子；

③ 把猜数字改成「不限次数、直到猜中」，只改 while 条件，不许改别的。

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

最容易错的地方

★ 顺序写反就全错：如果把 >= 60 放在最前面，100 分也会被判成 C。

★ for-else 这个语法（循环没被 break 打断才执行 else）Python 有，但可读性差 ——面试写代码时别用，评委要反应两秒。

★ 推导式只能放表达式，不能放 print、不能放赋值。要写两句以上就用普通 for 循环，硬塞进推导式只会让代码更难读（也更容易写错）。

（别用 for i in range(len(cities)) 再 cities[i] —— 能跑，但 enumerate 更短更不容易越界。）

这些句子都是「差一个字、结果差一截」的地方，考前只翻这一页也值。

合上视频，自测一遍（能打勾才算学会）

- [ ] 我能不看资料写出：和 SQL 的 AND/OR 优先级规则一模一样 —— 这条你已经会了，只是换个写法。
- [ ] 我能不看资料写出：忘了 n += 1 就是死循环 —— 按 Ctrl+C 停。VSCode 里用「运行和调试」面板跑，比在终端里跑更好中断。
- [ ] 我能不看资料写出：for-else 这个语法（循环没被 break 打断才执行 else）Python 有，但可读性差 ——面试写代码时别用，评委要反应两秒。
- [ ] 我能不看资料写出：推导式只能放表达式，不能放 print、不能放赋值。要写两句以上就用普通 for 循环，硬塞进推导式只会让代码更难读（也更容易写错）。
- [ ] 我能不看题源，把今天用到的每个函数「为什么要有它」说一遍
- [ ] 我能说出今天每个参数/返回值改了会怎样——换个写法结果差在哪

这些句子都是「差一个字、结果差一截」的地方；三个勾以上，今天就算过关。

今天收工前，先知道明天学什么

第 3 天 • 容器四件套：list / tuple / dict / set，切片与排序

不用提前学。明天点「① 开始今天的学习」，会换上明天那份 PPT。

收工三件事（5 分钟）

- 今天没跑通的脚本存进 90-我的作业\，命名 2026-11-03-第02天-*.py
- 打卡表「③ 每日打卡」记一笔：今天卡在哪、错在哪（一句话就够）
- 错题明天重做一遍，练习数据在 60-练习环境\Python-Env\data

全阶段 • 第 3 天 • 11/4

容器四件套：list tuple dict set，切片与排序

能用 list/dict 存起来，能按条件筛出子集并排序。

第 3 天 · 11/4 · 今日知识点回顾

容器四件套：list / tuple / dict / set，切片与排序

目标：拿到一份 CSV 的列名和几行数据，能用 list/dict 存起来，能按条件筛出子集并排序。

对着 B 站视频看：视频讲到的每一句，这张 PPT 里都有一条；复习时只翻这份，不用再回去翻分P。

今天结束时，你应该能做到

验收标准（做到这句就算过关）

- 拿到一份 CSV 的列名和几行数据，能用 list/dict 存起来，能按条件筛出子集并排序。

今天的做法（顺序别颠倒）

- 本机 VSCode 里写这三段：
- 题库地址：file:///D:/DSH工作区/00-学习中心/60-练习环境/Python-Env/data
- 做完记一句：今天卡在哪、错在哪 → 写进 3-每日打卡。

今天看哪几集、读哪一章

这一天的知识：5 集 • 合计约 82 分钟（边看边在本机敲一遍）

P62	数据容器入门	约 5 分钟
P63	列表的定义语法	约 11 分钟
P65	列表的常用操作方法	约 30 分钟
P71	数据容器（序列）的切片	约 13 分钟
P75	字典的定义	约 23 分钟

教材（已下载到 20-教材与讲义\python-data\，断网也能看） https://raw.githubusercontent.com/jackfrued/Python-100-Days/master/Day01-20/08.常用数据结构之列表-1.md

视频地址就在下面的「今天做哪些题」网页里，点一下直达分P。

知识点 1/7 • list：有序、可变、什么都能装

- 一句话：列表用方括号，能追加、能改、能排序，下标从 0 开始。
- 代码：

```
rows = []
rows.append(['100001', '北京', '1899.00'])
rows.append(['100002', '上海', '699.00'])
print(len(rows))          # 2
print(rows[0])            # ['100001', '北京', '1899.00']
print(rows[-1])           # ['100002', '上海', '699.00']  ← 负下标从尾巴数
```

知识点 1/7 • list：有序、可变、什么都能装（续）

- rows[0][1] = '杭州'

```
print(rows[0][1])          # 杭州
```

★ 二维列表就是「表的行」：读 CSV 时最常见的结构就是 list[list[str]]。

- 但它没有列名 —— 想按列名取值就得用 dict（知识点 5）。

知识点 2/7 • 切片：左闭右开，第三个参数是步长

- 一句话：lst[a:b:c] 从 a 取到 b（不含 b），步长 c；省略就取到头。
- 代码：

```
nums = list(range(10))       # [0,1,2,3,4,5,6,7,8,9]
print(nums[2:5])             # [2, 3, 4]
print(nums[:3])              # [0, 1, 2]
print(nums[-3:])             # [7, 8, 9]
print(nums[::2])             # [0, 2, 4, 6, 8]
print(nums[::-1])            # 倒序
```

★ 切片永远不报 IndexError（越界自动截断），而 nums[99] 会报。

- 「取前 N 行」写 rows[:5] 最安全，写 rows[0:5] 也行，但别写 rows[5]。

知识点 3/7 • list 的常用方法与排序

- 一句话：append/pop/sort/reverse 是原地改；sorted 是返回新表、不改原表。
- 代码：

```
a = [3, 1, 2]
b = sorted(a)                  # 新表，a 不变
print(a, b)                    # [3, 1, 2] [1, 2, 3]
a.sort(reverse=True)           # 原地排，a 变了，返回 None
print(a)                       # [3, 2, 1]
rows = [['A', 1899.0], ['B', 699.0], ['C', 1299.0]]
rows.sort(key=lambda r: r[1], reverse=True)
print(rows)                    # 按第 2 列金额降序
```

知识点 3/7 • list 的常用方法与排序（续）

★ a = a.sort() 是最常见的错：sort() 返回 None，赋值完 a 就成了 None，

- 下一行 a[0] 直接 TypeError: 'NoneType' object is not subscriptable。

知识点 4/7 • tuple：不可变，用来「一次返回多个值」

- 一句话：元组用圆括号，建好就不能改；函数返回多值时它就是元组。
- 代码：

```
def min_max(nums):
return min(nums), max(nums)      # 返回的是元组
```

这一页只讲「tuple：不可变，用来「一次返回多个值」」。讲完就翻页，下一页是另一个知识点。

知识点 4/7 • tuple：不可变，用来「一次返回多个值」（续）

- lo, hi = min_max([3, 1, 4, 1, 5])　# 拆包

```
print(lo, hi)                       # 1 5
point = (3, 4)
# point[0] = 9                      # TypeError: 'tuple' object does not support item assignment
```

★ 只有一个元素的元组要写 (5,) —— 写 (5) 就是整数 5，不是元组。这个坑在给函数传「单参数元组」时会静默出错。

知识点 5/7 • dict：按名字取值，不用记下标

- 一句话：字典是「键 → 值」的映射，用 d['键'] 取，用 d.get('键', 默认值) 防 KeyError。
- 代码：

```
row = {'order_id': '100001', 'city': '北京', 'amount': '1899.00'}
print(row['city'])                          # 北京
print(row.get('status', '（无此列）'))      # （无此列）  ← 不报错
print(list(row.keys()))                     # ['order_id', 'city', 'amount']
for k, v in row.items():
print(k, '=', v)
```

★ row['status'] 在键不存在时抛 KeyError: 'status'。读脏数据时列名常常对不上（多个空格、BOM、大小写），一律先 .get() 再判断。

这一页只讲「dict：按名字取值，不用记下标」。讲完就翻页，下一页是另一个知识点。

知识点 6/7 • set：去重 + 集合运算

- 一句话：集合自动去重、没有顺序，能做交集/并集/差集。
- 代码：

```
cities = ['北京', '上海', '北京', '杭州', '上海']
print(set(cities))                      # {'北京', '上海', '杭州'}
print(len(set(cities)))                 # 3
```

这一页只讲「set：去重 + 集合运算」。讲完就翻页，下一页是另一个知识点。

知识点 6/7 • set：去重 + 集合运算（续）

- a, b = {'北京', '上海'}, {'上海', '广州'}

```
print(a & b)                    # 交集 {'上海'}
print(a | b)                    # 并集
print(a - b)                    # 差集 {'北京'}
```

- （本机 orders_dirty.csv 的 city 列去掉空值后有 9 个不同城市 ——用 len(set(...)) 就能验，第 9 天用 pandas 的 nunique() 会得到同一个数。）

知识点 7/7 • 把 CSV 读成「字典的列表」——清洗脚本的标准骨架

- 一句话：第一行当表头，后面每行按位置配成 {列名: 值}，这是 pandas 之前的通用写法。
- 代码：

```
import csv, pathlib
DATA = pathlib.Path(r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
with open(DATA / 'orders_dirty.csv', encoding='utf-8-sig', newline='') as f:
reader = csv.DictReader(f)
rows = list(reader)
print(len(rows), rows[0]['order_id'], rows[0]['city'])
```

★ 用 csv 模块而不是 split(',')：商品名里有逗号（'三只松鼠坚果礼盒, 大包装'）时split 会把一列劈成两列，整行错位；csv.DictReader 会正确处理引号包裹。

代码速查：今天这几段，要在本机跑通（1/5）

```
import csv, pathlib
```

```
DATA = pathlib.Path(r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
```

```
with open(DATA / 'orders_dirty.csv', encoding='utf-8-sig', newline='') as f:
```

rows = list(csv.DictReader(f))

1) 一共几行、列名有哪些

```
print(len(rows))
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（2/5）

```
print(list(rows[0].keys()))
```

```
# 2) 用 list 推导式筛出 city == '北京' 的行，取前 3 行打印
```

```
bj = [r for r in rows if r['city'] == '北京']
```

```
print(len(bj))
```

```
for r in bj[:3]:
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（3/5）

```
    print(r['order_id'], r['product_name'], r['amount'])
```

3) 用 set 统计有多少个不同城市（去掉空值）

```
cities = {r['city'] for r in rows if r['city']}
```

```
print(len(cities), sorted(cities))
```

4) 按金额降序排前 5（注意 amount 里有 '¥' 和逗号，先洗）

```
def to_num(s):
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（4/5）

```
    return float(s.replace('¥', '').replace(',', '')) if s else 0.0
```

```
top5 = sorted(rows, key=lambda r: to_num(r['amount']), reverse=True)[:5]
```

```
for r in top5:
```

print(r['order_id'], r['amount'])

验收（做到才算过关）：

① 第 3 步打印出的城市个数是 9；

代码速查：今天这几段，要在本机跑通（5/5）

```
② 第 4 步能正确排出前 5 名（'¥1,899.00' 这种不能被当成字符串比大小）；
```

③ 能说清为什么「按字符串排序」和「按数字排序」结果不同。

最容易错的地方

「取前 N 行」写 rows[:5] 最安全，写 rows[0:5] 也行，但别写 rows[5]。

★ a = a.sort() 是最常见的错：sort() 返回 None，赋值完 a 就成了 None，

★ 只有一个元素的元组要写 (5,) —— 写 (5) 就是整数 5，不是元组。这个坑在给函数传「单参数元组」时会静默出错。

★ 用 csv 模块而不是 split(',')：商品名里有逗号（'三只松鼠坚果礼盒, 大包装'）时split 会把一列劈成两列，整行错位；csv.DictReader 会正确处理引号包裹。

这些句子都是「差一个字、结果差一截」的地方，考前只翻这一页也值。

合上视频，自测一遍（能打勾才算学会）

- [ ] 我能不看资料写出：切片永远不报 IndexError（越界自动截断），而 nums[99] 会报。
- [ ] 我能不看资料写出：只有一个元素的元组要写 (5,) —— 写 (5) 就是整数 5，不是元组。这个坑在给函数传「单参数元组」时会静默出错。
- [ ] 我能不看资料说出：「取前 N 行」写 rows[:5] 最安全，写 rows[0:5] 也行，但别写 rows[5]。
- [ ] 我能不看资料说出：一句话：集合自动去重、没有顺序，能做交集/并集/差集。
- [ ] 我能不看题源，把今天用到的每个函数「为什么要有它」说一遍
- [ ] 我能说出今天每个参数/返回值改了会怎样——换个写法结果差在哪

这些句子都是「差一个字、结果差一截」的地方；三个勾以上，今天就算过关。

今天收工前，先知道明天学什么

第 4 天 • ★Excel 一天速成：数据透视表 + XLOOKUP/VLOOKUP + SUMIFS

不用提前学。明天点「① 开始今天的学习」，会换上明天那份 PPT。

收工三件事（5 分钟）

- 今天没跑通的脚本存进 90-我的作业\，命名 2026-11-04-第03天-*.py
- 打卡表「③ 每日打卡」记一笔：今天卡在哪、错在哪（一句话就够）
- 错题明天重做一遍，练习数据在 60-练习环境\Python-Env\data

全阶段 • 第 4 天 • 11/5

★Excel 一天速成：数据透视表 + XLOOKUPVLOOKUP + SUMIFS

能说清每一步在干什么。

第 4 天 · 11/5 · 今日知识点回顾

★Excel 一天速成：数据透视表 + XLOOKUP/VLOOKUP + SUMIFS

目标：15 分钟内把一张脏表做成透视汇总表 —— 透视表 + 至少一个查找函数，且能说清每一步在干什么。

对着 B 站视频看：视频讲到的每一句，这张 PPT 里都有一条；复习时只翻这份，不用再回去翻分P。

今天结束时，你应该能做到

验收标准（做到这句就算过关）

- 15 分钟内把一张脏表做成透视汇总表 —— 透视表 + 至少一个查找函数，且能说清每一步在干什么。

今天的做法（顺序别颠倒）

- 题库地址：file:///D:/DSH工作区/00-学习中心/60-练习环境/Python-Env/data
- （今天的练习表是 sales_dirty.xlsx；第 20 天还有一天 Excel 进阶，学 Power Query）
- 做完记一句：今天卡在哪、错在哪 → 写进 3-每日打卡。

今天看哪几集、读哪一章

这一天的知识：5 集 • 合计约 62 分钟（边看边在本机敲一遍）

P44	数据透视表的创建	约 15 分钟
P45	数据透视表的结构	约 12 分钟
P49	数据透视表的值汇总方式	约 12 分钟
P28	函数-SUM函数	约 8 分钟
P30	函数-SUMIFS函数	约 15 分钟

教材（已下载到 20-教材与讲义\python-data\，断网也能看） https://support.microsoft.com/zh-cn/office/创建数据透视表以分析工作表数据-0b9a4a4e-1a0b-4d6e-8c4e-1a1b1c1d1e1f

视频地址就在下面的「今天做哪些题」网页里，点一下直达分P。

知识点 1/7 • 透视表只做三件事：分组、汇总、交叉

- 一句话：行 = 按什么分组，列 = 按什么拆开，值 = 算什么（求和/计数/平均）。
- 操作：
- 选中数据区域任意单元格 → 插入 → 数据透视表 → 确定把「品类」拖到【行】，把「城市」拖到【列】，把「金额」拖到【值】

★ 透视表不会自动刷新：源数据改了必须右键 → 刷新（或 Alt+F5）。

- 忘了刷新是「数字对不上」最常见的原因，而且它不报错。

知识点 2/7 • 值汇总方式决定「求和还是计数」

- 一句话：默认是「求和」，但源列里有文本时 Excel 会自动变成「计数」。
- 操作：
- 值区域 → 右键 → 值字段设置 → 求和 / 计数 / 平均值 / 最大值

★ 金额列里混了 '¥1,899.00' 这种文本，透视表会把它算成 0 或直接计数 ——进透视表之前必须先把这列清成纯数字（分列 / 查找替换 / VALUE 函数）。

知识点 3/7 • XLOOKUP：查找函数的当前版本

- 一句话：=XLOOKUP(找什么, 在哪一列找, 返回哪一列, 找不到时返回什么)。
- 操作：
- =XLOOKUP(A2, 商品表!A:A, 商品表!D:D, "未找到")
- =XLOOKUP(A2&B2, 商品表!A:A&商品表!B:B, 商品表!D:D, "未找到") ← 多条件查找

★ XLOOKUP 的第 2、3 参数是「整列」，不用数第几列 —— 这是它比 VLOOKUP 强的核心。

- VLOOKUP 只能往右找（返回列必须在查找列右边），XLOOKUP 往左往右都行。

知识点 4/7 • VLOOKUP 仍然要会，因为面试还在问

- 一句话：=VLOOKUP(找什么, 区域, 返回第几列, 0)，最后的 0 表示精确匹配。
- 操作：
- =VLOOKUP(A2, 商品表!A:D, 4, 0)

★ 省略第 4 参数（写成 =VLOOKUP(A2,商品表!A:D,4)）是模糊匹配，会返回一个「看起来像但不对」的值，且不报错 —— 这是 Excel 最经典的静默错误。

知识点 5/7 • SUMIFS：多条件求和，参数顺序是「先求和列」

- 一句话：=SUMIFS(求和列, 条件列1, 条件1, 条件列2, 条件2, ...)。
- 操作：
- =SUMIFS(明细!H:H, 明细!C:C, "北京", 明细!D:D, "数码")

★ SUMIFS 和 SUMIF 的参数顺序相反**：SUMIF 是（条件列, 条件, 求和列），SUMIFS 是（求和列, 条件列, 条件）。抄错一次就会算出 0。**

知识点 6/7 • 条件格式：把「异常」标出来

- 一句话：开始 → 条件格式 → 突出显示单元格规则 / 数据条 / 色阶。
- 操作：
- 选中金额列 → 条件格式 → 数据条（一眼看出谁高谁低）
- 选中金额列 → 条件格式 → 突出显示 → 大于 → 5000 → 红色填充

★ 条件格式跟着「单元格值」走，不跟着「行」走。筛选后隐藏的行仍会被染色，打印前记得检查。

知识点 7/7 • 15 分钟做完的固定动作清单

- 一句话：清洗 → 透视 → 查找 → 条件格式，四步，每步都有时间预算。
- 操作：
- ① 0–3 分钟：全选 → 删除重复项；把金额列分列成数字；日期列统一格式② 3–8 分钟：插入透视表（行=品类，列=城市，值=金额求和）
- ③ 8–12 分钟：在旁边用 XLOOKUP 把「大区」贴进来④ 12–15 分钟：条件格式标出 Top 10%，加一行总计

- （本机 Excel 2026 已装：C:\Program Files\Microsoft Office\root\Office16\EXCEL.EXE。
- 今天的练习表在 data\sales_dirty.xlsx，3 个 sheet：明细 400 行 / 商品表 15 行 / 城市表 9 行。）

代码速查：今天这几段，要在本机跑通（1/2）

打开 data\sales_dirty.xlsx，按下面顺序做，全程计时 15 分钟：

① 明细 sheet：删除重复订单号 → 金额列分列成数字 → 日期列统一成 yyyy-mm-dd

② 插入透视表：行=品类、列=城市、值=金额求和，放到新 sheet 命名「汇总」

③ 在「汇总」右侧用 XLOOKUP 把城市表里的大区贴进来（城市是键）

④ 用 SUMIFS 在明细下方算出「北京 + 数码」的合计金额，和透视表对应格子核对

⑤ 条件格式：金额列加数据条；大于 5000 的标红

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（2/2）

验收（做到才算过关）：

① 15 分钟内做出「汇总」表，行列都对得上；

② XLOOKUP 和 SUMIFS 各写出一条公式，并能讲清每个参数是什么；

③ 能说出透视表不刷新会导致什么后果（举一个具体场景）。

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

最容易错的地方

忘了刷新是「数字对不上」最常见的原因，而且它不报错。

★ 省略第 4 参数（写成 =VLOOKUP(A2,商品表!A:D,4)）是模糊匹配，会返回一个「看起来像但不对」的值，且不报错 —— 这是 Excel 最经典的静默错误。

★ SUMIFS 和 SUMIF 的参数顺序**相反**：SUMIF 是（条件列, 条件, 求和列），SUMIFS 是（求和列, 条件列, 条件）。抄错一次就会算出 0。

这些句子都是「差一个字、结果差一截」的地方，考前只翻这一页也值。

合上视频，自测一遍（能打勾才算学会）

- [ ] 我能不看资料写出：XLOOKUP 的第 2、3 参数是「整列」，不用数第几列 —— 这是它比 VLOOKUP 强的核心。
- [ ] 我能不看资料写出：条件格式跟着「单元格值」走，不跟着「行」走。筛选后隐藏的行仍会被染色，打印前记得检查。
- [ ] 我能不看资料说出：VLOOKUP 只能往右找（返回列必须在查找列右边），XLOOKUP 往左往右都行。
- [ ] 我能不看题源，把今天用到的每个函数「为什么要有它」说一遍
- [ ] 我能说出今天每个参数/返回值改了会怎样——换个写法结果差在哪
- [ ] 今天这几段代码我在本机都跑通、并看清了输出

这些句子都是「差一个字、结果差一截」的地方；三个勾以上，今天就算过关。

今天收工前，先知道明天学什么

第 5 天 • 函数：参数、返回值、作用域、lambda

不用提前学。明天点「① 开始今天的学习」，会换上明天那份 PPT。

收工三件事（5 分钟）

- 今天没跑通的脚本存进 90-我的作业\，命名 2026-11-05-第04天-*.py
- 打卡表「③ 每日打卡」记一笔：今天卡在哪、错在哪（一句话就够）
- 错题明天重做一遍，练习数据在 60-练习环境\Python-Env\data

全阶段 • 第 5 天 • 11/6

函数：参数、返回值、作用域、lambda

能说清参数是「值传递」还是「引用传递」。

第 5 天 · 11/6 · 今日知识点回顾

函数：参数、返回值、作用域、lambda

目标：把一段 30 行的流水账脚本拆成 4 个函数，每个函数只干一件事，且能说清参数是「值传递」还是「引用传递」。

对着 B 站视频看：视频讲到的每一句，这张 PPT 里都有一条；复习时只翻这份，不用再回去翻分P。

今天结束时，你应该能做到

验收标准（做到这句就算过关）

- 把一段 30 行的流水账脚本拆成 4 个函数，每个函数只干一件事，且能说清参数是「值传递」还是「引用传递」。

今天的做法（顺序别颠倒）

- 题库地址：file:///D:/DSH工作区/00-学习中心/60-练习环境/Python-Env/data
- 做完记一句：今天卡在哪、错在哪 → 写进 3-每日打卡。

今天看哪几集、读哪一章

这一天的知识：6 集 · 合计约 58 分钟（边看边在本机敲一遍）

P51	函数的初体验	约 12 分钟
P52	函数的基础定义语法	约 9 分钟
P54	函数的传入参数	约 10 分钟
P56	函数的返回值定义语法	约 8 分钟
P60	变量在函数中的作用域	约 10 分钟
P84	lambda匿名函数	约 9 分钟

教材（已下载到 20-教材与讲义\python-data\，断网也能看） https://raw.githubusercontent.com/jackfrued/Python-100-Days/master/Day01-20/14.函数和模块.md

视频地址就在下面的「今天做哪些题」网页里，点一下直达分P。

知识点 1/6 • def 定义，return 返回

- 一句话：def 名字(参数): 下面缩进写函数体；return 把结果交出去，没写 return 就返回 None。
- 代码：

```
def bmi(height_m, weight_kg):
return weight_kg / height_m ** 2
print(bmi(1.75, 70))        # 22.857142857142858
print(round(bmi(1.75, 70), 2))   # 22.86
print(bmi(weight_kg=70, height_m=1.75))   # 22.857... ← 关键字参数，顺序无所谓
```

★ 忘了 return 是最隐蔽的 bug：函数体里 print 了结果，但函数本身返回 None，外面 a = f() 之后 a 就是 None，后面 a + 1 报 TypeError。

知识点 2/6 • 默认参数与可变参数

- 一句话：默认参数写 def f(a, b=10)；可变参数写 *args（元组）和 **kwargs（字典）。
- 代码：

```
def to_num(s, default=0.0):
try:
return float(str(s).replace('¥', '').replace(',', ''))
except (TypeError, ValueError):
return default
print(to_num('¥1,899.00'))   # 1899.0
print(to_num(''))            # 0.0
print(to_num(None, -1))      # -1
```

知识点 2/6 • 默认参数与可变参数（续）

★ 默认参数只在「定义时」求值一次。def f(x, bucket=[]) 里的 bucket 会被所有调用

- 共享，追加几次就串了 —— 要用 None 兜：def f(x, bucket=None): bucket = bucket or []

知识点 3/6 • 作用域：函数里改不了外面的数字，但能改列表

- 一句话：函数内赋值会新建局部变量；global 才能改全局，但尽量别用。
- 代码：

```
counter = 0
def bump():
global counter
```

- counter += 1bump(); bump()

知识点 3/6 • 作用域：函数里改不了外面的数字，但能改列表（续）

```
print(counter)          # 2
nums = [1, 2, 3]
def add(x):
x.append(4)         # 没有赋值，直接改对象 → 外面也变
add(nums)
print(nums)             # [1, 2, 3, 4]
```

★ 「值传递 / 引用传递」这个面试题的正确答法：Python 传的是「对象引用」。

- 函数内重新赋值不影响外面，原地修改（append / sort / [0]=）会影响外面。

知识点 4/6 • lambda：一行的匿名函数，专门喂给 key

- 一句话：lambda 参数: 表达式，通常只用在 sorted / max / min / map / filter 的 key 上。
- 代码：

```
rows = [{'city': '北京', 'amount': 1899.0},
```

- {'city': '上海', 'amount': 699.0}]rows.sort(key=lambda r: r['amount'], reverse=True)

知识点 4/6 • lambda：一行的匿名函数，专门喂给 key（续）

```
print([r['city'] for r in rows])          # ['北京', '上海']
print(max(rows, key=lambda r: r['amount'])['city'])   # 北京
```

> **★ lambda 里不能写语句（不能 print、不能赋值、不能 if-else 的多分支），只能是一个表达式。超过一行的逻辑就老老实实写 def。**

知识点 5/6 • map / filter / 列表推导式，选一个就行

- 一句话：三者都能做转换和筛选，团队代码里推导式最易读，map/filter 次之。
- 代码：

```
nums = [1, 2, 3, 4, 5]
print([n * 2 for n in nums])                # 推导式（推荐）
print(list(map(lambda n: n * 2, nums)))     # map
print([n for n in nums if n % 2 == 0])      # 筛选
print(list(filter(lambda n: n % 2 == 0, nums)))
```

★ map/filter 返回的是迭代器，不是列表 —— 直接 print 会得到 <map object at 0x...>，要么套 list()，要么改成推导式。这个坑在 Python 3 里天天有人踩。

知识点 6/6 • 写函数的三条硬规矩

- 一句话：一个函数只干一件事；参数不超过 4 个；有副作用（读写文件/打印）要写进名字。
- 代码：

```
def load_orders(path): ...          # 读文件
def clean_amount(rows): ...         # 清洗
def summarize_by_city(rows): ...    # 汇总
def save_csv(rows, path): ...       # 写出
```

- （这四步就是第 14 天 W5 验收脚本的骨架。今天先按这个结构把昨天的练习拆成函数，明天开始每个函数单独能测。）

代码速查：今天这几段，要在本机跑通（1/5）

把第 3 天的练习脚本重写成 4 个函数，每个函数单独能跑：

```
import csv, pathlib
```

```
DATA = pathlib.Path(r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
```

```
def load_orders(path):
```

```
    with open(path, encoding='utf-8-sig', newline='') as f:
```

return list(csv.DictReader(f))

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（2/5）

```
def to_num(s, default=0.0):
```

```
    try:
```

```
        return float(str(s).replace('¥', '').replace(',', ''))
```

```
    except (TypeError, ValueError):
```

```
        return default
```

```
def summarize_by_city(rows, status='paid'):
```

```
    agg = {}
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（3/5）

for r in rows:

```
        if r['status'].strip().lower() != status:
```

continue

city = r['city'] or '（未知）'

```
        agg[city] = agg.get(city, 0.0) + to_num(r['amount'])
```

```
    return dict(sorted(agg.items(), key=lambda kv: kv[1], reverse=True))
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（4/5）

```
if __name__ == '__main__':
```

```
    rows = load_orders(DATA / 'orders_dirty.csv')
```

print(len(rows), '行')

```
    for city, total in list(summarize_by_city(rows).items())[:5]:
```

print(f'{city}\t{total:,.2f}')

验收（做到才算过关）：

① 四个函数各自能单独调用、单独验证；

代码速查：今天这几段，要在本机跑通（5/5）

② `if __name__ == '__main__':` 那一行的作用能讲清楚（模块被 import 时不执行）；

③ 把 summarize_by_city 改成按 category 汇总，只改一行参数。

最容易错的地方

共享，追加几次就串了 —— 要用 None 兜：def f(x, bucket=None): bucket = bucket or []◆ 知识点 3 | 作用域：函数里改不了外面的数字，但能改列表一句话：函数内赋值会新建局部变量；global 才能改全局，但尽量别用。

这些句子都是「差一个字、结果差一截」的地方，考前只翻这一页也值。

合上视频，自测一遍（能打勾才算学会）

- [ ] 我能不看资料写出：lambda 里不能写语句（不能 print、不能赋值、不能 if-else 的多分支），只能是一个表达式。超过一行的逻辑就老老实实写 def。
- [ ] 我能不看题源，把今天用到的每个函数「为什么要有它」说一遍
- [ ] 我能说出今天每个参数/返回值改了会怎样——换个写法结果差在哪
- [ ] 今天这几段代码我在本机都跑通、并看清了输出

这些句子都是「差一个字、结果差一截」的地方；三个勾以上，今天就算过关。

今天收工前，先知道明天学什么

第 6 天 • 面向对象 + 模块与包 + 常用标准库

不用提前学。明天点「① 开始今天的学习」，会换上明天那份 PPT。

收工三件事（5 分钟）

- 今天没跑通的脚本存进 90-我的作业\，命名 2026-11-06-第05天-*.py
- 打卡表「③ 每日打卡」记一笔：今天卡在哪、错在哪（一句话就够）
- 错题明天重做一遍，练习数据在 60-练习环境\Python-Env\data

全阶段 · 第 6 天 · 11/7

面向对象 + 模块与包 + 常用标准库

能说清 import 的三种写法差别。

第 6 天 · 11/7 · 今日知识点回顾

面向对象 + 模块与包 + 常用标准库

目标：写一个带 __init__ 和 __repr__ 的类，把昨天的 4 个函数收进一个类里；能说清 import 的三种写法差别。

对着 B 站视频看：视频讲到的每一句，这张 PPT 里都有一条；复习时只翻这份，不用再回去翻分P。

今天结束时，你应该能做到

验收标准（做到这句就算过关）

- 写一个带 __init__ 和 __repr__ 的类，把昨天的 4 个函数收进一个类里；能说清 import 的三种写法差别。

今天的做法（顺序别颠倒）

- 题库地址：file:///D:/DSH工作区/00-学习中心/60-练习环境/Python-Env/data
- 做完记一句：今天卡在哪、错在哪 → 写进 3-每日打卡。

今天看哪几集、读哪一章

这一天的知识：5 集 • 合计约 73 分钟（边看边在本机敲一遍）

P111	初识对象	约 10 分钟
P112	类的成员方法	约 18 分钟
P114	构造方法	约 12 分钟
P115	魔术方法	约 15 分钟
P94	模块的概念和导入	约 18 分钟

教材（已下载到 20-教材与讲义\python-data\，断网也能看） https://raw.githubusercontent.com/jackfrued/Python-100-Days/master/Day01-20/18.面向对象编程入门.md

视频地址就在下面的「今天做哪些题」网页里，点一下直达分P。

知识点 1/7 • 类 = 模板，对象 = 按模板造出来的东西

- 一句话：class 定义模板，__init__ 在造对象时自动跑，self 指「这个对象自己」。
- 代码：

```
class Order:
def __init__(self, order_id, city, amount):
```

- self.order_id = order_idself.city = cityself.amount = amount

这一页只讲「类 = 模板，对象 = 按模板造出来的东西」。讲完就翻页，下一页是另一个知识点。

知识点 1/7 • 类 = 模板，对象 = 按模板造出来的东西（续）

```
o = Order('100001', '北京', 1899.0)
print(o.city)          # 北京
print(o.amount)        # 1899.0
```

★ self 不能省，也不能换成别的名字（虽然语法允许）。类里每个方法的第一个参数都是 self，调用时不传 —— o.city 会自动变成 Order.city(o)。

知识点 2/7 • __repr__ 决定「print 出来长什么样」

- 一句话：没写 __repr__ 时 print(o) 打印 <__main__.Order object at 0x...>，看不出内容。
- 代码：

```
class Order:
def __init__(self, order_id, city, amount):
```

- self.order_id, self.city, self.amount = order_id, city, amount

知识点 2/7 • __repr__ 决定「print 出来长什么样」（续）

```
def __repr__(self):
return f'Order({self.order_id}, {self.city}, {self.amount:.2f})'
print(Order('100001', '北京', 1899.0))     # Order(100001, 北京, 1899.00)
```

★ 调试时 90% 的时间花在「看对象里有什么」。__repr__ 写好了，print 一次就够；

- 不写，就得 print(o.__dict__) 或进调试器。

知识点 3/7 • 方法：把「针对这个对象的操作」写进类里

- 一句话：方法就是第一个参数是 self 的函数，能直接访问 self.xxx。
- 代码：

```
class Order:
def __init__(self, order_id, city, amount):
```

- self.order_id, self.city, self.amount = order_id, city, amount

知识点 3/7 · 方法：把「针对这个对象的操作」写进类里（续）

```
def is_big(self, threshold=1000):
return self.amount >= threshold
def discounted(self, rate=0.9):
return round(self.amount * rate, 2)
o = Order('100001', '北京', 1899.0)
print(o.is_big(), o.discounted())      # True 1709.1
```

★ 类里的 amount 用 float 存，别存 '¥1,899.00' 字符串 —— 清洗放在构造之前（第 5 天的 to_num 就是干这个的），类里只放干净数据。

知识点 4/7 • 继承：把公共部分提到父类

- 一句话：class 子类(父类): 自动拥有父类的属性和方法，需要改就覆写。
- 代码：

```
class Base:
def describe(self):
return f'{type(self).__name__}: {self.__dict__}'
class PaidOrder(Base):
def __init__(self, oid, amount):
```

知识点 4/7 • 继承：把公共部分提到父类（续）

- self.oid, self.amount = oid, amount

```
print(PaidOrder('100001', 1899.0).describe())
```

★ 数据分析岗的代码里继承用得很少 —— 别为了「用上继承」而硬造类层级。

- 真正高频的是「一个类装一份数据 + 几个方法算指标」，今天写出来就够。

知识点 5/7 • import 的三种写法，差别在「名字进不进当前命名空间」

- 一句话：import x / from x import y / import x as z，优先用第一种或第三种。
- 代码：

```
import pathlib                  # 用 pathlib.Path
from pathlib import Path        # 直接用 Path
import numpy as np              # 惯例缩写
import pandas as pd
```

★ from module import * 会把模块里所有公开名字倒进来，容易和你的变量撞名，而且看不出某个函数是哪来的 —— 项目里禁止用（PEP 8 明确不建议）。

这一页只讲「import 的三种写法，差别在「名字进不进当前命名空间」」。讲完就翻页，下一页是另一个知识点。

知识点 6/7 • 四个天天要用的标准库

- 一句话：pathlib 管路径、datetime 管日期、collections 管计数、json 管序列化。
- 代码：

```
import pathlib, datetime as dt, collections, json
DATA = pathlib.Path(r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
print(DATA.exists(), (DATA / 'orders_dirty.csv').stat().st_size)
print(dt.date(2026, 11, 7).strftime('%Y-%m-%d %A'))
c = collections.Counter(['北京', '上海', '北京'])
print(c.most_common())            # [('北京', 2), ('上海', 1)]
print(json.dumps({'city': '北京'}, ensure_ascii=False))   # {"city": "北京"}
```

★ json.dumps 默认 ensure_ascii=True，中文会变成 \u5317\u4eac。写文件给人看时一定要 ensure_ascii=False，否则打开是一堆转义。

知识点 7/7 • 把函数收进类：什么时候值得

- 一句话：一份数据要反复算多个指标、或要带着状态（清洗前后的行数）时，才值得建类。
- 代码：

```
class Orders:
def __init__(self, rows):
```

- self.rows = rowsself.dropped = 0

```
def filter_paid(self):
before = len(self.rows)
```

知识点 7/7 • 把函数收进类：什么时候值得（续）

- self.rows = [r for r in self.rows if r['status'].strip().lower() == 'paid']self.dropped += before - len(self.rows)

```
return self
```

（self.dropped 这种「顺手记一笔」的状态，是类比纯函数好用的地方：

- 纯函数每次都要把计数传进去再传出来。）

代码速查：今天这几段，要在本机跑通（1/5）

把第 5 天的四个函数收进一个类，并加两个方法：

```
import csv, pathlib
```

```
DATA = pathlib.Path(r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
```

```
class Orders:
```

```
    def __init__(self, rows):
        self.rows = rows
        self.log = []
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（2/5）

def __repr__(self):

```
        return f'Orders({len(self.rows)} 行，已丢弃 {sum(self.log) if self.log else 0})'
```

```
    def filter_status(self, status='paid'):
```

before = len(self.rows)

```
        self.rows = [r for r in self.rows if r['status'].strip().lower() == status]
```

```
        self.log.append(before - len(self.rows))
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（3/5）

```
        return self
    def sum_by(self, col='city'):
        agg = {}
        for r in self.rows:
            k = r[col] or '（未知）'
```

```
            v = float(r['amount'].replace('¥', '').replace(',', '') or 0)
```

```
            agg[k] = agg.get(k, 0.0) + v
```

```
        return dict(sorted(agg.items(), key=lambda kv: kv[1], reverse=True))
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（4/5）

```
if __name__ == '__main__':
```

```
    with open(DATA / 'orders_dirty.csv', encoding='utf-8-sig', newline='') as f:
```

```
    rows = list(csv.DictReader(f))
o = Orders(rows)
print(o)
print(o.filter_status('paid'))
top = o.sum_by('city')
for k, v in list(top.items())[:3]:
    print(f'{k}\t{v:,.2f}')
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（5/5）

验收（做到才算过关）：

```
① print(o) 打印的是 Orders(1200 行, 已丢弃 0), 不是 <object at 0x...>;
```

② filter_status 之后 print(o) 能看出丢了多少行；

```
③ 能说出「为什么 sum_by 里对 amount 又要洗一遍」— 因为第 3 天没洗,
```

原始 rows 里的 amount 还带着 ¥ 和逗号。

最容易错的地方

★ self 不能省，也不能换成别的名字（虽然语法允许）。类里每个方法的第一个参数都是 self，调用时不传 —— o.city 会自动变成 Order.city(o)。

★ 类里的 amount 用 float 存，别存 '¥1,899.00' 字符串 —— 清洗放在构造之前（第 5 天的 to_num 就是干这个的），类里只放干净数据。

★ 数据分析岗的代码里继承用得很少 —— 别为了「用上继承」而硬造类层级。

这些句子都是「差一个字、结果差一截」的地方，考前只翻这一页也值。

合上视频，自测一遍（能打勾才算学会）

- [ ] 我能不看资料写出：数据分析岗的代码里继承用得很少 —— 别为了「用上继承」而硬造类层级。
- [ ] 我能不看题源，把今天用到的每个函数「为什么要有它」说一遍
- [ ] 我能说出今天每个参数/返回值改了会怎样——换个写法结果差在哪
- [ ] 今天这几段代码我在本机都跑通、并看清了输出

这些句子都是「差一个字、结果差一截」的地方；三个勾以上，今天就算过关。

今天收工前，先知道明天学什么

第 7 天 • 文件 IO + 异常处理 + CSV/JSON 读写（W4 收尾验收）

不用提前学。明天点「① 开始今天的学习」，会换上明天那份 PPT。

收工三件事（5 分钟）

- 今天没跑通的脚本存进 90-我的作业\，命名 2026-11-07-第06天-*.py
- 打卡表「③ 每日打卡」记一笔：今天卡在哪、错在哪（一句话就够）
- 错题明天重做一遍，练习数据在 60-练习环境\Python-Env\data

全阶段 • 第 7 天 • 11/8

文件 IO + 异常处理 + CSVJSON 读写（W4 收尾验收）

写一个脚本：读脏 CSV → 清洗 → 统计 → 写出干净 CSV + 一份 JSON 摘要，全程用 try/except 兜住，中途出错不崩。

第 7 天 · 11/8 · 今日知识点回顾

文件 IO + 异常处理 + CSV/JSON 读写（W4 收尾验收）

目标：写一个脚本：读脏 CSV → 清洗 → 统计 → 写出干净 CSV + 一份 JSON 摘要，全程用 try/except 兜住，中途出错不崩。

对着 B 站视频看：视频讲到的每一句，这张 PPT 里都有一条；复习时只翻这份，不用再回去翻分P。

今天结束时，你应该能做到

验收标准（做到这句就算过关）

- 写一个脚本：读脏 CSV → 清洗 → 统计 → 写出干净 CSV + 一份 JSON 摘要，全程用 try/except 兜住，中途出错不崩。

今天的做法（顺序别颠倒）

- 题库地址：file:///D:/DSH工作区/00-学习中心/60-练习环境/Python-Env/data
- 做完记一句：今天卡在哪、错在哪 → 写进 3-每日打卡。

今天看哪几集、读哪一章

这一天的知识：5 集 · 合计约 79 分钟（边看边在本机敲一遍）

P86	文件的读取操作	约 28 分钟
P88	文件的写出操作	约 10 分钟
P91	了解异常	约 5 分钟
P92	异常的捕获	约 21 分钟
P100	JSON数据格式的转换	约 15 分钟

教材（已下载到 20-教材与讲义\python-data\，断网也能看） https://raw.githubusercontent.com/jackfrued/Python-100-Days/master/Day21-30/21.文件读写和异常处理.md

视频地址就在下面的「今天做哪些题」网页里，点一下直达分P。

知识点 1/7 • with open 是唯一正确写法

- 一句话：with 会在块结束时自动关文件，不用手写 f.close()。
- 代码：

```
import pathlib
DATA = pathlib.Path(r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
with open(DATA / 'orders_dirty.csv', encoding='utf-8-sig', newline='') as f:
head = f.readline()
print(head.strip()[:60])
```

★ 三个参数缺一不可：encoding='utf-8-sig' 吃掉 BOM；newline='' 让 csv 模块自己处理换行（不写的话 Windows 上会出现空行翻倍）。模式 'r' 读、'w' 覆盖写、'a' 追加。

知识点 2/7 • 'w' 会瞬间清空文件

- 一句话：open(path, 'w') 一打开就把原文件清成 0 字节，写失败也回不来。
- 代码：

```
out = DATA / '_tmp_test.csv'
with open(out, 'w', encoding='utf-8-sig', newline='') as f:
f.write('a,b\n1,2\n')
print(out.exists(), out.stat().st_size)     # True 8
out.unlink()                                 # 用完删掉
```

★ 往已有文件里补内容要用 'a'；要「读改写」就写新文件再替换，别直接 'w' 覆盖 ——这一步出错，原数据就没了。

知识点 3/7 • try/except：只兜你预期的错

- 一句话：except 后面写具体异常类型，兜不住的错误要让它照常报出来。
- 代码：

```
def to_num(s, default=0.0):
try:
return float(str(s).replace('¥', '').replace(',', ''))
except (TypeError, ValueError) as e:
return default
print(to_num('¥1,899.00'), to_num(''), to_num(None))
```

★ 裸 except: 会把 KeyboardInterrupt（Ctrl+C）和语法错误一起吞掉，程序该停不停 ——永远写具体类型，至少写 except Exception。

这一页只讲「try/except：只兜你预期的错」。讲完就翻页，下一页是另一个知识点。

知识点 4/7 • 文件不存在、编码不对，各报什么错

- 一句话：FileNotFoundError 是路径错；UnicodeDecodeError 是编码错；KeyError 是列名错。
- 代码：

```
try:
open(DATA / '不存在.csv', encoding='utf-8')
except FileNotFoundError as e:
print('路径错：', e)
try:
open(DATA / 'orders_dirty.csv', encoding='utf-8').readline()
except UnicodeDecodeError as e:
print('编码错，要加 -sig：', str(e)[:60])
```

知识点 4/7 • 文件不存在、编码不对，各报什么错（续）

- （本机实测：不带 utf-8-sig 读 orders_dirty.csv 会报
- UnicodeDecodeError: 'utf-8' codec can't decode byte 0xef in position 0 —— 0xef 就是 BOM 的 EF BB BF。）

知识点 5/7 • csv.DictWriter：按表头写出

- 一句话：先 writer.writeheader() 写表头，再 writer.writerow(dict) 一行行写。
- 代码：

```
import csv
out = DATA / '_clean_sample.csv'
with open(out, 'w', encoding='utf-8-sig', newline='') as f:
w = csv.DictWriter(f, fieldnames=['order_id', 'city', 'amount'])
w.writeheader()
w.writerows([{'order_id': '100001', 'city': '北京', 'amount': 1899.0}])
print(out.stat().st_size)
out.unlink()
```

知识点 5/7 • csv.DictWriter：按表头写出（续）

★ fieldnames 里的列名必须和 dict 的键完全一致，否则报 ValueError:

- dict contains fields not in fieldnames。多出来的键用 extrasaction='ignore' 忽略。

知识点 6/7 • JSON：把统计结果存成「人能读、程序能吃」的格式

- 一句话：json.dump(obj, f, ensure_ascii=False, indent=2)。
- 代码：

```
import json
summary = {'rows': 1200, 'paid_rows': 417, 'paid_sum': 422436.0,
```

- 'top_city': {'南京': 60532.0}}

知识点 6/7 • JSON：把统计结果存成「人能读、程序能吃」的格式（续）

```
with open(DATA / '_summary.json', 'w', encoding='utf-8') as f:
json.dump(summary, f, ensure_ascii=False, indent=2)
print((DATA / '_summary.json').read_text(encoding='utf-8')[:80])
(DATA / '_summary.json').unlink()
```

★ JSON 不支持 datetime / Decimal / numpy 类型 —— 直接 dump 会报

- TypeError: Object of type Timestamp is not JSON serializable。
- 先转成字符串或 float（后面 pandas 阶段天天遇到）。

知识点 7/7 • W4 验收脚本的完整骨架

- 一句话：load → clean → summarize → save，四段各一个函数，main 里只做编排。
- 代码：

```
def main():
rows = load_orders(DATA / 'orders_dirty.csv')      # 读
```

- clean, report = clean_orders(rows)   # 洗 + 记清洗账

知识点 7/7 • W4 验收脚本的完整骨架（续）

```
summary = summarize(clean)                          # 汇总
save_csv(clean, DATA / 'orders_clean.csv')          # 写干净数据
save_json({**report, **summary}, DATA / 'orders_summary.json')
print(f'清洗后 {len(clean)} 行，摘要已写出')
```

- （这份 orders_clean.csv / orders_summary.json 就是 W4 的交付物，W5 直接拿来用 ——今天写不好，后面 7 天都在脏数据上打转。）

代码速查：今天这几段，要在本机跑通（1/9）

写一个 orders_clean.py，跑出 orders_clean.csv + orders_summary.json：

```
import csv, json, pathlib
```

```
DATA = pathlib.Path(r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
```

```
def to_num(s, default=None):
```

try:

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（2/9）

```
        return float(str(s).replace('¥', '').replace(',', ''))
```

except (TypeError, ValueError):

return default

```
def load_orders(path):
```

```
    with open(path, encoding='utf-8-sig', newline='') as f:
```

return list(csv.DictReader(f))

代码速查：今天这几段，要在本机跑通（3/9）

```
def clean_orders(rows):
```

```
    seen, clean = set(), []
```

```
    dropped = {'dup': 0, 'bad_amount': 0, 'bad_qty': 0}
```

```
    for r in rows:
        oid = r['order_id']
        if oid in seen:
            dropped['dup'] += 1
            continue
        seen.add(oid)
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（4/9）

```
amt = to_num(r['amount'])
if amt is None:
    dropped['bad_amount'] += 1
    continue
qty = to_num(r['quantity'])
if qty is None or qty <= 0:
    dropped['bad_qty'] += 1
    continue
r['amount'] = round(amt, 2)
r['quantity'] = int(qty)
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（5/9）

```
        r['status'] = (r['status'] or '').strip().lower()
```

```
        r['city'] = (r['city'] or '').strip() or '（未知）'
```

clean.append(r)

return clean, dropped

```
def summarize(rows):
```

```
    paid = [r for r in rows if r['status'] == 'paid']
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（6/9）

```
    return {'rows_clean': len(rows), 'paid_rows': len(paid),
```

```
            'paid_sum': round(sum(r['amount'] for r in paid), 2),
```

```
            'cities': len({r['city'] for r in rows})}
```

```
def save_csv(rows, path):
```

```
    with open(path, 'w', encoding='utf-8-sig', newline='') as f:
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（7/9）

```
        w = csv.DictWriter(f, fieldnames=list(rows[0].keys()))
```

w.writeheader()

w.writerows(rows)

```
def save_json(obj, path):
```

```
    with open(path, 'w', encoding='utf-8') as f:
```

```
        json.dump(obj, f, ensure_ascii=False, indent=2)
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（8/9）

```
if __name__ == '__main__':
```

```
    rows = load_orders(DATA / 'orders_dirty.csv')
```

clean, dropped = clean_orders(rows)

s = summarize(clean)

```
    save_csv(clean, DATA / 'orders_clean.csv')
```

```
    save_json({**dropped, **s}, DATA / 'orders_summary.json')
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（9/9）

```
print(dropped, s)
```

验收（做到才算过关，这是 W4 的总验收）：

```
① 脚本一次跑通，data 目录下多出 orders_clean.csv 和 orders_summary.json；
```

② 打印出来的 dropped 和 s 里每个数字，你都能说出它是怎么来的；

```
③ 把 clean_orders 里删掉一行 r['city'] = ...，再跑一遍，说清为什么 cities 变了；
```

```
④ 能讲清 utf-8-sig 和 utf-8 的差别（用 0xef 那个报错讲）。
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

最容易错的地方

★ 往已有文件里补内容要用 'a'；要「读改写」就写新文件再替换，别直接 'w' 覆盖 ——这一步出错，原数据就没了。

★ 裸 except: 会把 KeyboardInterrupt（Ctrl+C）和语法错误一起吞掉，程序该停不停 ——永远写具体类型，至少写 except Exception。

这些句子都是「差一个字、结果差一截」的地方，考前只翻这一页也值。

合上视频，自测一遍（能打勾才算学会）

- [ ] 我能不看资料写出：往已有文件里补内容要用 'a'；要「读改写」就写新文件再替换，别直接 'w' 覆盖 ——这一步出错，原数据就没了。
- [ ] 我能不看资料写出：JSON 不支持 datetime / Decimal / numpy 类型 —— 直接 dump 会报
- [ ] 我能不看题源，把今天用到的每个函数「为什么要有它」说一遍
- [ ] 我能说出今天每个参数/返回值改了会怎样——换个写法结果差在哪
- [ ] 今天这几段代码我在本机都跑通、并看清了输出

这些句子都是「差一个字、结果差一截」的地方；三个勾以上，今天就算过关。

今天收工前，先知道明天学什么

第 8 天 • numpy：ndarray、向量化、广播、布尔索引

不用提前学。明天点「① 开始今天的学习」，会换上明天那份 PPT。

收工三件事（5 分钟）

- 今天没跑通的脚本存进 90-我的作业\，命名 2026-11-08-第07天-*.py
- 打卡表「③ 每日打卡」记一笔：今天卡在哪、错在哪（一句话就够）
- 错题明天重做一遍，练习数据在 60-练习环境\Python-Env\data

全阶段 · 第 8 天 · 11/9

numpy：ndarray、向量化、广播、布尔索引

不用 for 循环，把一份 1200 行的金额列一次性做完「解析 + 判空 + 缩放 + 条件筛选」。

第 8 天 · 11/9 · 今日知识点回顾

numpy：ndarray、向量化、广播、布尔索引

目标：不用 for 循环，把一份 1200 行的金额列一次性做完「解析 + 判空 + 缩放 + 条件筛选」。

对着 B 站视频看：视频讲到的每一句，这张 PPT 里都有一条；复习时只翻这份，不用再回去翻分P。

今天结束时，你应该能做到

验收标准（做到这句就算过关）

- 不用 for 循环，把一份 1200 行的金额列一次性做完「解析 + 判空 + 缩放 + 条件筛选」。

今天的做法（顺序别颠倒）

- 题库地址：file:///D:/DSH工作区/00-学习中心/60-练习环境/Python-Env/data
- 做完记一句：今天卡在哪、错在哪 → 写进 3-每日打卡。

今天看哪几集、读哪一章

这一天的知识：4 集 • 合计约 97 分钟（边看边在本机敲一遍）

P14	numpy的数组的创建	约 16 分钟
P15	数组的计算和数组的计算	约 42 分钟
P17	numpy中的索引和切片	约 18 分钟
P21	numpy中的nan和常用统计方法	约 21 分钟

教材（已下载到 20-教材与讲义\python-data\，断网也能看）

`https://raw.githubusercontent.com/datawhalechina/joyful-pandas/master/source/Content/ch1.rst`

第一章 预备知识：列表推导式、匿名函数、numpy 基础

视频地址就在下面的「今天做哪些题」网页里，点一下直达分P。

知识点 1/7 • ndarray 是「同类型 + 定长」的数组，这是它快的原因

- 一句话：一个 ndarray 里所有元素类型相同，内存连续，所以整列运算能一次性算完。
- 代码：

```
import numpy as np
a = np.array([1, 2, 3, 4])
print(a.dtype, a.shape, a.ndim)      # int64 (4,) 1
b = np.array([[1, 2, 3], [4, 5, 6]])
print(b.shape)                        # (2, 3)  2 行 3 列
print(b.dtype)
```

★ np.array([1, 2, 3.5]) 会整体变成 float64 —— 一个 float 就把全数组升成 float。

- 这和 Python 的 list 完全不同（list 里什么都能混）。

知识点 2/7 • 向量化：不写 for，整列一起算

- 一句话：a + 1、a * 2、a > 3 都是「对每个元素做同一件事」，不用循环。
- 代码：

```
a = np.array([1, 2, 3, 4])
print(a + 1)          # [2 3 4 5]
print(a * 2)          # [2 4 6 8]
print(a > 2)          # [False False  True  True]
print(a[a > 2])       # [3 4]      ← 布尔索引
print(a.sum(), a.mean(), a.max(), a.std())
```

★ 不要写 for i in range(len(a)): b[i] = a[i] * 2 —— 1200 行时差别不明显，到 120 万行时慢 100 倍以上。向量化是 pandas 快过 Excel 的根源。

知识点 3/7 • 广播：形状不同也能算，但有规则

- 一句话：从右往左比形状，要么相等、要么其中一个是 1，否则报错。
- 代码：

```
m = np.array([[1, 2, 3], [4, 5, 6]])     # shape (2,3)
print(m + 10)                             # 标量广播：每个元素 +10
print(m + np.array([10, 20, 30]))         # (3,) 广播到每一行
print(m + np.array([[10], [20]]))         # (2,1) 广播到每一列
# m + np.array([1, 2])                    # ValueError: operands could not be broadcast
```

★ 报错原文：ValueError: operands could not be broadcast together with shapes (2,3) (2,)

- —— 最后一位 3 和 2 既不相等也不是 1，所以失败。看报错先看 shapes。

知识点 4/7 • nan 是「缺失值」，它和任何数比较都是 False

- 一句话：np.nan != np.nan，判空要用 np.isnan() 或 pd.isna()。
- 代码：

```
a = np.array([1.0, np.nan, 3.0])
print(a.sum(), a.mean())          # nan nan   ← 一个 nan 污染整个统计
print(np.nansum(a), np.nanmean(a))   # 4.0 2.0  ← nansum/nanmean 跳过 nan
print(np.isnan(a))                # [False  True False]
print(a == a)                     # [ True False  True]   ← nan 自己都不等于自己
```

★ a == np.nan 永远全 False，a != np.nan 永远全 True —— 这是最经典的错。

- 真实数据里 nan 有三种来源：空字符串、'NULL'、Excel 空单元格，读进来都会变 nan。

这一页只讲「nan 是「缺失值」，它和任何数比较都是 False」。讲完就翻页，下一页是另一个知识点。

知识点 5/7 • 真数据上手：把 amount 列解析成数字数组

- 一句话：str.replace 在 numpy 里也能向量化（np.char 或 pd 的字符串方法），但更快的是 pandas。
- 代码：

```
import numpy as np, pandas as pd, pathlib
DATA = pathlib.Path(r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
s = pd.read_csv(DATA / 'orders_dirty.csv')['amount']
num = pd.to_numeric(s.astype(str).str.replace('¥', '', regex=False)
.str.replace(',', '', regex=False), errors='coerce')
print(len(num), num.isna().sum(), round(num.sum(), 2))
```

- （本机实测：1200 行里 35 行金额为空 → 解析后 35 个 nan；
- 全列合计 1,199,463.00。这个数第 12 天 groupby 之后还会再出现一次，如果两次对不上，就是清洗口径变了。）

知识点 6/7 • 形状变换：reshape / 转置 / 拼接

- 一句话：reshape 改形状不改数据；.T 转置；concatenate 按轴拼。
- 代码：

```
a = np.arange(6)
print(a.reshape(2, 3))
print(a.reshape(2, 3).T)                  # 转置
print(np.concatenate([a, a]))             # 拼成一维 12 个
print(np.vstack([a.reshape(2, 3), a.reshape(2, 3)]))   # 竖着摞，变成 4 行
```

★ reshape 的元素个数必须对得上，否则报ValueError: cannot reshape array of size 6 into shape (4,2)。

- 先用 a.size 算一下再 reshape。

知识点 7/7 • numpy 2.x 删了一堆别名，别再抄老教程

- 一句话：np.float_ / np.NaN / np.Inf / np.unicode_ / np.in1d / np.trapz 在本机 2.5.4 上全部 AttributeError。
- 代码：

```
import numpy as np
print(np.__version__)        # 2.5.4
# np.float_          → AttributeError: module 'numpy' has no attribute 'float_'
# np.NaN             → AttributeError
# np.in1d            → AttributeError（改用 np.isin）
# np.trapz           → AttributeError（改用 np.trapezoid）
```

★ 本机实测确认过的替代：np.float64 / np.nan / np.inf / np.str_ / np.isin / np.trapezoid。

- 抄 B 站 2020 年的代码前，先看它用的是哪个版本。

代码速查：今天这几段，要在本机跑通（1/5）

写一个 numpy_probe.py，把下面每一条都跑出真实数字：

```
import numpy as np, pandas as pd, pathlib
```

```
DATA = pathlib.Path(r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
```

```
df = pd.read_csv(DATA / 'orders_dirty.csv')
```

1) 向量化：quantity 列整体判断

q = df['quantity'].to_numpy()

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（2/5）

```
print('数量为负的行数：', (q < 0).sum())          # 应为 25
```

```
print('数量分布：', np.unique(q, return_counts=True))
```

2) 广播：给每个金额打九折

```
num = pd.to_numeric(df['amount'].astype(str).str.replace('¥', '', regex=False)
```

```
                    .str.replace(',', '', regex=False), errors='coerce')
```

arr = num.to_numpy()

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（3/5）

```
print('九折后合计：', round(np.nansum(arr) * 0.9, 2))
```

3) nan 处理：三种统计口径对比

```
print('含 nan 直接 sum：', arr.sum())
```

```
print('nansum：', round(np.nansum(arr), 2))
```

```
print('nan 个数：', np.isnan(arr).sum())          # 应为 35
```

4) 布尔索引：筛出金额 > 2000 的行

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（4/5）

big = df[num > 2000]

```
print('金额 > 2000 的行数: ', len(big))
```

```
print(big[['order_id', 'amount']].head(3).to_string(index=False))
```

验收（做到才算过关）：

① 数量为负的行数 = 25，nan 个数 = 35，两个数都要对上；

```
② 能说清 arr.sum() 为什么是 nan，而 np.nansum 能给出数；
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（5/5）

```
③ 把 (q < 0).sum() 改成 sum(q < 0)，看结果一样吗，并解释为什么。
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

最容易错的地方

★ 不要写 for i in range(len(a)): b[i] = a[i] * 2 —— 1200 行时差别不明显，到 120 万行时慢 100 倍以上。向量化是 pandas 快过 Excel 的根源。

★ 报错原文：ValueError: operands could not be broadcast together with shapes (2,3) (2,)

—— 最后一位 3 和 2 既不相等也不是 1，所以失败。看报错先看 shapes。

★ a == np.nan 永远全 False，a != np.nan 永远全 True —— 这是最经典的错。

这些句子都是「差一个字、结果差一截」的地方，考前只翻这一页也值。

合上视频，自测一遍（能打勾才算学会）

- [ ] 我能不看资料写出：np.array([1, 2, 3.5]) 会整体变成 float64 —— 一个 float 就把全数组升成 float。
- [ ] 我能不看资料写出：a == np.nan 永远全 False，a != np.nan 永远全 True —— 这是最经典的错。
- [ ] 我能不看题源，把今天用到的每个函数「为什么要有它」说一遍
- [ ] 我能说出今天每个参数/返回值改了会怎样——换个写法结果差在哪
- [ ] 今天这几段代码我在本机都跑通、并看清了输出

这些句子都是「差一个字、结果差一截」的地方；三个勾以上，今天就算过关。

今天收工前，先知道明天学什么

第 9 天 • pandas 读数据 + Series/DataFrame + 一眼看清一张表

不用提前学。明天点「① 开始今天的学习」，会换上明天那份 PPT。

收工三件事（5 分钟）

- 今天没跑通的脚本存进 90-我的作业\，命名 2026-11-09-第08天-*.py
- 打卡表「③ 每日打卡」记一笔：今天卡在哪、错在哪（一句话就够）
- 错题明天重做一遍，练习数据在 60-练习环境\Python-Env\data

全阶段 • 第 9 天 • 11/10

pandas 读数据 + SeriesDataFrame + 一眼看清一张表

拿到任意一份 CSV，30 秒内说出：几行几列、每列什么类型、缺多少、数值列的范围。

第 9 天 · 11/10 · 今日知识点回顾

pandas 读数据 + Series/DataFrame + 一眼看清一张表

目标：拿到任意一份 CSV，30 秒内说出：几行几列、每列什么类型、缺多少、数值列的范围。

对着 B 站视频看：视频讲到的每一句，这张 PPT 里都有一条；复习时只翻这份，不用再回去翻分P。

今天结束时，你应该能做到

验收标准（做到这句就算过关）

- 拿到任意一份 CSV，30 秒内说出：几行几列、每列什么类型、缺多少、数值列的范围。

今天的做法（顺序别颠倒）

- 题库地址：file:///D:/DSH工作区/00-学习中心/60-练习环境/Python-Env/data
- 做完记一句：今天卡在哪、错在哪 → 写进 3-每日打卡。

今天看哪几集、读哪一章

这一天的知识：4 集 • 合计约 42 分钟（边看边在本机敲一遍）

P15	Pandas2数据读取	约 14 分钟
P16	Pandas2读取CSV	约 16 分钟
P20	Pandas2数据信息查看操作	约 6 分钟
P21	Pandas2统计计算操作	约 6 分钟

教材（已下载到 20-教材与讲义\python-data\，断网也能看）

`https://raw.githubusercontent.com/datawhalechina/joyful-pandas/master/source/Content/ch2.rst`

第二章：文件的读取与写入、Series/DataFrame、常用基本函数

视频地址就在下面的「今天做哪些题」网页里，点一下直达分P。

知识点 1/7 • read_csv 的四个参数决定成败

- 一句话：encoding、dtype、parse_dates、na_values —— 四个参数一次给全，后面少踩一半坑。
- 代码：

```
import pandas as pd, pathlib
DATA = pathlib.Path(r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
df = pd.read_csv(DATA / 'orders_dirty.csv', encoding='utf-8-sig')
print(df.shape)          # (1200, 10)
print(list(df.columns))
print(df.dtypes)
```

★ 不加 encoding='utf-8-sig' 时，第一列列名会变成 '\ufefforder_id'，

- 后面 df['order_id'] 直接 KeyError: 'order_id' —— 报错信息里那个方框字符就是 BOM。

知识点 2/7 • head / info / describe / value_counts：四眼看全表

- 一句话：head 看长相、info 看类型和缺失、describe 看数值分布、value_counts 看分类计数。
- 代码：

```
print(df.head(3).to_string())
print(df.info())
print(df[['quantity']].describe().round(2))
print(df['status'].value_counts(dropna=False))
```

★ info() 输出的「Non-Null Count」比总行数少多少，就是这一列缺多少。

- 比 isnull().sum() 更省一次调用，且一眼看到 dtype —— 脏数据最常见的错就是「金额列被读成了 object」。

知识点 3/7 • Series 和 DataFrame 的关系

- 一句话：DataFrame 是一张表；df['col'] 取出的是 Series（一列，带索引）。
- 代码：

```
s = df['city']
print(type(s))                    # <class 'pandas.Series'>
print(s.shape, s.dtype)           # (1200,) object
print(s.head(3).tolist())
print(df[['city']].shape)         # (1200, 1) ← 双层方括号拿到的是 DataFrame
```

★ df['city'] 是 Series，df[['city']] 是 DataFrame。两者长得像，但

- df['city'].mean() 能用、df[['city']].mean() 返回 Series —— 差一层方括号，行为不同。

知识点 4/7 • 索引：默认 0..n-1，可以换

- 一句话：df.index 是行号，set_index 能把它换成业务主键。
- 代码：

```
print(df.index[:3])                      # RangeIndex(start=0, stop=3, step=1)
d2 = df.set_index('order_id')
print(d2.index[:3].tolist())
print(d2.loc[d2.index[0], ['city', 'amount']].to_dict())
```

★ set_index 之后 order_id 不再是普通列。想保留成列要用 reset_index() 还原，或者用 df.set_index('order_id', drop=False)。

知识点 5/7 • 真数据的「第一印象」：本机 1200 行订单

- 一句话：先跑一遍四眼，把所有异常记下来，别急着清洗。
- 代码：

```
print('行数', len(df))
print('重复 order_id', df['order_id'].duplicated().sum())      # 37
print('金额缺失', df['amount'].isna().sum())                   # 35
print('城市缺失', df['city'].isna().sum())                     # 56
print('状态取值', df['status'].unique())                       # 6 种写法
print('日期前缀分布', df['create_time'].astype(str).str[:4].value_counts().to_dict())
```

- （本机实测这 6 个数：1200 行 / 37 个重复单号 / 35 行金额空 / 56 行城市空 /status 有 PAID、Paid、paid、'paid '、pending、refunded 六种写法。
- 第 10 天就是专门洗这六个问题。）

知识点 6/7 • 数值列的三种统计口径要分清

- 一句话：mean 跳过 nan，sum 也跳过 nan（pandas 默认 skipna=True），和 numpy 相反。
- 代码：

```
import numpy as np
s = pd.Series([1.0, np.nan, 3.0])
print(s.sum(), s.mean(), s.count(), len(s))    # 4.0 2.0 2 3
print(s.sum(skipna=False))                      # nan  ← 关掉跳过
```

★ 这条和 numpy 的 arr.sum() 行为相反**：numpy 遇 nan 就变 nan，pandas 默认跳过。**

- 混用两套 API 时，同一个「合计」会算出两个数。

知识点 7/7 • 读 Excel 只多一个参数

- 一句话：pd.read_excel(path, sheet_name=...)；不给 sheet_name 读第一个 sheet。
- 代码：

```
xl = pd.ExcelFile(DATA / 'sales_dirty.xlsx')
print(xl.sheet_names)                    # ['明细', '商品表', '城市表']
d = pd.read_excel(DATA / 'sales_dirty.xlsx', sheet_name='明细')
print(d.shape, list(d.columns))
```

★ 本机实测：明细 sheet 400 行 9 列，列名是「订单号/日期/城市/品类/商品/数量/单价/金额/销售员」。

- 读 Excel 比 CSV 慢 5–10 倍，能导出 CSV 就先导。

代码速查：今天这几段，要在本机跑通（1/3）

写一个 look_at_data.py，对三份数据各跑一遍「四眼」：

```
import pandas as pd, pathlib
```

```
DATA = pathlib.Path(r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
```

```
def look(path, **kw):
```

```
    df = pd.read_csv(path, encoding='utf-8-sig', **kw)
```

print('=' * 60)

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（2/3）

```
    print(path.name, df.shape)
    print(df.dtypes.to_string())
    print('缺失: ', df.isna().sum()[df.isna().sum() > 0].to_dict())
    num = df.select_dtypes('number')
    if not num.empty:
        print(num.describe().round(2).to_string())
    return df
o = look(DATA / 'orders_dirty.csv')
u = look(DATA / 'users_dirty.csv')
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（3/3）

e = look(DATA / 'events_dirty.csv')

验收（做到才算过关）：

① orders 报出 1200 行 10 列、37 个重复单号、35 行金额缺失、56 行城市缺失；

② users 报出 500 行 7 列，并说出 age 列的最大值是多少（提示：会有 999）；

③ 能说出为什么 amount 列被读成了 object 而不是 float64。

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

最容易错的地方

后面 df['order_id'] 直接 KeyError: 'order_id' —— 报错信息里那个方框字符就是 BOM。

比 isnull().sum() 更省一次调用，且一眼看到 dtype —— 脏数据最常见的错就是「金额列被读成了 object」。

这些句子都是「差一个字、结果差一截」的地方，考前只翻这一页也值。

合上视频，自测一遍（能打勾才算学会）

- [ ] 我能不看资料写出：info() 输出的「Non-Null Count」比总行数少多少，就是这一列缺多少。
- [ ] 我能不看资料写出：df['city'] 是 Series，df[['city']] 是 DataFrame。两者长得像，但
- [ ] 我能不看资料说出：一句话：encoding、dtype、parse_dates、na_values —— 四个参数一次给全，后面少踩一半坑。
- [ ] 我能不看资料说出：一句话：先跑一遍四眼，把所有异常记下来，别急着清洗。
- [ ] 我能不看题源，把今天用到的每个函数「为什么要有它」说一遍
- [ ] 我能说出今天每个参数/返回值改了会怎样——换个写法结果差在哪

这些句子都是「差一个字、结果差一截」的地方；三个勾以上，今天就算过关。

今天收工前，先知道明天学什么

第 10 天 • 清洗第一课：缺失值、重复行、类型转换、字符串规整

不用提前学。明天点「① 开始今天的学习」，会换上明天那份 PPT。

收工三件事（5 分钟）

- 今天没跑通的脚本存进 90-我的作业\，命名 2026-11-10-第09天-*.py
- 打卡表「③ 每日打卡」记一笔：今天卡在哪、错在哪（一句话就够）
- 错题明天重做一遍，练习数据在 60-练习环境\Python-Env\data

全阶段 · 第 10 天 · 11/11

清洗第一课：缺失值、重复行、类型转换、字符串规整

把 1200 行脏订单洗成「类型全对、无重复、缺失有交代」的干净表，并写一份清洗账（每步丢了多少行、为什么）。

第 10 天 • 11/11 • 今日知识点回顾

清洗第一课：缺失值、重复行、类型转换、字符串规整

目标：把 1200 行脏订单洗成「类型全对、无重复、缺失有交代」的干净表，并写一份清洗账（每步丢了多少行、为什么）。

对着 B 站视频看：视频讲到的每一句，这张 PPT 里都有一条；复习时只翻这份，不用再回去翻分P。

今天结束时，你应该能做到

验收标准（做到这句就算过关）

- 把 1200 行脏订单洗成「类型全对、无重复、缺失有交代」的干净表，并写一份清洗账（每步丢了多少行、为什么）。

今天的做法（顺序别颠倒）

- 题库地址：file:///D:/DSH工作区/00-学习中心/60-练习环境/Python-Env/data
- 做完记一句：今天卡在哪、错在哪 → 写进 3-每日打卡。

今天看哪几集、读哪一章

这一天的知识：3 集 • 合计约 47 分钟（边看边在本机敲一遍）

P32	Pandas2数据清洗与准备	约 15 分钟
P25	Pandas2数据类型转换操作	约 8 分钟
P27	Pandas2数据添加修改删除操作	约 24 分钟

教材（已下载到 20-教材与讲义\python-data\，断网也能看）

`https://raw.githubusercontent.com/datawhalechina/joyful-pandas/master/source/Content/ch7.rst`

第七章 缺失数据：isna / dropna / fillna / 插值

视频地址就在下面的「今天做哪些题」网页里，点一下直达分P。

知识点 1/7 • isna 找缺失，dropna 删，fillna 填 —— 先想清楚删还是填

- 一句话：能算出来的缺失就填（金额=单价×数量），算不出来的才删；每一步都要记账。
- 代码：

```
import pandas as pd, numpy as np, pathlib
DATA = pathlib.Path(r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
df = pd.read_csv(DATA / 'orders_dirty.csv', encoding='utf-8-sig')
print(df.isna().sum().to_dict())
print('删掉任一缺失的行：', len(df.dropna()))
print('只删 amount 缺的行：', len(df.dropna(subset=['amount'])))
print('city 填空：', len(df['city'].fillna('（未知）')))
```

知识点 1/7 • isna 找缺失，dropna 删，fillna 填 —— 先想清楚删还是填（续）

★ dropna() 不给参数是「任一行有任一缺失就删」—— 本机这 1200 行里会被删掉 90 行（35 金额 + 56 城市，可能有重叠）。先 dropna(subset=[...])

- 指定列，别一上来就 dropna()。

知识点 2/7 • 类型转换：to_numeric(errors='coerce') 是主力

- 一句话：errors='coerce' 把转不了的值变成 nan（而不是抛异常），配合先洗字符串。
- 代码：

```
s = df['amount'].astype(str).str.replace('¥', '', regex=False).str.replace(',', '', regex=False)
df['amount'] = pd.to_numeric(s, errors='coerce')
print(df['amount'].dtype, df['amount'].isna().sum())    # float64 35
# 对照：不加 coerce 会直接抛错
# pd.to_numeric(df['amount'])  →  ValueError: Unable to parse string "¥1,899.00"
```

★ 报错原文：ValueError: Unable to parse string "¥1,899.00" at position 3。

- 「at position 3」告诉你第 3 行有问题，但真实数据里往往是几千行都有问题 ——所以一律先 coerce 再统计，别指望它一行行报。

这一页只讲「类型转换：to_numeric(errors='coerce') 是主力」。讲完就翻页，下一页是另一个知识点。

知识点 3/7 • 字符串规整三件套：strip / lower / replace

- 一句话：前后空格、大小写、内部空格 —— 清洗状态列时必须三样都做。
- 代码：

```
print(df['status'].unique())                    # 6 种写法
df['status'] = df['status'].astype(str).str.strip().str.lower()
print(df['status'].value_counts().to_dict())
# {'paid': 417, 'pending': 406, 'refunded': 377}
```

★ 'paid ' 和 'paid' 在 groupby 里是两组 —— 这一条错，第 12 天的品类汇总会多出一行、金额少一截，而且不报错。

知识点 4/7 • drop_duplicates：先想清楚「重复」的定义

- 一句话：drop_duplicates(subset=['order_id'], keep='first') 按业务主键去重。
- 代码：

```
print('重复单号行数：', df['order_id'].duplicated().sum())      # 37
before = len(df)
df = df.drop_duplicates(subset=['order_id'], keep='first').reset_index(drop=True)
print('去重：', before, '→', len(df))                            # 1200 → 1163
```

★ 不给 subset 是「整行完全相同才删」—— 本机这 1200 行的整行重复是 0 行，所以 df.drop_duplicates() 一行都不会删。业务上说的「重复」几乎都是主键重复，一定要给 subset。

知识点 5/7 • 日期解析：本机这份数据有 145 行中文日期，format='mixed' 救不了

- 一句话：中文日期（2026年8月1日）必须单独用 format='%Y年%m月%d日' 再解析。
- 代码：

```
ct = df['create_time'].astype(str)
print(ct.str.contains('年').sum())                  # 145
a = pd.to_datetime(ct, format='mixed', errors='coerce')
print('mixed 解析失败：', a.isna().sum())            # 145
cn = pd.to_datetime(ct, format='%Y年%m月%d日 %H:%M:%S', errors='coerce')
print('中文格式单独解析成功：', cn.notna().sum())
```

知识点 5/7 • 日期解析：本机这份数据有 145 行中文日期，format='mixed' 救不了...

★ 本机实测：pandas 3.0.6 的 format='mixed' 也不认「2026年8月1日」，

- 145 行全部变 NaT。必须先用 str.replace('年','-').replace('月','-').replace('日','')
- 或单独指定 format。这是「看起来能跑、结果悄悄少了 145 行」的典型。

知识点 6/7 • 日期统一成标准格式：一次 replace 解决

- 一句话：把「年月日」换成「-」，再统一 parse，比写三个分支简单。
- 代码：

```
norm = (ct.str.replace('年', '-', regex=False)
.str.replace('月', '-', regex=False)
.str.replace('日', '', regex=False)
.str.replace('/', '-', regex=False))
df['create_time'] = pd.to_datetime(norm, format='mixed', errors='coerce')
print(df['create_time'].isna().sum())          # 0
print(df['create_time'].min(), df['create_time'].max())
```

- （本机实测：清洗后 1163 行日期全部解析成功，范围 2026-08-01 到 2026-10-29。
- 注意「只有日期没有时间」的 42 行会被补成 00:00:00 —— 这是对的，但如果按小时做分析，要单独标出来。）

知识点 7/7 • 清洗账：每步都记一笔，最后一起打印

- 一句话：用 dict 记 {步骤: 影响行数}，跑完打印 —— 这是能拿去面试讲的「清洗口径」。
- 代码：

```
log = {}
log['原始行数'] = len(df)
```

知识点 7/7 • 清洗账：每步都记一笔，最后一起打印（续）

- log['去重删除'] = 37log['金额无法解析'] = int(df['amount'].isna().sum())
- log['日期无法解析'] = int(df['create_time'].isna().sum())
- log['城市缺失已填'] = int((df['city'] == '（未知）').sum())

```
for k, v in log.items():
print(f'{k:12s}{v:>6d}')
```

★ 面试问「你怎么处理脏数据」时，答「我有 6 条规则，每条影响多少行」比答「我用了 dropna 和 fillna」高一个量级 —— 因为前者证明你量过**。**

这一页只讲「清洗账：每步都记一笔，最后一起打印」。讲完就翻页，下一页是另一个知识点。

代码速查：今天这几段，要在本机跑通（1/8）

写 clean_orders_v2.py，产出 clean_v2.csv + clean_log.json：

```
import json, pathlib
```

```
import pandas as pd
```

```
DATA = pathlib.Path(r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
```

```
def clean(path):
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（2/8）

```
    df = pd.read_csv(path, encoding='utf-8-sig')
```

log = {'raw_rows': len(df)}

1) 金额

```
    s = (df['amount'].astype(str).str.replace('¥', '', regex=False)
```

```
           .str.replace(',', '', regex=False))
```

```
    df['amount'] = pd.to_numeric(s, errors='coerce')
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（3/8）

```
    log['amount_na'] = int(df['amount'].isna().sum())
```

```
    # 2) 状态规整
```

```
    df['status'] = df['status'].astype(str).str.strip().str.lower()
```

```
    # 3) 城市填空
```

```
    df['city'] = df['city'].fillna('（未知）').str.strip().replace('', '（未知）')
```

```
    log['city_filled'] = int((df['city'] == '（未知）').sum())
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（4/8）

```
# 4) 日期统一

ct = df['create_time'].astype(str)
```

```
    norm = (ct.str.replace('年', '-', regex=False)
```

```
            .str.replace('月', '-', regex=False)
```

```
            .str.replace('日', '', regex=False)
```

```
            .str.replace('/', '-', regex=False))
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（5/8）

```
    df['create_time'] = pd.to_datetime(norm, format='mixed', errors='coerce')
```

```
    log['time_na'] = int(df['create_time'].isna().sum())
```

5) 去重

before = len(df)

```
    df = df.drop_duplicates(subset=['order_id'], keep='first').reset_index(drop=True)
```

```
    log['dup_dropped'] = before - len(df)
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（6/8）

```
    # 6) 数量：负数当录入错误，置空
    neg = int((df['quantity'] < 0).sum())
```

```
    df.loc[df['quantity'] < 0, 'quantity'] = pd.NA
```

```
    log['qty_negative'] = neg
    log['clean_rows'] = len(df)
    return df, log

if __name__ == '__main__':
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（7/8）

```
    df, log = clean(DATA / 'orders_dirty.csv')
```

```
    df.to_csv(DATA / 'clean_v2.csv', index=False, encoding='utf-8-sig')
```

```
    (DATA / 'clean_log.json').write_text(
```

```
        json.dumps(log, ensure_ascii=False, indent=2), encoding='utf-8')
```

```
    print(json.dumps(log, ensure_ascii=False, indent=2))
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（8/8）

```
print(df.dtypes)
```

验收（做到才算过关）：

① 六个数字对得上：raw 1200 / amount_na 35 / city_filled 56 / time_na 0 / dup 37 / clean 1163；

② df.dtypes 里 amount 是 float64、create_time 是 datetime64[ns]；

③ 能解释「为什么去重放在日期清洗之后、数量置空之前」——顺序影响 clean_rows。

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

最容易错的地方（1/2）

指定列，别一上来就 dropna()。

★ 报错原文：ValueError: Unable to parse string "¥1,899.00" at position 3。

「at position 3」告诉你第 3 行有问题，但真实数据里往往是几千行都有问题 ——所以一律先 coerce 再统计，别指望它一行行报。

★ 'paid ' 和 'paid' 在 groupby 里是两组 —— 这一条错，第 12 天的品类汇总会多出一行、金额少一截，而且不报错。

这些句子都是「差一个字、结果差一截」的地方，考前只翻这一页也值。

最容易错的地方（2/2）

> 注意「只有日期没有时间」的 42 行会被补成 00:00:00 —— 这是对的，但如果按小时做分析，要单独标出来。）

这些句子都是「差一个字、结果差一截」的地方，考前只翻这一页也值。

合上视频，自测一遍（能打勾才算学会）

- [ ] 我能不看资料写出：'paid ' 和 'paid' 在 groupby 里是两组 —— 这一条错，第 12 天的品类汇总会多出一行、金额少一截，而且不报错。
- [ ] 我能不看资料说出：「at position 3」告诉你第 3 行有问题，但真实数据里往往是几千行都有问题 ——所以一律先 coerce 再统计，别指望它一行行报。
- [ ] 我能不看资料说出：一句话：前后空格、大小写、内部空格 —— 清洗状态列时必须三样都做。
- [ ] 我能不看资料说出：一句话：中文日期（2026年8月1日）必须单独用 format='%Y年%m月%d日' 再解析。
- [ ] 我能不看资料说出：145 行全部变 NaT。必须先用 str.replace('年','-').replace('月','-').replace('日','')
- [ ] 我能不看题源，把今天用到的每个函数「为什么要有它」说一遍

这些句子都是「差一个字、结果差一截」的地方；三个勾以上，今天就算过关。

今天收工前，先知道明天学什么

第 11 天 • 筛选与排序：布尔索引、query、loc/iloc、多列排序

不用提前学。明天点「① 开始今天的学习」，会换上明天那份 PPT。

收工三件事（5 分钟）

- 今天没跑通的脚本存进 90-我的作业\，命名 2026-11-11-第10天-*.py
- 打卡表「③ 每日打卡」记一笔：今天卡在哪、错在哪（一句话就够）
- 错题明天重做一遍，练习数据在 60-练习环境\Python-Env\data

全阶段 · 第 11 天 · 11/12

筛选与排序：布尔索引、query、lociloc、多列排序

用三种写法（布尔索引 / query / loc）写出同一个筛选条件，并说清各自的适用场景。

第 11 天 • 11/12 • 今日知识点回顾

筛选与排序：布尔索引、query、loc/iloc、多列排序

目标：用三种写法（布尔索引 / query / loc）写出同一个筛选条件，并说清各自的适用场景。

对着 B 站视频看：视频讲到的每一句，这张 PPT 里都有一条；复习时只翻这份，不用再回去翻分P。

今天结束时，你应该能做到

验收标准（做到这句就算过关）

- 用三种写法（布尔索引 / query / loc）写出同一个筛选条件，并说清各自的适用场景。

今天的做法（顺序别颠倒）

- 题库地址：file:///D:/DSH工作区/00-学习中心/60-练习环境/Python-Env/data
- 做完记一句：今天卡在哪、错在哪 → 写进 3-每日打卡。

今天看哪几集、读哪一章

这一天的知识：3 集 • 合计约 37 分钟（边看边在本机敲一遍）

P23	Pandas2数据选择操作	约 18 分钟
P24	Pandas2复杂数据查询操作	约 15 分钟
P28	Pandas2高级查询过滤	约 4 分钟

教材（已下载到 20-教材与讲义\python-data\，断网也能看）

`https://raw.githubusercontent.com/datawhalechina/joyful-pandas/master/source/Content/ch3.rst`

第三章 索引：loc / iloc / 布尔索引 / 多层索引

视频地址就在下面的「今天做哪些题」网页里，点一下直达分P。

知识点 1/7 • 布尔索引：一行写完所有条件

- 一句话：df[df['列'] > 值] 是核心写法；多个条件用 & | ~，每个条件都要加括号。
- 代码：

```
import pandas as pd, pathlib
DATA = pathlib.Path(r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
df = pd.read_csv(DATA / 'clean_v2.csv', encoding='utf-8-sig')
print(len(df[df['amount'] > 2000]))
print(len(df[(df['status'] == 'paid') & (df['city'] == '北京')]))
print(len(df[(df['status'] == 'paid') | (df['amount'] > 5000)]))
print(len(df[~(df['status'] == 'refunded')]))
```

知识点 1/7 • 布尔索引：一行写完所有条件（续）

★ 不加括号会报错：

• ValueError: The truth value of a Series is ambiguous. Use a.empty, a.bool(),a.item(), a.any() or a.all(). 原因是 & 的优先级比 > 高，Python 先算 'paid' & df['city']，整串就乱了。

知识点 2/7 • query：条件写成字符串，适合「条件很多、还要给业务看」

- 一句话：df.query("amount > 2000 and city == '北京'")，用 and/or 不用 &/|。
- 代码：

```
print(len(df.query("amount > 2000")))
print(len(df.query("status == 'paid' and city == '北京'")))
city = '上海'
print(len(df.query("city == @city")))      # 用 @ 引用外部变量
```

★ query 里列名有空格或中文要反引号：df.query("`销售员` == '孙悦'")。

- 本机 clean_v2.csv 的列名是英文，暂时用不上，但读 Excel 时天天遇到。

知识点 3/7 • loc 与 iloc：一个按名字、一个按位置

- 一句话：loc[行标签, 列名]；iloc[行号, 列号] —— 行号从 0 开始，不含右端。
- 代码：

```
d2 = df.set_index('order_id')
print(d2.loc[100001, ['city', 'amount']].to_dict())
print(d2.iloc[0:2, 0:3].to_string())
print(df.loc[0:2, 'city'].tolist())        # loc 的切片**含右端**
```

★ 最容易错的一条：loc[0:2] 取 3 行（含 2），iloc[0:2] 取 2 行（不含 2）。

- 记法：loc 是「标签切片」，像字典；iloc 是「位置切片」，像 list。

知识点 4/7 • 排序：sort_values 多列 + 稳定排序

- 一句话：df.sort_values(['列1', '列2'], ascending=[True, False])。
- 代码：

```
top = (df.sort_values(['amount', 'order_id'], ascending=[False, True])
.head(5)[['order_id', 'city', 'amount']])
print(top.to_string(index=False))
print(df.nlargest(3, 'amount')[['order_id', 'amount']].to_string(index=False))
```

★ nlargest/nsmallest 比 sort_values().head() 快（内部用堆，不全排），只要前 N 名时优先用它。sort_values 默认 ascending=True，写反了会得到最小的一批。

知识点 5/7 • isin / between / str.contains：三个高频条件写法

- 一句话：isin 判「属于集合」，between 判区间（含两端），str.contains 做模糊匹配。
- 代码：

```
print(len(df[df['city'].isin(['北京', '上海', '广州'])]))
print(len(df[df['amount'].between(1000, 2000)]))
print(len(df[df['product_name'].str.contains('小米', na=False)]))
print(len(df[df['product_name'].str.startswith('罗技', na=False)]))
```

★ str.contains 默认 na=nan 会返回 nan，在布尔索引里被当成 False —— 看着没事，但 df[mask] 的行数会比你算的少。一律显式写 na=False。

知识点 6/7 • 真数据：几个能直接拿去讲的筛选结果

- 一句话：每个筛选都要能说出「筛出来多少行、为什么是这个数」。
- 代码：

```
paid = df[df['status'] == 'paid']
print('paid 行数', len(paid), '合计', round(paid['amount'].sum(), 2))
print('北京 paid', len(paid[paid['city'] == '北京']))
print('金额 1000-2000', len(df[df['amount'].between(1000, 2000)]))
print('品类是数码', len(df[df['category'] == '数码']))
```

- （本机实测 clean_v2.csv：paid 417 行，合计 422,436.00；
- 北京 paid 若干行；金额 1000-2000 的行数可以直接数。
- 注意 paid 合计要和第 12 天 groupby 的结果一致 —— 不一致就是筛选口径不同。）

这一页只讲「真数据：几个能直接拿去讲的筛选结果」。讲完就翻页，下一页是另一个知识点。

知识点 7/7 • 筛选完记得 copy，别改到原表

- 一句话：df[mask] 拿到的是视图或副本不确定，要改就显式 .copy()。
- 代码：

```
sub = df[df['status'] == 'paid'].copy()
```

知识点 7/7 • 筛选完记得 copy，别改到原表（续）

- sub['amount_disc'] = sub['amount'] * 0.9　　# 安全

```
# 不 copy 直接赋值会报 SettingWithCopyWarning
```

★ pandas 3.0 强制开启 Copy-on-Write，SettingWithCopyWarning 基本消失，但「改了子集、原表没变」这个语义更容易让人困惑 —— 显式 .copy() 最稳。

代码速查：今天这几段，要在本机跑通（1/5）

写 filter_practice.py，每题打印行数 + 一行解释：

```
import pandas as pd, pathlib
```

```
DATA = pathlib.Path(r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
```

```
df = pd.read_csv(DATA / 'clean_v2.csv', encoding='utf-8-sig')
```

1) 三种写法写同一个条件：status=paid 且 city=北京

```
a = len(df[(df['status'] == 'paid') & (df['city'] == '北京')])
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（2/5）

```
b = len(df.query("status == 'paid' and city == '北京'"))
```

```
c = len(df.loc[(df['status'] == 'paid') & (df['city'] == '北京')])
```

```
print('三种写法：', a, b, c)          # 三个数必须相同
```

2) 金额 Top 10，用 nlargest

```
print(df.nlargest(10, 'amount')[['order_id', 'city', 'category', 'amount']].to_string(index=False))
```

3) 区间 + 集合 + 模糊

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（3/5）

```
print('1000-2000: ', len(df[df['amount'].between(1000, 2000)]))
```

```
print('北上广: ', len(df[df['city'].isin(['北京', '上海', '广州'])]))
```

```
print('商品名含 小米: ', len(df[df['product_name'].str.contains('小米', na=False)]))
```

4) 排除退款

```
print('非退款行数: ', len(df[df['status'] != 'refunded']))
```

5) 按两列排序取前 5

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（4/5）

```
print(df.sort_values(['city', 'amount'], ascending=[True, False])
```

```
        .groupby('city').head(1)[['city', 'order_id', 'amount']]
```

.to_string(index=False))

验收（做到才算过关）：

① 第 1 题三个数完全一致；

```
② 能说清为什么 (df['a'] > 1) & (df['b'] < 2) 不加括号会报 ValueError；
```

代码速查：今天这几段，要在本机跑通（5/5）

```
③ 第 5 题能解释 groupby('city').head(1) 和直接 head(1) 的差别。
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

最容易错的地方

★ 最容易错的一条：loc[0:2] 取 3 行（含 2），iloc[0:2] 取 2 行（不含 2）。

注意 paid 合计要和第 12 天 groupby 的结果一致 —— 不一致就是筛选口径不同。）

这些句子都是「差一个字、结果差一截」的地方，考前只翻这一页也值。

合上视频，自测一遍（能打勾才算学会）

- [ ] 我能不看资料说出：注意 paid 合计要和第 12 天 groupby 的结果一致 —— 不一致就是筛选口径不同。）
- [ ] 我能不看题源，把今天用到的每个函数「为什么要有它」说一遍
- [ ] 我能说出今天每个参数/返回值改了会怎样——换个写法结果差在哪
- [ ] 今天这几段代码我在本机都跑通、并看清了输出

这些句子都是「差一个字、结果差一截」的地方；三个勾以上，今天就算过关。

今天收工前，先知道明天学什么

第 12 天 • groupby 分组聚合：agg、transform、多列分组

不用提前学。明天点「① 开始今天的学习」，会换上明天那份 PPT。

收工三件事（5 分钟）

- 今天没跑通的脚本存进 90-我的作业\，命名 2026-11-12-第11天-*.py
- 打卡表「③ 每日打卡」记一笔：今天卡在哪、错在哪（一句话就够）
- 错题明天重做一遍，练习数据在 60-练习环境\Python-Env\data

全阶段 · 第 12 天 · 11/13

groupby 分组聚合：agg、transform、多列分组

给出「每个品类每个城市的实付合计与订单数」，并说清 agg 和 transform 的结果形状差别。

第 12 天 • 11/13 • 今日知识点回顾

groupby 分组聚合：agg、transform、多列分组

目标：给出「每个品类每个城市的实付合计与订单数」，并说清 agg 和 transform 的结果形状差别。

对着 B 站视频看：视频讲到的每一句，这张 PPT 里都有一条；复习时只翻这份，不用再回去翻分P。

今天结束时，你应该能做到

验收标准（做到这句就算过关）

- 给出「每个品类每个城市的实付合计与订单数」，并说清 agg 和 transform 的结果形状差别。

今天的做法（顺序别颠倒）

- 题库地址：file:///D:/DSH工作区/00-学习中心/60-练习环境/Python-Env/data
- 做完记一句：今天卡在哪、错在哪 → 写进 3-每日打卡。

今天看哪几集、读哪一章

这一天的知识：3 集 • 合计约 37 分钟（边看边在本机敲一遍）

P34	Pandas2数据分组与聚合	约 6 分钟
P33	【数据的合并和分组聚合】03数据分组聚合	约 15 分钟
P34	【数据的合并和分组聚合】04数据分组聚合02	约 16 分钟

教材（已下载到 20-教材与讲义\python-data\，断网也能看）

`https://raw.githubusercontent.com/datawhalechina/joyful-pandas/master/source/Content/ch4.rst`

第四章 分组：groupby 的三大操作（聚合/变换/过滤）

视频地址就在下面的「今天做哪些题」网页里，点一下直达分P。

知识点 1/7 • groupby 三步：拆、算、合

- 一句话：df.groupby('列')['要算的列'].agg(函数) —— 拆开、每块算、拼回来。
- 代码：

```
import pandas as pd, pathlib
DATA = pathlib.Path(r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
df = pd.read_csv(DATA / 'clean_v2.csv', encoding='utf-8-sig')
paid = df[df['status'] == 'paid']
g = paid.groupby('category')['amount'].agg(['count', 'sum', 'mean']).round(2)
print(g.sort_values('sum', ascending=False).to_string())
```

这一页只讲「groupby 三步：拆、算、合」。讲完就翻页，下一页是另一个知识点。

知识点 1/7 • groupby 三步：拆、算、合（续）

★ 结果（本机实测）：数码 132 单 / 192,231.00 / 均价 1,456.30；

- 家居 113 单 / 147,895.00 / 1,308.81；美妆 74 单 / 59,704.00 / 806.81；
- 食品 85 单 / 22,606.00 / 265.95。四个数加起来 = 422,436.00 = 昨天的 paid 合计。

知识点 2/7 • as_index=False 让分组键留在列里

- 一句话：groupby(..., as_index=False) 等价于 .reset_index()，少写一步。
- 代码：

```
t = paid.groupby('city', as_index=False)['amount'].sum().sort_values('amount', ascending=False)
print(t.head(3).to_string(index=False))
print(t.columns.tolist())      # ['city', 'amount']
```

★ 不加 as_index=False 时，city 变成索引，后面 to_csv 会丢一列 ——这是「导出的文件少一列」最常见的原因。

知识点 3/7 • 命名聚合：一个列一次算多个指标，输出列名自己定

- 一句话：agg(总单数=('order_id', 'count'), 实付=('amount', 'sum'))。
- 代码：

```
r = paid.groupby('city').agg(
总单数=('order_id', 'count'),
实付合计=('amount', 'sum'),
客单价=('amount', 'mean'),
最大单=('amount', 'max'),
).round(2).sort_values('实付合计', ascending=False)
print(r.head(5).to_string())
```

★ 命名聚合要求 pandas ≥ 0.25，本机 3.0.6 完全支持。老写法agg({'amount': ['sum', 'mean']}) 会得到多级列名，导出 CSV 后表头是两行 ——给业务看时一律用命名聚合。

知识点 4/7 • transform：算完把结果「铺回原表的每一行」

- 一句话：agg 每个组返回一行；transform 每个组返回和原组一样多的行。
- 代码：

```
df['city_total'] = df.groupby('city')['amount'].transform('sum')
df['city_share'] = (df['amount'] / df['city_total'] * 100).round(2)
print(df[['order_id', 'city', 'amount', 'city_total', 'city_share']].head(5).to_string(index=False))
```

★ 用 agg 做这件事要先 merge 回去，transform 一步到位 —— 算「占比」时必用。

- 注意分母含 nan 时结果是 nan，要先用 fillna(0) 或只对 paid 算。

知识点 5/7 • 多列分组 + 交叉表

- 一句话：groupby(['城市', '品类']) 得到多层索引，unstack() 摊成二维表。
- 代码：

```
pv = (paid.groupby(['city', 'category'])['amount'].sum()
.unstack(fill_value=0).round(0))
print(pv.shape)                                  # (9, 4)
print(pv.sum(axis=1).sort_values(ascending=False).head(3).to_string())
```

★ 本机实测：9 个城市 × 4 个品类，实付合计最高的城市是南京（60,532.00）。

- 这个结果第 15 天用 pivot_table 会再算一次，两个数必须一致。

知识点 6/7 • size 与 count 的差别（老坑）

- 一句话：size 数「行数（含 nan）」，count 数「非 nan 的个数」。
- 代码：

```
print(paid.groupby('city').size().head(3).to_string())
print(paid.groupby('city')['amount'].count().head(3).to_string())
```

★ 只要有一个金额是 nan，这两个数就不一样。回答「一共多少单」时，永远用 size() 或 count 在非空列上 —— 否则会少算。

知识点 7/7 • groupby 的两个性能与正确性开关

- 一句话：observed=True（分类列只保留出现过的组合）、sort=False（不排序更快）。
- 代码：

```
df['city'] = df['city'].astype('category')
print(len(df.groupby(['city', 'category'], observed=True)))
print(len(df.groupby(['city', 'category'], observed=False)))   # 笛卡尔积，可能更大
```

这一页只讲「groupby 的两个性能与正确性开关」。讲完就翻页，下一页是另一个知识点。

知识点 7/7 • groupby 的两个性能与正确性开关（续）

- （pandas 3.0 的 groupby 默认 observed=True 了，但显式写出来更保险；

```
observed=False 时分类列会补齐所有组合，行数暴涨且大量为 0 —
```

- 这是「分组结果莫名其妙多了几百行」的元凶。）

代码速查：今天这几段，要在本机跑通（1/5）

写 groupby_practice.py，产出五个能直接贴进报告的表：

```
import pandas as pd, pathlib
```

```
DATA = pathlib.Path(r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
```

```
df = pd.read_csv(DATA / 'clean_v2.csv', encoding='utf-8-sig')
```

paid = df[df['status'] == 'paid']

```
# 1) 品类汇总（对照本机数字：数码 192231 / 家居 147895 / 美妆 59704 / 食品 22606）
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（2/5）

```
print(paid.groupby('category').agg(
```

```
    单数=('order_id', 'count'), 实付=('amount', 'sum'), 客单价=('amount', 'mean')
```

```
).round(2).sort_values('实付', ascending=False).to_string())
```

2) 城市汇总，as_index=False

```
print(paid.groupby('city', as_index=False)['amount'].sum()
```

代码速查：今天这几段，要在本机跑通（3/5）

```
        .sort_values('amount', ascending=False).head(5).to_string(index=False))
```

3) 城市 × 品类 交叉（unstack）

```
print(paid.groupby(['city', 'category'])['amount'].sum()
```

```
        .unstack(fill_value=0).round(0).to_string())
```

4) 每个订单占所在城市实付的百分比（transform）

```
df['city_total'] = df.groupby('city')['amount'].transform('sum')
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（4/5）

```
df['share'] = (df['amount'] / df['city_total'] * 100).round(2)
```

```
print(df.nlargest(5, 'share')[['order_id', 'city', 'amount', 'city_total', 'share']]
```

.to_string(index=False))

```
# 5) 每个城市的 Top1 订单（groupby + nlargest 的组合写法）
```

```
print(paid.sort_values('amount', ascending=False)
```

代码速查：今天这几段，要在本机跑通（5/5）

```
        .groupby('city').head(1)[['city', 'order_id', 'amount']]
```

```
        .sort_values('amount', ascending=False).to_string(index=False))
```

验收（做到才算过关）：

① 第 1 题四个品类的实付合计相加 = 422,436.00；

② 第 3 题交叉表是 9 行 × 4 列，合计最高的城市是南京；

③ 能说清 transform 和 agg 的结果形状差别，并各举今天代码里的一处用法。

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

最容易错的地方

> 注意分母含 nan 时结果是 nan，要先用 fillna(0) 或只对 paid 算。

这些句子都是「差一个字、结果差一截」的地方，考前只翻这一页也值。

合上视频，自测一遍（能打勾才算学会）

- [ ] 我能不看资料写出：不加 as_index=False 时，city 变成索引，后面 to_csv 会丢一列 ——这是「导出的文件少一列」最常见的原因。
- [ ] 我能不看资料写出：用 agg 做这件事要先 merge 回去，transform 一步到位 —— 算「占比」时必用。
- [ ] 我能不看资料写出：只要有一个金额是 nan，这两个数就不一样。回答「一共多少单」时，永远用 size() 或 count 在非空列上 —— 否则会少算。
- [ ] 我能不看资料说出：注意分母含 nan 时结果是 nan，要先用 fillna(0) 或只对 paid 算。
- [ ] 我能不看资料说出：这个结果第 15 天用 pivot_table 会再算一次，两个数必须一致。
- [ ] 我能不看题源，把今天用到的每个函数「为什么要有它」说一遍

这些句子都是「差一个字、结果差一截」的地方；三个勾以上，今天就算过关。

今天收工前，先知道明天学什么

第 13 天 • merge 连接：how 四种取值、连接键对不上怎么办

不用提前学。明天点「① 开始今天的学习」，会换上明天那份 PPT。

收工三件事（5 分钟）

- 今天没跑通的脚本存进 90-我的作业\，命名 2026-11-13-第12天-*.py
- 打卡表「③ 每日打卡」记一笔：今天卡在哪、错在哪（一句话就够）
- 错题明天重做一遍，练习数据在 60-练习环境\Python-Env\data

全阶段 · 第 13 天 · 11/14

merge 连接：how 四种取值、连接键对不上怎么办

能解释「为什么 left join 之后行数变多了」。

第 13 天 • 11/14 • 今日知识点回顾

merge 连接：how 四种取值、连接键对不上怎么办

目标：说清 left / inner / outer / right 四种连接的差别，并能解释「为什么 left join 之后行数变多了」。

对着 B 站视频看：视频讲到的每一句，这张 PPT 里都有一条；复习时只翻这份，不用再回去翻分P。

今天结束时，你应该能做到

验收标准（做到这句就算过关）

- 说清 left / inner / outer / right 四种连接的差别，并能解释「为什么 left join 之后行数变多了」。

今天的做法（顺序别颠倒）

- 题库地址：file:///D:/DSH工作区/00-学习中心/60-练习环境/Python-Env/data
- 做完记一句：今天卡在哪、错在哪 → 写进 3-每日打卡。

今天看哪几集、读哪一章

这一天的知识：3 集 • 合计约 52 分钟（边看边在本机敲一遍）

P31	Pandas2数据合并与对比	约 21 分钟
P32	【数据的合并和分组聚合】02数据合并	约 19 分钟
P35	【数据的合并和分组聚合】05数据的索引学习	约 12 分钟

教材（已下载到 20-教材与讲义\python-data\，断网也能看）

`https://raw.githubusercontent.com/datawhalechina/joyful-pandas/master/source/Content/ch6.rst`

第六章 连接：merge / join / concat 的区别与 on/left_on/right_on

视频地址就在下面的「今天做哪些题」网页里，点一下直达分P。

知识点 1/7 • merge 的四个 how，差别只在「留谁的行」

- 一句话：inner 交集、left 留左表全部、right 留右表全部、outer 并集。
- 代码：

```
import pandas as pd, pathlib
DATA = pathlib.Path(r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
df = pd.read_csv(DATA / 'clean_v2.csv', encoding='utf-8-sig')
u = pd.read_csv(DATA / 'users_dirty.csv', encoding='utf-8-sig')
for how in ['inner', 'left', 'right', 'outer']:
m = df.merge(u[['user_id', 'gender', 'age']], on='user_id', how=how)
print(how, len(m), m['gender'].isna().sum())
```

★ 本机实测：left 连接后 1163 行，其中 39 行的 gender 是 nan ——说明订单里有 39 个 user_id 在 users 表里查不到。这 39 行就是「脏数据」，清洗时要单独交代（是用户被删了，还是订单表的 user_id 写错了）。

这一页只讲「merge 的四个 how，差别只在「留谁的行」」。讲完就翻页，下一页是另一个知识点。

知识点 2/7 • left join 行数变多 = 右表在连接键上不唯一

- 一句话：右表一个键对应多行，左表一行就会被复制成多行 —— 这是最常见的「数据翻倍」事故。
- 代码：

```
print('右表 user_id 唯一? ', u['user_id'].is_unique)      # True
dup = pd.DataFrame({'user_id': [1001, 1001], 'tag': ['新客', '老客']})
m = df.head(3).merge(dup, on='user_id', how='left')
print(len(m))                                             # 4 — 1001 那一行变两行
```

★ 连接前先跑 df['键'].is_unique 和 df['键'].duplicated().sum()。

- 多对多连接会让汇总金额翻倍，而且不报错 —— 这是最贵的一类 bug。

知识点 3/7 • 连接键名字不同：left_on / right_on

- 一句话：左表叫 user_id、右表叫 uid 时，用 left_on='user_id', right_on='uid'。
- 代码：

```
u2 = u.rename(columns={'user_id': 'uid'})
m = df.merge(u2[['uid', 'gender']], left_on='user_id', right_on='uid', how='left')
print(m.columns.tolist()[:6])
print(len(m))
```

★ 这样连完 user_id 和 uid 两列都在，值一样 —— 记得 drop 掉一列，否则后面groupby 会出现两列重复的键，导出 CSV 时业务方会问「怎么有两个 ID」。

知识点 4/7 • indicator 参数：一眼看出哪行没连上

- 一句话：merge(..., indicator=True) 会多一列 _merge，值是 both / left_only / right_only。
- 代码：

```
m = df.merge(u[['user_id', 'gender']], on='user_id', how='left', indicator=True)
print(m['_merge'].value_counts().to_dict())
miss = m[m['_merge'] == 'left_only']
print('连不上的订单数：', len(miss), '涉及金额：', round(miss['amount'].sum(), 2))
```

★ 连不上的行不是「小事」—— 如果它们占了 5% 的金额，任何按用户维度的分析都会系统性偏低。报数字，别只说「有少量缺失」。

知识点 5/7 • validate：让 merge 主动检查「一对几」

- 一句话：validate='one_to_one' / 'one_to_many' / 'many_to_one'，不符合直接报错。
- 代码：

```
m = df.merge(u[['user_id', 'gender']], on='user_id', how='left',
validate='many_to_one')       # 左多右一，符合预期
print(len(m))
# validate='one_to_one' 会报
# MergeError: Merge keys are not unique in either left or right dataset
```

★ 加上 validate 之后，「不小心连成多对多」会在当场炸掉，而不是三天后发现报表数字翻倍。生产脚本里建议每个 merge 都带。

这一页只讲「validate：让 merge 主动检查「一对几」」。讲完就翻页，下一页是另一个知识点。

知识点 6/7 • concat：纵向摞，不是横向连

- 一句话：pd.concat([df1, df2], ignore_index=True) 把两个同结构的表摞起来。
- 代码：

```
a = df.head(3)
b = df.tail(3)
print(len(pd.concat([a, b], ignore_index=True)))      # 6
print(pd.concat([a, b]).index.tolist())                # 0,1,2,1160,1161,1162 重复
```

★ 不加 ignore_index=True，索引会重复（0,1,2,1160,1161,1162），后面df.loc[0] 会返回两行 —— 「莫名其妙多了几行」的另一个来源。

知识点 7/7 • 真数据：把用户信息接到订单上，做分性别汇总

- 一句话：merge 之后 groupby 才有意义 —— 单表只能按已有列分组。
- 代码：

```
m = df.merge(u[['user_id', 'gender', 'age']], on='user_id', how='left')
paid = m[m['status'] == 'paid']
g = (paid.groupby('gender', dropna=False)
.agg(单数=('order_id', 'count'), 实付=('amount', 'sum'))
.round(2))
print(g.to_string())
```

- （users 表的 gender 有六种写法：男 / 女 / M / F / 未知 / 空。
- 直接 groupby 会得到六行 —— 第 16 天会专门讲怎么归并成两行 + 一行未知。）

这一页只讲「真数据：把用户信息接到订单上，做分性别汇总」。讲完就翻页，下一页是另一个知识点。

代码速查：今天这几段，要在本机跑通（1/5）

写 merge_practice.py，每题打印行数并解释：

```
import pandas as pd, pathlib
```

```
DATA = pathlib.Path(r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
```

```
df = pd.read_csv(DATA / 'clean_v2.csv', encoding='utf-8-sig')
```

```
u = pd.read_csv(DATA / 'users_dirty.csv', encoding='utf-8-sig')
```

1) 四种 how 的行数与缺失数

代码速查：今天这几段，要在本机跑通（2/5）

```
for how in ['inner', 'left', 'right', 'outer']:
```

```
    m = df.merge(u[['user_id', 'gender']], on='user_id', how=how)
```

```
    print(f'{how:6s} rows={len(m):5d} gender_na={m["gender"].isna().sum():4d}')
```

2) indicator 找出连不上的订单

```
m = df.merge(u[['user_id', 'gender']], on='user_id', how='left', indicator=True)
```

代码速查：今天这几段，要在本机跑通（3/5）

```
print(m['_merge'].value_counts().to_dict())
```

miss = m[m['_merge'] == 'left_only']

```
print('连不上：', len(miss), '金额合计', round(miss['amount'].sum(), 2))
```

3) validate 检查一对多

```
ok = df.merge(u[['user_id', 'gender']], on='user_id', how='left', validate='many_to_one')
```

```
print('many_to_one 通过，行数', len(ok))
```

代码速查：今天这几段，要在本机跑通（4/5）

```
# 4) 接上用户表后按性别汇总
paid = ok[ok['status'] == 'paid']
print(paid.groupby('gender', dropna=False)
          .agg(单数=('order_id', 'count'), 实付=('amount', 'sum'))
          .round(2).to_string())
```

验收（做到才算过关）：

```
① 第 1 题四个 how 的行数都能解释（inner < left ≤ outer）；
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（5/5）

② 连不上的订单数是 39，金额合计能报出来；

③ 能说清「为什么 left join 可能比左表行数多」，并举一个具体例子。

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

最容易错的地方

★ 本机实测：left 连接后 1163 行，其中 39 行的 gender 是 nan ——说明订单里有 39 个 user_id 在 users 表里查不到。这 39 行就是「脏数据」，清洗时要单独交代（是用户被删了，还是订单表的 user_id 写错了）。

多对多连接会让汇总金额翻倍，而且不报错 —— 这是最贵的一类 bug。

★ 连不上的行不是「小事」—— 如果它们占了 5% 的金额，任何按用户维度的分析都会系统性偏低。报数字，别只说「有少量缺失」。

这些句子都是「差一个字、结果差一截」的地方，考前只翻这一页也值。

合上视频，自测一遍（能打勾才算学会）

- [ ] 我能不看资料写出：连接前先跑 df['键'].is_unique 和 df['键'].duplicated().sum()。
- [ ] 我能不看资料写出：连不上的行不是「小事」—— 如果它们占了 5% 的金额，任何按用户维度的分析都会系统性偏低。报数字，别只说「有少量缺失」。
- [ ] 我能不看资料写出：加上 validate 之后，「不小心连成多对多」会在当场炸掉，而不是三天后发现报表数字翻倍。生产脚本里建议每个 merge 都带。
- [ ] 我能不看题源，把今天用到的每个函数「为什么要有它」说一遍
- [ ] 我能说出今天每个参数/返回值改了会怎样——换个写法结果差在哪
- [ ] 今天这几段代码我在本机都跑通、并看清了输出

这些句子都是「差一个字、结果差一截」的地方；三个勾以上，今天就算过关。

今天收工前，先知道明天学什么

第 14 天 • ★W5 总验收：给一份脏 CSV，独立清洗成可分析表并说清每一步

不用提前学。明天点「① 开始今天的学习」，会换上明天那份 PPT。

收工三件事（5 分钟）

- 今天没跑通的脚本存进 90-我的作业\，命名 2026-11-14-第13天-*.py
- 打卡表「③ 每日打卡」记一笔：今天卡在哪、错在哪（一句话就够）
- 错题明天重做一遍，练习数据在 60-练习环境\Python-Env\data

全阶段 • 第 14 天 • 11/15

★W5 总验收：给一份脏 CSV，独立清洗成可分析表并说清每一步

能交给别人的汇总表 + 一段 200 字的清洗说明，中途不查教程。

第 14 天 • 11/15 • 今日知识点回顾

★W5 总验收：给一份脏 CSV，独立清洗成可分析表并说清每一步

目标：90 分钟内从 orders_dirty.csv 走到一份能交给别人的汇总表 + 一段 200 字的清洗说明，中途不查教程。

对着 B 站视频看：视频讲到的每一句，这张 PPT 里都有一条；复习时只翻这份，不用再回去翻分P。

今天结束时，你应该能做到

验收标准（做到这句就算过关）

- 90 分钟内从 orders_dirty.csv 走到一份能交给别人的汇总表 + 一段 200 字的清洗说明，中途不查教程。

今天的做法（顺序别颠倒）

- 题库地址：file:///D:/DSH工作区/00-学习中心/60-练习环境/Python-Env/data
- 做完记一句：今天卡在哪、错在哪 → 写进 3-每日打卡。

今天看哪几集、读哪一章

这一天的知识：3 集 • 合计约 20 分钟（边看边在本机敲一遍）

P28	Pandas2高级查询过滤	约 4 分钟
P29	Pandas2数据迭代操作	约 6 分钟
P30	Pandas2高级函数应用	约 10 分钟

教材（已下载到 20-教材与讲义\python-data\，断网也能看）

`https://raw.githubusercontent.com/datawhalechina/joyful-pandas/master/source/Content/ch1.rst`

回看第一章「列表推导式」那一节，检查自己是不是还在写 for + append

视频地址就在下面的「今天做哪些题」网页里，点一下直达分P。

知识点 1/5 • 验收的判据是「能复现」，不是「跑通一次」

- 一句话：同一份脚本换个数据目录还能跑出结果，才算真会；只在自己电脑上跑通一次不算。
- 代码：

```
import pathlib, sys
DATA = pathlib.Path(sys.argv[1]) if len(sys.argv) > 1 else pathlib.Path(
r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
print('数据目录：', DATA, DATA.exists())
```

★ 把路径写死在代码里，是「换个环境就跑不了」的第一大原因。

- 今天所有脚本都改成命令行参数或环境变量。

知识点 2/5 • 清洗口径要写成「可复查的数字」，不是形容词

- 一句话：不说「有很多重复」，说「37 个重复单号，按 first 保留」。
- 代码：

```
LOG = {
'raw_rows': 1200,
'dup_order_id': 37,
'amount_unparsable': 35,
'city_missing': 56,
'time_cn_format': 145,
'qty_negative': 25,
'clean_rows': 1163,
```

这一页只讲「清洗口径要写成「可复查的数字」，不是形容词」。讲完就翻页，下一页是另一个知识点。

知识点 2/5 · 清洗口径要写成「可复查的数字」，不是形容词（续）

- }

```
for k, v in LOG.items():
print(f'{k:20s}{v:>6d}')
```

★ 这七个数字就是今天要背下来的「清洗口径」。面试被问「你洗过什么脏数据」时，报数字比报函数名有说服力。

知识点 3/5 • 汇总表要能回答一个业务问题

- 一句话：每个输出表都要对应一句「谁看了会做什么决定」。
- 代码：

```
paid = df[df['status'] == 'paid']
out = (paid.groupby(['city', 'category'])
.agg(单数=('order_id', 'count'), 实付=('amount', 'sum'))
.round(2).reset_index())
print(out.sort_values('实付', ascending=False).head(8).to_string(index=False))
```

★ 「城市 × 品类」这张表的业务问题：哪些城市在哪个品类上还有空间?

- 南京 60,532 排第一，但如果它只在数码上强、家居弱，那就是铺货问题。

知识点 4/5 • 验收脚本的结构：一个文件、四段、能重跑

- 一句话：load / clean / analyze / report 四段，main 里只做编排，每段可单独测。
- 代码：

```
def main(argv):
data_dir = pathlib.Path(argv[1]) if len(argv) > 1 else DEFAULT
raw = load(data_dir / 'orders_dirty.csv')
clean, log = clean(raw)
tables = analyze(clean)
report(clean, tables, log, data_dir / 'w5_report')
print('完成：', log)
```

- （今天产出的 w5_report/ 目录里应该有：clean.csv、summary_city_category.csv、summary_category.csv、clean_log.json —— 四个文件，缺一个都算没做完。）

知识点 5/5 • 自查清单：这 8 条全过才算 W5 过关

- 一句话：逐条打勾，别靠「感觉学会了」。
- 代码：

```
CHECK = [
'能不看资料写出 groupby + agg + 命名聚合',
'能说清 agg 与 transform 的结果形状差别',
'能说清 merge 的 how 四种取值的差别',
'能解释为什么 loc[0:2] 是 3 行、iloc[0:2] 是 2 行',
'能把三种日期格式统一成 datetime 且不丢行',
'能写出 6 条清洗规则并报出每一条影响多少行',
'能用 pivot_table 做一张城市 × 品类的交叉表',
'能给出一份 200 字的清洗说明，不含形容词只有数字',
```

知识点 5/5 • 自查清单：这 8 条全过才算 W5 过关（续）

-]

★ 8 条里有任何一条说不清，明天补 —— W6 的 pivot_table 和时间序列都建立在这上面。

代码速查：今天这几段，要在本机跑通（1/7）

独立完成 w5_report.py（不许抄前面的代码），跑出四个文件：

结构示意，具体实现自己写

```
import json, pathlib, sys
```

```
import pandas as pd
```

```
DEFAULT = pathlib.Path(r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
```

```
def load(path):
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（2/7）

```
    return pd.read_csv(path, encoding='utf-8-sig')
```

```
def clean(df):
```

```
    log = {}
```

```
    # 金额 / 状态 / 城市 / 日期 / 去重 / 数量 —— 六条规则，每条都记数
```

```
    ...
```

```
    return df, log
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（3/7）

```
def analyze(df):
```

```
    paid = df[df['status'] == 'paid']
    return {
        'city_category': paid.groupby(['city', 'category'])
                            .agg(单数=('order_id', 'count'), 实付=('amount', 'sum'))
                            .round(2).reset_index(),
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（4/7）

```
        'category': paid.groupby('category')
```

```
                    .agg(单数=('order_id', 'count'), 实付=('amount', 'sum'),
```

```
                         客单价=('amount', 'mean'))
```

```
                    .round(2).reset_index(),
```

```
}
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（5/7）

```
def report(df, tables, log, outdir):
```

```
    outdir.mkdir(parents=True, exist_ok=True)
```

```
    df.to_csv(outdir / 'clean.csv', index=False, encoding='utf-8-sig')
```

for k, t in tables.items():

```
        t.to_csv(outdir / f'summary_{k}.csv', index=False, encoding='utf-8-sig')
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（6/7）

```
    (outdir / 'clean_log.json').write_text(
```

```
        json.dumps(log, ensure_ascii=False, indent=2), encoding='utf-8')
```

```
if __name__ == '__main__':
```

```
    d = pathlib.Path(sys.argv[1]) if len(sys.argv) > 1 else DEFAULT
```

```
raw = load(d / 'orders_dirty.csv')

clean_df, log = clean(raw)
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（7/7）

```
    report(clean_df, analyze(clean_df), log, d / 'w5_report')
```

```
print(log)
```

验收（做到才算过关，这是 W5 的总验收）：

① 四个文件都产出，clean.csv 是 1163 行；

② 品类汇总四个数字相加 = 422,436.00；

③ 200 字清洗说明能背下来讲，只含数字不含形容词；

④ 把脚本里的 DEFAULT 改成一个不存在的目录再跑一次，报错信息要能看懂。

合上视频，自测一遍（能打勾才算学会）

- [ ] 我能不看资料写出：把路径写死在代码里，是「换个环境就跑不了」的第一大原因。
- [ ] 我能不看资料写出：这七个数字就是今天要背下来的「清洗口径」。面试被问「你洗过什么脏数据」时，报数字比报函数名有说服力。
- [ ] 我能不看资料说出：「城市 × 品类」这张表的业务问题：哪些城市在哪个品类上还有空间？
- [ ] 我能不看资料写出：8 条里有任何一条说不清，明天补 —— W6 的 pivot_table 和时间序列都建立在这上面。
- [ ] 我能不看题源，把今天用到的每个函数「为什么要有它」说一遍
- [ ] 我能说出今天每个参数/返回值改了会怎样——换个写法结果差在哪

这些句子都是「差一个字、结果差一截」的地方；三个勾以上，今天就算过关。

今天收工前，先知道明天学什么

第 15 天 • pivot_table 透视表：index/columns/values/aggfunc 四件套

不用提前学。明天点「① 开始今天的学习」，会换上明天那份 PPT。

收工三件事（5 分钟）

- 今天没跑通的脚本存进 90-我的作业\，命名 2026-11-15-第14天-*.py
- 打卡表「③ 每日打卡」记一笔：今天卡在哪、错在哪（一句话就够）
- 错题明天重做一遍，练习数据在 60-练习环境\Python-Env\data

全阶段 · 第 15 天 · 11/16

pivot_table 透视表：indexcolumnsvaluesaggfunc 四件套

不用 Excel，30 秒内把 1163 行明细做成「城市 × 品类」的实付矩阵，并说出行列各是什么、空白怎么填。

第 15 天 • 11/16 • 今日知识点回顾

pivot_table 透视表：index/columns/values/aggfunc 四件套

目标：不用 Excel，30 秒内把 1163 行明细做成「城市 × 品类」的实付矩阵，并说出行列各是什么、空白怎么填。

对着 B 站视频看：视频讲到的每一句，这张 PPT 里都有一条；复习时只翻这份，不用再回去翻分P。

今天结束时，你应该能做到

验收标准（做到这句就算过关）

- 不用 Excel，30 秒内把 1163 行明细做成「城市 × 品类」的实付矩阵，并说出行列各是什么、空白怎么填。

今天的做法（顺序别颠倒）

- 题库地址：file:///D:/DSH工作区/00-学习中心/60-练习环境/Python-Env/data
- 做完记一句：今天卡在哪、错在哪 → 写进 3-每日打卡。

今天看哪几集、读哪一章

这一天的知识：2 集 • 合计约 33 分钟（边看边在本机敲一遍）

P33	Pandas2数据重塑透视	约 18 分钟
P33	【数据的合并和分组聚合】03数据分组聚合	约 15 分钟

教材（已下载到 20-教材与讲义\python-data\，断网也能看）

`https://raw.githubusercontent.com/datawhalechina/joyful-pandas/master/source/Content/ch5.rst`

第五章 变形：pivot / pivot_table / melt / crosstab 全在这一章

知识点 1/7 • pivot_table = groupby + unstack，一步出二维表

- 一句话：index 放行、columns 放列、values 放要算的数、aggfunc 放怎么算。
- 代码：

```
import pandas as pd, pathlib
DATA = pathlib.Path(r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
df = pd.read_csv(DATA / 'clean_v2.csv', encoding='utf-8-sig')
paid = df[df['status'] == 'paid']
pv = pd.pivot_table(paid, index='city', columns='category',
values='amount', aggfunc='sum', fill_value=0)
print(pv.shape)          # (9, 4)
print(pv.round(0).to_string())
```

这一页只讲「pivot_table = groupby + unstack，一步出二维表」。讲完就翻页，下一页是另一个知识点。

知识点 1/7 • pivot_table = groupby + unstack，一步出二维表（续）

★ 本机实测：9 个城市 × 4 个品类，行合计最高的是南京 60,532.00。

• 这个数必须和昨天 groupby(['city','category']).sum().unstack() 的结果完全一致 ——不一致说明有一个用了不同的筛选条件。

知识点 2/7 • aggfunc 可以给多个，也可以给 dict 分别指定

- 一句话：aggfunc=['sum','count'] 出多级列；aggfunc={'amount':'sum','order_id':'count'} 按列分。
- 代码：

```
pv2 = pd.pivot_table(paid, index='city', columns='category',
values='amount', aggfunc=['sum', 'count'], fill_value=0)
print(pv2.columns.tolist()[:4])
pv3 = pd.pivot_table(paid, index='city', columns='category',
values=['amount', 'order_id'],
aggfunc={'amount': 'sum', 'order_id': 'count'}, fill_value=0)
print(pv3.shape)
```

★ 多级列名导出 CSV 后表头是两行，Excel 打开看着乱。给业务看时用

- pv.columns = [f'{a}_{b}' for a, b in pv.columns] 拍平，或者只算一个指标。

知识点 3/7 • margins=True 加一行一列合计

- 一句话：margins=True 加「All」行列，margins_name 改名字。
- 代码：

```
pv4 = pd.pivot_table(paid, index='city', columns='category', values='amount',
aggfunc='sum', fill_value=0, margins=True, margins_name='合计')
print(pv4.round(0).to_string())
print('总计：', round(pv4.loc['合计', '合计'], 2))      # 422436.00
```

★ 422,436.00 这个数已经出现三次了（第 12 天 agg、昨天 unstack、今天 margins）。

- 三次都对上，才说明你对「paid」的定义没变。

知识点 4/7 • fill_value=0 与 dropna 的取舍

- 一句话：没成交的组合到底算 0 还是算「没有」——这是业务问题，不是技术问题。
- 代码：

```
a = pd.pivot_table(paid, index='city', columns='category', values='amount', aggfunc='sum')
print('不填: ', a.isna().sum().sum(), '个空')
b = pd.pivot_table(paid, index='city', columns='category', values='amount',
aggfunc='sum', fill_value=0)
print('填 0: ', b.isna().sum().sum(), '个空')
```

★ 算「合计」时填 0 和不填结果一样（sum 跳过 nan），但算「平均客单价」时填 0 会把均价拉低 —— 均价场景绝对不要 fill_value=0。

知识点 5/7 • crosstab：专做「计数」的透视表，比 pivot_table 短

- 一句话：pd.crosstab(df['city'], df['category']) 直接数每个组合多少单。
- 代码：

```
ct = pd.crosstab(paid['city'], paid['category'], margins=True, margins_name='合计')
print(ct.to_string())
ct2 = pd.crosstab(paid['city'], paid['category'], values=paid['amount'],
aggfunc='sum', margins=True, margins_name='合计')
print(ct2.round(0).to_string())
```

★ crosstab 默认 aggfunc='count'（数行数），pivot_table 默认 'mean' ——默认值不同是新手最容易踩的坑：pivot_table 忘了写 aggfunc 会得到平均数。

这一页只讲「crosstab：专做「计数」的透视表，比 pivot_table 短」。讲完就翻页，下一页是另一个知识点。

知识点 6/7 • pivot 和 pivot_table 的区别

- 一句话：pivot 只重塑、不聚合（索引必须唯一）； pivot_table 会聚合（索引可重复）。
- 代码：

```
small = paid.head(3)[['city', 'category', 'amount']]
print(small.pivot(index='city', columns='category', values='amount').to_string())
# 有重复组合时 pivot 报：ValueError: Index contains duplicate entries, cannot reshape
```

★ 真实数据几乎一定有重复组合，所以日常用 pivot_table，pivot 只在「确定唯一」的小表上用。报错原文记住：Index contains duplicate entries。

这一页只讲「pivot 和 pivot_table 的区别」。讲完就翻页，下一页是另一个知识点。

知识点 7/7 • melt：透视表的逆操作，宽表变长表

- 一句话：pd.melt(pv.reset_index(), id_vars='city') 把列名变成一列「品类」。
- 代码：

```
long = pd.melt(pv.reset_index(), id_vars='city',
var_name='category', value_name='amount')
print(long.shape, long.head(3).to_string(index=False))
```

- （9 城市 × 4 品类 = 36 行长表。做 BI 看板时几乎都要长表，Tableau 尤其吃长表 —— 第 24 天会用到这个转换。）

代码速查：今天这几段，要在本机跑通（1/6）

写 pivot_practice.py，产出四张表并存 CSV：

```
import pandas as pd, pathlib
```

```
DATA = pathlib.Path(r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
```

```
df = pd.read_csv(DATA / 'clean_v2.csv', encoding='utf-8-sig')
```

paid = df[df['status'] == 'paid']

```
out = DATA / 'pivot_out'; out.mkdir(exist_ok=True)
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（2/6）

1) 城市 × 品类 实付矩阵（9×4）

```
pv = pd.pivot_table(paid, index='city', columns='category', values='amount',
```

```
                    aggfunc='sum', fill_value=0, margins=True, margins_name='合计')
```

```
pv.round(2).to_csv(out / 'city_category_amount.csv', encoding='utf-8-sig')
```

```
print(pv.round(0).to_string())
```

代码速查：今天这几段，要在本机跑通（3/6）

```
print('总计', round(pv.loc['合计', '合计'], 2))     # 422436.00
```

2) 城市 × 品类 单数矩阵（用 crosstab）

```
ct = pd.crosstab(paid['city'], paid['category'], margins=True, margins_name='合计')
```

```
ct.to_csv(out / 'city_category_count.csv', encoding='utf-8-sig')
```

```
print(ct.to_string())
```

代码速查：今天这几段，要在本机跑通（4/6）

```
# 3) 城市 × 状态 实付（含 pending/refunded，看退款占比）
```

```
pv3 = pd.pivot_table(df, index='city', columns='status', values='amount',
```

```
                     aggfunc='sum', fill_value=0)
```

```
pv3['退款占比%'] = (pv3['refunded'] / pv3.sum(axis=1) * 100).round(1)
```

```
print(pv3.round(0).to_string())
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（5/6）

4) 宽表转长表

```
long = pd.melt(pv.reset_index().query("city != '合计'"), id_vars='city',
```

```
               var_name='category', value_name='amount')
```

long = long.query("category != '合计'")

```
long.to_csv(out / 'city_category_long.csv', index=False, encoding='utf-8-sig')
```

```
print('长表', long.shape)
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（6/6）

验收（做到才算过关）：

① 第 1 题 9×4 且总计 = 422,436.00；

② 第 3 题能说出哪个城市的退款占比最高、是多少；

③ 能解释 pivot_table 不写 aggfunc 会得到什么（默认 mean），以及为什么危险。

最容易错的地方

★ 真实数据几乎一定有重复组合，所以日常用 pivot_table，pivot 只在「确定唯一」的小表上用。报错原文记住：Index contains duplicate entries。

这些句子都是「差一个字、结果差一截」的地方，考前只翻这一页也值。

合上视频，自测一遍（能打勾才算学会）

- [ ] 我能不看资料写出：422,436.00 这个数已经出现三次了（第 12 天 agg、昨天 unstack、今天 margins）。
- [ ] 我能不看资料写出：算「合计」时填 0 和不填结果一样（sum 跳过 nan），但算「平均客单价」时填 0 会把均价拉低 —— 均价场景绝对不要 fill_value=0。
- [ ] 我能不看资料说出：一句话：pivot 只重塑、不聚合（索引必须唯一）；pivot_table 会聚合（索引可重复）。
- [ ] 我能不看题源，把今天用到的每个函数「为什么要有它」说一遍
- [ ] 我能说出今天每个参数/返回值改了会怎样——换个写法结果差在哪
- [ ] 今天这几段代码我在本机都跑通、并看清了输出

这些句子都是「差一个字、结果差一截」的地方；三个勾以上，今天就算过关。

今天收工前，先知道明天学什么

第 16 天 • 时间序列：to_datetime、resample、rolling、同比环比

不用提前学。明天点「① 开始今天的学习」，会换上明天那份 PPT。

收工三件事（5 分钟）

- 今天没跑通的脚本存进 90-我的作业\，命名 2026-11-16-第15天-*.py
- 打卡表「③ 每日打卡」记一笔：今天卡在哪、错在哪（一句话就够）
- 错题明天重做一遍，练习数据在 60-练习环境\Python-Env\data

全阶段 · 第 16 天 · 11/17

时间序列：to_datetime、resample、rolling、同比环比

把订单按周、按月各汇总一次，并算出环比增长率；对 quant_returns 算滚动 20 日波动率。

第 16 天 • 11/17 • 今日知识点回顾

时间序列：to_datetime、resample、rolling、同比环比

目标：把订单按周、按月各汇总一次，并算出环比增长率；对 quant_returns 算滚动 20 日波动率。

对着 B 站视频看：视频讲到的每一句，这张 PPT 里都有一条；复习时只翻这份，不用再回去翻分P。

今天结束时，你应该能做到

验收标准（做到这句就算过关）

- 把订单按周、按月各汇总一次，并算出环比增长率；对 quant_returns 算滚动 20 日波动率。

今天的做法（顺序别颠倒）

- 题库地址：file:///D:/DSH工作区/00-学习中心/60-练习环境/Python-Env/data
- 做完记一句：今天卡在哪、错在哪 → 写进 3-每日打卡。

今天看哪几集、读哪一章

这一天的知识：3 集 • 合计约 48 分钟（边看边在本机敲一遍）

P35	Pandas2时间序列数据	约 16 分钟
P36	Pandas2数据可视化	约 20 分钟
P37	【案例】时间序列	约 12 分钟

教材（已下载到 20-教材与讲义\python-data\，断网也能看）

`https://raw.githubusercontent.com/datawhalechina/joyful-pandas/master/source/Content/ch8.rst`

第八章 文本数据（开头讲 to_datetime 的部分先看）

视频地址就在下面的「今天做哪些题」网页里，点一下直达分P。

知识点 1/7 • 时间列必须是 datetime64，否则 resample 用不了

- 一句话：先 to_datetime，再 set_index，最后 resample —— 顺序不能乱。
- 代码：

```
import pandas as pd, pathlib
DATA = pathlib.Path(r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
df = pd.read_csv(DATA / 'clean_v2.csv', encoding='utf-8-sig', parse_dates=['create_time'])
print(df['create_time'].dtype)            # datetime64[ns]
print(df['create_time'].min(), df['create_time'].max())
```

★ 本机实测范围：2026-08-01 到 2026-10-29，共 90 天。

- 如果 dtype 是 object，resample 会报 TypeError: Only valid with DatetimeIndex,TimedeltaIndex or PeriodIndex —— 看到这个报错就是没转 datetime。

知识点 2/7 • resample：按周/月重采样，等价于「按时间分组的 groupby」

- 一句话：df.set_index('时间').resample('W')['金额'].sum()。
- 代码：

```
s = df.set_index('create_time')
weekly = s.resample('W')['amount'].sum().round(2)
print(weekly.head().to_string())
monthly = s.resample('ME')['amount'].sum().round(2)      # ME = 月末
print(monthly.to_string())
```

★ pandas 3.0 里 'M' 已弃用，必须写 'ME'（月末）或 'MS'（月初），否则告警

- FutureWarning: 'M' is deprecated, use 'ME'。同理 'Y'→'YE'、'Q'→'QE'。

知识点 3/7 • 环比与同比：pct_change 一行搞定

- 一句话：s.pct_change() 算相邻两期增长率；同比要看 12 期前，用 shift(12)。
- 代码：

```
monthly = s.resample('ME')['amount'].sum()
mom = (monthly.pct_change() * 100).round(2)
print(pd.DataFrame({'实付': monthly.round(0), '环比%': mom}).to_string())
print('首月无环比：', mom.isna().sum())
```

★ 第一期一定是 nan（没有上一期）。填 0 会让「首月增长 0%」这个假结论进报告 —— 保留 nan，或者在报告里写「首月为基期」。

知识点 4/7 • rolling：滑动窗口，算移动平均与滚动波动率

- 一句话：s.rolling(20).std() 是「最近 20 期」的标准差。
- 代码：

```
q = pd.read_csv(DATA / 'quant_returns.csv', encoding='utf-8-sig', parse_dates=['date'])
q = q.set_index('date')
```

知识点 4/7 • rolling：滑动窗口，算移动平均与滚动波动率（续）

- q['vol20'] = q['strategy_ret'].rolling(20).std() * (252 ** 0.5) * 100

```
print(q[['strategy_ret', 'vol20']].dropna().head().round(4).to_string())
print('最新 20 日年化波动%: ', round(q['vol20'].dropna().iloc[-1], 2))
```

★ 前 19 行一定是 nan（窗口没满）。rolling(20).std() 默认 ddof=1（样本标准差），和 numpy 的 np.std 默认 ddof=0 不一样 —— 两套算出来的数会有细微差别。

知识点 5/7 • quant 数据实测：356 行、总收益 89.53%

- 一句话：本机 quant_returns.csv 是 2025-01 起的日频回测，策略总收益 89.53%，基准 62.37%。
- 代码：

```
q2 = pd.read_csv(DATA / 'quant_returns.csv', encoding='utf-8-sig', parse_dates=['date'])
print(q2.shape, q2['date'].min(), q2['date'].max())     # (356,)
total = (1 + q2['strategy_ret']).prod() - 1
bench = (1 + q2['bench_ret']).prod() - 1
print('策略', round(total * 100, 2), '%  基准', round(bench * 100, 2), '%')
```

- （本机实测：356 个交易日，策略总收益 89.53%、基准 62.37%；
- 最大回撤 -18.59%、Sharpe 2.73、胜率 57.0%、年化波动 17.15%。
- 注意：总收益要用累乘 (1+r).prod()-1，**不是** r.sum() ——用求和算收益率是新手最常犯的错。）

知识点 6/7 • 日期抽取：dt 访问器

- 一句话：s.dt.year / .month / .dayofweek / .hour 直接从 datetime 列取成分。
- 代码：

```
d = df['create_time']
print(d.dt.year.value_counts().to_dict())
print(d.dt.month.value_counts().sort_index().to_dict())
print(d.dt.dayofweek.value_counts().sort_index().to_dict())   # 0=周一
df['is_weekend'] = d.dt.dayofweek >= 5
```

★ dt.dayofweek 是 0=周一 … 6=周日（不是 1=周一）。dt.day 是「几号」不是「星期几」，这两个搞混会算出一堆没意义的「按星期分布」。

知识点 7/7 • 时间对齐：两个序列要按同一个日历合并

- 一句话：reindex 到统一日期轴，缺的补 nan 或前值填充。
- 代码：

```
a = q['strategy_ret'].resample('ME').sum()
b = q['bench_ret'].resample('ME').sum()
cmp = pd.concat([a.rename('策略'), b.rename('基准')], axis=1)
print(cmp.round(4).to_string())
print('月胜率: ', round((cmp['策略'] > cmp['基准']).mean() * 100, 1), '%')
```

★ 直接 pd.concat 两个不同长度的 Series 会按索引并集对齐，缺的地方是 nan ——不会报错，但会让「月胜率」的分母变大。先 resample 到同一频率再比。

代码速查：今天这几段，要在本机跑通（1/6）

写 timeseries_practice.py，产出四份时间维度的表：

```
import pandas as pd, pathlib
```

```
DATA = pathlib.Path(r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
```

```
out = DATA / 'ts_out'; out.mkdir(exist_ok=True)
```

```
df = pd.read_csv(DATA / 'clean_v2.csv', encoding='utf-8-sig', parse_dates=['create_time'])
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（2/6）

```
paid = df[df['status'] == 'paid'].set_index('create_time')
```

1) 周汇总 + 环比

```
weekly = paid['amount'].resample('W').sum()
```

```
print(pd.DataFrame({'周实付': weekly.round(0), '环比%': (weekly.pct_change() * 100).round(1)}).to_string())
```

2) 月汇总（ME）

```
monthly = paid['amount'].resample('ME').sum()
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（3/6）

```
monthly.to_csv(out / 'monthly.csv', encoding='utf-8-sig')
```

```
print(monthly.round(2).to_string())
```

3) 按星期几看订单分布

```
print(df['create_time'].dt.dayofweek.value_counts().sort_index().to_string())
```

4) quant：滚动 20 日年化波动 + 月度策略 vs 基准

```
q = pd.read_csv(DATA / 'quant_returns.csv', encoding='utf-8-sig',
```

代码速查：今天这几段，要在本机跑通（4/6）

```
                parse_dates=['date']).set_index('date')
```

```
q['vol20'] = q['strategy_ret'].rolling(20).std() * (252 ** 0.5) * 100
```

cmp = pd.concat([

```
    q['strategy_ret'].resample('ME').sum().rename('策略'),
```

```
    q['bench_ret'].resample('ME').sum().rename('基准'),
```

], axis=1)

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（5/6）

```
cmp['超额'] = (cmp['策略'] - cmp['基准']).round(4)
```

```
cmp.round(4).to_csv(out / 'monthly_strategy_vs_bench.csv', encoding='utf-8-sig')
```

```
print(cmp.round(4).to_string())
```

```
print('最新年化波动%: ', round(q['vol20'].dropna().iloc[-1], 2))
```

验收（做到才算过关）：

① 周汇总第一行是 nan 环比（不是 0）；

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（6/6）

② 月度表能报出三个月各多少、哪个月最高；

```
③ quant 月度表里「策略」列相加 ≈ 89.53%（累乘口径），能解释为什么月度和 ≠ 总收益。
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

最容易错的地方

如果 dtype 是 object，resample 会报 TypeError: Only valid with DatetimeIndex,TimedeltaIndex or PeriodIndex —— 看到这个报错就是没转 datetime。

★ 前 19 行一定是 nan（窗口没满）。rolling(20).std() 默认 ddof=1（样本标准差），和 numpy 的 np.std 默认 ddof=0 不一样 —— 两套算出来的数会有细微差别。

注意：总收益要用累乘 (1+r).prod()-1，**不是** r.sum() ——用求和算收益率是新手最常犯的错。）

★ 直接 pd.concat 两个不同长度的 Series 会按索引并集对齐，缺的地方是 nan ——不会报错，但会让「月胜率」的分母变大。先 resample 到同一频率再比。

这些句子都是「差一个字、结果差一截」的地方，考前只翻这一页也值。

合上视频，自测一遍（能打勾才算学会）

- [ ] 我能不看资料写出：pandas 3.0 里 'M' 已弃用，必须写 'ME'（月末）或 'MS'（月初），否则告警
- [ ] 我能不看资料写出：第一期一定是 nan（没有上一期）。填 0 会让「首月增长 0%」这个假结论进报告 —— 保留 nan，或者在报告里写「首月为基期」。
- [ ] 我能不看资料写出：dt.dayofweek 是 0=周一 … 6=周日（不是 1=周一）。dt.day 是「几号」不是「星期几」，这两个搞混会算出一堆没意义的「按星期分布」。
- [ ] 我能不看资料说出：一句话：先 to_datetime，再 set_index，最后 resample —— 顺序不能乱。
- [ ] 我能不看资料说出：注意：总收益要用累乘 (1+r).prod()-1，**不是** r.sum() ——用求和算收益率是新手最常犯的错。)
- [ ] 我能不看题源，把今天用到的每个函数「为什么要有它」说一遍

这些句子都是「差一个字、结果差一截」的地方；三个勾以上，今天就算过关。

今天收工前，先知道明天学什么

第 17 天 • apply/map/lambda + 多级索引：什么时候该用、什么时候别用

不用提前学。明天点「① 开始今天的学习」，会换上明天那份 PPT。

收工三件事（5 分钟）

- 今天没跑通的脚本存进 90-我的作业\，命名 2026-11-17-第16天-*.py
- 打卡表「③ 每日打卡」记一笔：今天卡在哪、错在哪（一句话就够）
- 错题明天重做一遍，练习数据在 60-练习环境\Python-Env\data

全阶段 · 第 17 天 · 11/18

applymaplambda + 多级索引：什么时候该用、什么时候别用

能用多级索引取出任意切片。

第 17 天 • 11/18 • 今日知识点回顾

apply/map/lambda + 多级索引：什么时候该用、什么时候别用

目标：判断一段代码「该向量化还是该 apply」，并能用多级索引取出任意切片。

对着 B 站视频看：视频讲到的每一句，这张 PPT 里都有一条；复习时只翻这份，不用再回去翻分P。

今天结束时，你应该能做到

验收标准（做到这句就算过关）

- 判断一段代码「该向量化还是该 apply」，并能用多级索引取出任意切片。

今天的做法（顺序别颠倒）

- 题库地址：file:///D:/DSH工作区/00-学习中心/60-练习环境/Python-Env/data
- 做完记一句：今天卡在哪、错在哪 → 写进 3-每日打卡。

今天看哪几集、读哪一章

这一天的知识：3 集 • 合计约 28 分钟（边看边在本机敲一遍）

P30	Pandas2高级函数应用	约 10 分钟
P29	Pandas2数据迭代操作	约 6 分钟
P84	函数进阶之函数作为参数传递	约 12 分钟

教材（已下载到 20-教材与讲义\python-data\，断网也能看）

`https://raw.githubusercontent.com/datawhalechina/joyful-pandas/master/source/Content/ch3.rst`

第三章 索引：多层索引的 loc / xs / IndexSlice 三种取法

视频地址就在下面的「今天做哪些题」网页里，点一下直达分P。

知识点 1/7 • apply 是「最后手段」，先问能不能向量化

- 一句话：能用 .str / .dt / 算术 / where 解决的，绝不用 apply。
- 代码：

```
import pandas as pd, numpy as np, pathlib, time
DATA = pathlib.Path(r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
df = pd.read_csv(DATA / 'clean_v2.csv', encoding='utf-8-sig')
# 慢：apply + 自定义函数
t0 = time.perf_counter()
a = df['amount'].apply(lambda x: x * 0.9 if pd.notna(x) else x)
t1 = time.perf_counter()
# 快：向量化
b = df['amount'].where(df['amount'].isna(), df['amount'] * 0.9)
t2 = time.perf_counter()
print(f'apply {t1-t0:.4f}s  where {t2-t1:.4f}s  结果相同 {a.equals(b)}')
```

这一页只讲「apply 是「最后手段」，先问能不能向量化」。讲完就翻页，下一页是另一个知识点。

知识点 1/7 • apply 是「最后手段」，先问能不能向量化（续）

★ 1163 行时 apply 可能只慢 3–5 倍，看着无所谓；到 100 万行时是 50 倍以上。

- apply 本质是 Python 循环，向量化走的是 C。

知识点 2/7 • map：按字典做值替换，比 replace 更可控

- 一句话：s.map({'M': '男', 'F': '女'}) 找不到的变 nan —— 这一点很关键。
- 代码：

```
u = pd.read_csv(DATA / 'users_dirty.csv', encoding='utf-8-sig')
print(u['gender'].value_counts(dropna=False).to_dict())
m = {'M': '男', 'F': '女', '男': '男', '女': '女'}
u['g2'] = u['gender'].map(m)
print(u['g2'].value_counts(dropna=False).to_dict())
```

★ 本机实测：gender 有六种写法（男/女/M/F/未知/空），map 之后「未知」和空变成 nan。想保留成「未知」就写 m.get(x, '未知') 用 apply，或者先在字典里补 {'未知': '未知'}。map 的「找不到→nan」是隐式行为，最容易悄悄丢掉一类数据。

这一页只讲「map：按字典做值替换，比 replace 更可控」。讲完就翻页，下一页是另一个知识点。

知识点 3/7 • apply 的正确用法：一行里要看好几列

- 一句话：df.apply(func, axis=1) 把每一行当 Series 传给函数 —— 这是向量化做不到的场景。
- 代码：

知识点 3/7 • apply 的正确用法：一行里要看好几列（续）

```
def level(r):
if pd.isna(r['amount']):
return '金额缺失'
if r['amount'] >= 2000:
return '高客单'
if r['amount'] >= 500:
return '中客单'
return '低客单'
df['level'] = df.apply(level, axis=1)
print(df['level'].value_counts().to_dict())
```

> **★ axis=1 比 axis=0 慢得多（逐行构造 Series）。数据超过 10 万行时，改用 np.select 或 pd.cut —— 下面两条就是替代方案。**

知识点 4/7 • 两个向量化替代：np.select 与 pd.cut

- 一句话：多条件分支用 np.select；等距分箱用 pd.cut；分位分箱用 pd.qcut。
- 代码：

```
cond = [df['amount'].isna(), df['amount'] >= 2000, df['amount'] >= 500]
df['level2'] = np.select(cond, ['金额缺失', '高客单', '中客单'], default='低客单')
print((df['level'] == df['level2']).mean())          # 1.0 完全一致
df['bin'] = pd.cut(df['amount'], bins=[0, 500, 2000, np.inf],
labels=['低', '中', '高'], right=False)
print(df['bin'].value_counts(dropna=False).to_dict())
```

★ np.select 的条件是「从上往下第一个为 True 的生效」，顺序错了结果就错 ——和 if/elif 一样的规则。pd.cut 的 right=False 表示左闭右开 [0,500)，默认 right=True 是 (0,500]，边界值会掉进不同的桶。

知识点 5/7 • 多级索引：groupby 两个键之后发生了什么

- 一句话：groupby(['city','category']) 得到一个 MultiIndex，行标签是元组。
- 代码：

```
paid = df[df['status'] == 'paid']
g = paid.groupby(['city', 'category'])['amount'].sum()
print(type(g.index))                    # MultiIndex
print(g.index.names)                    # ['city', 'category']
print(g.head(3).to_string())
print(g.loc[('北京', '数码')])          # 按元组取单个值
```

★ 单个键用 g.loc['北京'] 也能取，但会返回一个 Series（4 个品类）。

- 要取「北京 + 数码」必须写成元组 ('北京', '数码')。

知识点 6/7 • xs 与 IndexSlice：取某一层的切片

- 一句话：g.xs('数码', level='category') 取所有城市在数码上的值。
- 代码：

```
print(g.xs('数码', level='category').sort_values(ascending=False).to_string())
print(g.loc[(slice(None), '数码'), ].sort_values(ascending=False).head(3).to_string())
idx = pd.IndexSlice
print(g.loc[idx[:, '数码']].head(3).to_string())
```

★ 三种写法等价，但 xs 最短、最不容易写错。IndexSlice 适合「同时切两层」的复杂场景（比如 loc[idx['北京':'上海', '数码':'家居']]）。

知识点 7/7 • reset_index / unstack / stack：三层索引的三个出口

- 一句话：reset_index 变平表、unstack 变宽表、stack 变长表。
- 代码：

```
print(g.reset_index().head(3).to_string(index=False))   # 变回普通列
print(g.unstack(fill_value=0).shape)                    # (9, 4) 宽表
print(g.unstack().stack().shape)                        # 又变回来（nan 会被丢）
```

- （stack() 默认 dropna=True，会**丢掉** nan 的组合 ——unstack().stack() 不一定还原成原样。要严格还原写 stack(dropna=False)。）

代码速查：今天这几段，要在本机跑通（1/7）

写 apply_practice.py，同时验证「两种写法结果一致」：

```
import pandas as pd, numpy as np, pathlib, time
```

```
DATA = pathlib.Path(r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
```

```
df = pd.read_csv(DATA / 'clean_v2.csv', encoding='utf-8-sig')
```

1) apply vs 向量化 计时

t0 = time.perf_counter()

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（2/7）

```
a = df['amount'].apply(lambda x: round(x * 0.9, 2) if pd.notna(x) else x)
```

t1 = time.perf_counter()

b = (df['amount'] * 0.9).round(2)

t2 = time.perf_counter()

```
print(f'apply {t1-t0:.4f}s  vector {t2-t1:.4f}s  equal={a.equals(b)}')
```

```
# 2) 分档：apply / np.select / pd.cut 三种写法对齐
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（3/7）

```
def level(r):
```

if pd.isna(r['amount']): return '缺失'

if r['amount'] >= 2000: return '高'

if r['amount'] >= 500: return '中'

return '低'

```
df['lv_apply'] = df.apply(level, axis=1)
```

```
df['lv_select'] = np.select(
```

代码速查：今天这几段，要在本机跑通（4/7）

```
    [df['amount'].isna(), df['amount'] >= 2000, df['amount'] >= 500],
```

['缺失', '高', '中'], default='低')

```
print('apply==select: ', (df['lv_apply'] == df['lv_select']).mean())
```

```
print(df['lv_select'].value_counts().to_dict())
```

3) map 归并 gender

```
u = pd.read_csv(DATA / 'users_dirty.csv', encoding='utf-8-sig')
```

代码速查：今天这几段，要在本机跑通（5/7）

```
m = {'M': '男', 'F': '女', '男': '男', '女': '女', '未知': '未知'}
```

u['g2'] = u['gender'].map(m)

```
print('map 后分布: ', u['g2'].value_counts(dropna=False).to_dict())
```

```
print('map 丢掉的行: ', u['g2'].isna().sum())
```

4) 多级索引三种取法

paid = df[df['status'] == 'paid']

代码速查：今天这几段，要在本机跑通（6/7）

```
g = paid.groupby(['city', 'category'])['amount'].sum()
```

```
print(g.xs('数码', level='category').sort_values(ascending=False).head(3).to_string())
```

```
print(g.reset_index().shape, g.unstack(fill_value=0).shape)
```

验收（做到才算过关）：

① 第 1 题 equal=True，并报出两种写法的耗时比；

② 第 2 题 apply 与 np.select 结果 100% 一致；

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（7/7）

③ 第 3 题能报出 map 之后有多少行变 nan（就是「未知」+ 空的那些）。

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

最容易错的地方

★ np.select 的条件是「从上往下第一个为 True 的生效」，顺序错了结果就错 ——和 if/elif 一样的规则。pd.cut 的 right=False 表示左闭右开 [0,500)，默认 right=True 是 (0,500]，边界值会掉进不同的桶。

★ 三种写法等价，但 xs 最短、最不容易写错。IndexSlice 适合「同时切两层」的复杂场景（比如 loc[idx['北京':'上海', '数码':'家居']]）。

这些句子都是「差一个字、结果差一截」的地方，考前只翻这一页也值。

合上视频，自测一遍（能打勾才算学会）

- [ ] 我能不看资料写出：1163 行时 apply 可能只慢 3–5 倍，看着无所谓；到 100 万行时是 50 倍以上。
- [ ] 我能不看资料写出：单个键用 g.loc['北京'] 也能取，但会返回一个 Series（4 个品类）。
- [ ] 我能不看资料说出：要取「北京 + 数码」必须写成元组 ('北京', '数码')。
- [ ] 我能不看题源，把今天用到的每个函数「为什么要有它」说一遍
- [ ] 我能说出今天每个参数/返回值改了会怎样——换个写法结果差在哪
- [ ] 今天这几段代码我在本机都跑通、并看清了输出

这些句子都是「差一个字、结果差一截」的地方；三个勾以上，今天就算过关。

今天收工前，先知道明天学什么

第 18 天 • 可视化：matplotlib 中文设置 + 三张有业务含义的图

不用提前学。明天点「① 开始今天的学习」，会换上明天那份 PPT。

收工三件事（5 分钟）

- 今天没跑通的脚本存进 90-我的作业\，命名 2026-11-18-第17天-*.py
- 打卡表「③ 每日打卡」记一笔：今天卡在哪、错在哪（一句话就够）
- 错题明天重做一遍，练习数据在 60-练习环境\Python-Env\data

全阶段 · 第 18 天 · 11/19

可视化：matplotlib 中文设置 + 三张有业务含义的图

能配一句话结论，中文字体不乱码。

第 18 天 • 11/19 • 今日知识点回顾

可视化：matplotlib 中文设置 + 三张有业务含义的图

目标：产出 3 张图（趋势 / 结构 / 对比），每张图能配一句话结论，中文字体不乱码。

对着 B 站视频看：视频讲到的每一句，这张 PPT 里都有一条；复习时只翻这份，不用再回去翻分P。

今天结束时，你应该能做到

验收标准（做到这句就算过关）

- 产出 3 张图（趋势 / 结构 / 对比），每张图能配一句话结论，中文字体不乱码。

今天的做法（顺序别颠倒）

- 题库地址：file:///D:/DSH工作区/00-学习中心/60-练习环境/Python-Env/data
- 做完记一句：今天卡在哪、错在哪 → 写进 3-每日打卡。

今天看哪几集、读哪一章

这一天的知识：3 集 • 合计约 42 分钟（边看边在本机敲一遍）

P3	matplotlib折线图基本使用	约 16 分钟
P9	matplotlib常用统计图	约 12 分钟
P11	matplotlib直方图	约 14 分钟

教材（已下载到 20-教材与讲义\python-data\，断网也能看）

`https://raw.githubusercontent.com/datawhalechina/joyful-pandas/master/source/Content/ch2.rst`

第二章末尾：Series.plot / DataFrame.plot 的常用参数

视频地址就在下面的「今天做哪些题」网页里，点一下直达分P。

知识点 1/7 • 中文字体：一行设置，否则全是方框

- 一句话：plt.rcParams['font.sans-serif'] = ['Microsoft YaHei'] + axes.unicode_minus=False。
- 代码：

```
import matplotlib
matplotlib.use('Agg')          # 不弹窗，直接存文件
import matplotlib.pyplot as plt
plt.rcParams['font.sans-serif'] = ['Microsoft YaHei']
plt.rcParams['axes.unicode_minus'] = False
plt.rcParams['figure.dpi'] = 120
```

★ 不设 font.sans-serif 时中文全是方框（MissingGlyphWarning）；

- 不设 axes.unicode_minus=False 时负号也是方框 —— 画带负值的折线必踩。

这一页只讲「中文字体：一行设置，否则全是方框」。讲完就翻页，下一页是另一个知识点。

知识点 2/7 • 图 1 趋势图：月度实付折线 + 标注

- 一句话：折线图回答「在变好还是变坏」，一定要标出最高点和最低点。
- 代码：

这一页只讲「图 1 趋势图：月度实付折线 + 标注」。讲完就翻页，下一页是另一个知识点。

知识点 2/7 • 图 1 趋势图：月度实付折线 + 标注（续）

```
import pandas as pd, pathlib
DATA = pathlib.Path(r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
df = pd.read_csv(DATA / 'clean_v2.csv', encoding='utf-8-sig', parse_dates=['create_time'])
paid = df[df['status'] == 'paid'].set_index('create_time')
m = paid['amount'].resample('ME').sum()
fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(m.index.strftime('%Y-%m'), m.values, marker='o', color='#0F2E35')
ax.set_title('月度实付金额（paid 订单）')
ax.set_ylabel('元')
ax.grid(alpha=.3)
top = m.idxmax()
ax.annotate(f'最高 {m.max():,.0f}', xy=(m.index.get_loc(top), m.max()),
xytext=(0, 12), textcoords='offset points', ha='center', color='#C0392B')
fig.tight_layout(); fig.savefig(DATA / 'fig1_trend.png'); plt.close(fig)
```

这一页只讲「图 1 趋势图：月度实付折线 + 标注」。讲完就翻页，下一页是另一个知识点。

知识点 2/7 • 图 1 趋势图：月度实付折线 + 标注（续）

> ★ 本机数据只有 3 个月（2026-08 / 09 / 10），折线只有 3 个点 ——这种时候**不要**画折线，画柱状图更诚实。图表选型跟着数据量走。

知识点 3/7 • 图 2 结构图：城市 × 品类堆叠条形，看谁贡献最大

- 一句话：堆叠条形回答「总量由哪几块组成」，横向条形比竖向更好读中文标签。
- 代码：

知识点 3/7 • 图 2 结构图：城市 × 品类堆叠条形，看谁贡献最大（续）

```
pv = pd.pivot_table(paid, index='city', columns='category',
values='amount', aggfunc='sum', fill_value=0)
pv = pv.loc[pv.sum(axis=1).sort_values().index]      # 按总量排序
fig, ax = plt.subplots(figsize=(8, 5))
pv.plot(kind='barh', stacked=True, ax=ax, width=.72)
ax.set_title('各城市实付构成（按品类）')
ax.set_xlabel('元'); ax.legend(title='品类', fontsize=9)
fig.tight_layout(); fig.savefig(DATA / 'fig2_structure.png'); plt.close(fig)
```

★ 横向条形（barh）时 y 轴是从下往上排，所以要先 sort_values() 升序，画出来才是「最大的在最上面」。忘了排序 = 图看着乱但没错，最难发现。

这一页只讲「图 2 结构图：城市 × 品类堆叠条形，看谁贡献最大」。讲完就翻页，下一页是另一个知识点。

知识点 4/7 • 图 3 对比图：漏斗或分布，二选一

- 一句话：对比图回答「差多少」，用条形；看分布形状，用直方图。
- 代码：

知识点 4/7 • 图 3 对比图：漏斗或分布，二选一（续）

```
e = pd.read_csv(DATA / 'events_dirty.csv', encoding='utf-8-sig')
step = ['view', 'cart', 'order', 'pay']
cnt = e.groupby('event')['user_id'].nunique().reindex(step)
fig, ax = plt.subplots(figsize=(7, 4))
ax.bar(step, cnt.values, color=['#0F2E35', '#2E7D8A', '#5BA3AE', '#9CC9D0'])
for i, v in enumerate(cnt.values):
ax.text(i, v, f'{v:,}\n{v/cnt.iloc[0]*100:.1f}%', ha='center', va='bottom', fontsize=9)
ax.set_title('用户转化漏斗（去重用户数）')
ax.set_ylim(0, cnt.max() * 1.25)
fig.tight_layout(); fig.savefig(DATA / 'fig3_funnel.png'); plt.close(fig)
```

★ 本机实测：view 500 / cart 150 / order 74 / pay 28，漏斗率 100% / 30.0% / 14.8% / 5.6% —— 这四个数要能和 events 表对得上。

这一页只讲「图 3 对比图：漏斗或分布，二选一」。讲完就翻页，下一页是另一个知识点。

知识点 5/7 • figsize 与 dpi：进简历的图要多大

- 一句话：figsize=(8,4) + dpi=150 → 1200×600 px，贴进 Word 不糊。
- 代码：
- fig, ax = plt.subplots(figsize=(8, 4), dpi=150)
- fig.savefig(DATA / 'x.png', bbox_inches='tight') # 裁掉多余白边

★ 不写 bbox_inches='tight'，图四周会留一大圈白 —— 贴进文档后看着很小。

- 这是「图看着糊/小」最常见的原因，不是 dpi 不够。

知识点 6/7 • 三张图的结论怎么写

- 一句话：每张图配一句话，格式是「谁 / 多少 / 所以呢」。
- 代码：

```
CONCL = {
'fig1_trend.png': '8–10 月实付逐月变化，最高月是 10 月，说明活动集中在季末。',
'fig2_structure.png': '南京实付 60,532 排第一，其中数码占大头，家居偏弱。',
'fig3_funnel.png': '浏览到加购转化 30%，加购到下单只有 14.8%，卡点在加购后。',
```

- }

- （结论里必须出现数字。「增长明显」「表现较好」这类词在面试里等于没说话。）

这一页只讲「三张图的结论怎么写」。讲完就翻页，下一页是另一个知识点。

知识点 7/7 • 图别只存 PNG，也存一份数据

- 一句话：图会过时，数据不会 —— 每张图旁边存一份它用的 CSV。
- 代码：
- pv.round(2).to_csv(DATA / 'fig2_data.csv', encoding='utf-8-sig')
- cnt.rename('users').to_csv(DATA / 'fig3_data.csv', encoding='utf-8-sig')

★ 面试官问「这张图的数据哪来的」，能当场打开 CSV 比能打开 PPT 有说服力。

代码速查：今天这几段，要在本机跑通（1/9）

```
写 viz_three.py，产出 3 张 PNG + 3 份数据 CSV + 一段结论：
```

```
import pathlib
```

```
import matplotlib
```

matplotlib.use('Agg')

```
import matplotlib.pyplot as plt
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（2/9）

```
import pandas as pd
```

```
plt.rcParams['font.sans-serif'] = ['Microsoft YaHei']
```

```
plt.rcParams['axes.unicode_minus'] = False
```

```
DATA = pathlib.Path(r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
```

```
OUT = DATA / 'figs'; OUT.mkdir(exist_ok=True)
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（3/9）

图1 月度实付（3 个月 → 用柱状，不用折线）

```
df = pd.read_csv(DATA / 'clean_v2.csv', encoding='utf-8-sig', parse_dates=['create_time'])
```

```
paid = df[df['status'] == 'paid'].set_index('create_time')
```

m = paid['amount'].resample('ME').sum()

```
fig, ax = plt.subplots(figsize=(7, 4), dpi=150)
```

```
ax.bar(m.index.strftime('%Y-%m'), m.values, color='#0F2E35', width=.5)
```

代码速查：今天这几段，要在本机跑通（4/9）

```
for i, v in enumerate(m.values):
```

```
    ax.text(i, v, f'{v:,.0f}', ha='center', va='bottom', fontsize=9)
```

```
ax.set_title('月度实付金额（paid）'); ax.set_ylabel('元')
```

ax.set_ylim(0, m.max() * 1.15)

```
fig.tight_layout(); fig.savefig(OUT / 'fig1_month.png', bbox_inches='tight'); plt.close(fig)
```

代码速查：今天这几段，要在本机跑通（5/9）

```
m.round(2).to_csv(OUT / 'fig1_data.csv', encoding='utf-8-sig')
```

图2 城市 × 品类 堆叠（横向）

```
pv = pd.pivot_table(paid, index='city', columns='category',
```

```
                    values='amount', aggfunc='sum', fill_value=0)
```

```
pv = pv.loc[pv.sum(axis=1).sort_values().index]
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（6/9）

```
fig, ax = plt.subplots(figsize=(8, 5), dpi=150)
```

```
pv.plot(kind='barh', stacked=True, ax=ax, width=.72)
```

```
ax.set_title('各城市实付构成'); ax.set_xlabel('元')
```

```
fig.tight_layout(); fig.savefig(OUT / 'fig2_city.png', bbox_inches='tight'); plt.close(fig)
```

```
pv.round(2).to_csv(OUT / 'fig2_data.csv', encoding='utf-8-sig')
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（7/9）

```
# 图3 漏斗
e = pd.read_csv(DATA / 'events_dirty.csv', encoding='utf-8-sig')
step = ['view', 'cart', 'order', 'pay']
cnt = e.groupby('event')['user_id'].nunique().reindex(step)
fig, ax = plt.subplots(figsize=(7, 4), dpi=150)
ax.bar(step, cnt.values, color=['#0F2E35', '#2E7D8A', '#5BA3AE', '#9CC9D0'])
```

代码速查：今天这几段，要在本机跑通（8/9）

```
for i, v in enumerate(cnt.values):
```

```
    ax.text(i, v, f'{v:,}\n{v/cnt.iloc[0]*100:.1f}%', ha='center', va='bottom', fontsize=9)
```

```
ax.set_title('用户转化漏斗'); ax.set_ylim(0, cnt.max() * 1.3)
```

```
fig.tight_layout(); fig.savefig(OUT / 'fig3_funnel.png', bbox_inches='tight'); plt.close(fig)
```

```
cnt.to_csv(OUT / 'fig3_data.csv', encoding='utf-8-sig')
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（9/9）

验收（做到才算过关）：

① 三张 PNG 都能打开、中文不乱码、负号（如有）不是方框；

```
② fig3 的四个数 = 500 / 150 / 74 / 28，漏斗率 100% / 30.0% / 14.8% / 5.6%；
```

③ 每张图能配一句带数字的结论，念出来不超过 25 个字。

最容易错的地方

★ 横向条形（barh）时 y 轴是从下往上排，所以要先 sort_values() 升序，画出来才是「最大的在最上面」。忘了排序 = 图看着乱但没错，最难发现。

这些句子都是「差一个字、结果差一截」的地方，考前只翻这一页也值。

合上视频，自测一遍（能打勾才算学会）

- [ ] 我能不看资料写出：本机数据只有 3 个月（2026-08 / 09 / 10），折线只有 3 个点 ——这种时候**不要**画折线，画柱状图更诚实。图表选型跟着数据量走。
- [ ] 我能不看资料写出：不写 bbox_inches='tight'，图四周会留一大圈白 —— 贴进文档后看着很小。
- [ ] 我能不看资料写出：面试官问「这张图的数据哪来的」，能当场打开 CSV 比能打开 PPT 有说服力。
- [ ] 我能不看资料说出：一句话：折线图回答「在变好还是变坏」，一定要标出最高点和最低点。
- [ ] 我能不看题源，把今天用到的每个函数「为什么要有它」说一遍
- [ ] 我能说出今天每个参数/返回值改了会怎样——换个写法结果差在哪

这些句子都是「差一个字、结果差一截」的地方；三个勾以上，今天就算过关。

今天收工前，先知道明天学什么

第 19 天 • ★Excel 进阶：Power Query 清洗 + 条件格式 + 图表；下午开投

不用提前学。明天点「① 开始今天的学习」，会换上明天那份 PPT。

收工三件事（5 分钟）

- 今天没跑通的脚本存进 90-我的作业\，命名 2026-11-19-第18天-*.py
- 打卡表「③ 每日打卡」记一笔：今天卡在哪、错在哪（一句话就够）
- 错题明天重做一遍，练习数据在 60-练习环境\Python-Env\data

全阶段 • 第 19 天 • 11/20

★Excel 进阶：Power Query 清洗 + 条件格式 + 图表；下午开投

用 Power Query 把脏 Excel 一键清洗（可刷新），并做出带条件格式的汇总表；下午投出 ≥8 家。

第 19 天 • 11/20 • 今日知识点回顾

★Excel 进阶：Power Query 清洗 + 条件格式 + 图表；下午开投

目标：用 Power Query 把脏 Excel 一键清洗（可刷新），并做出带条件格式的汇总表；下午投出 ≥8 家。

对着 B 站视频看：视频讲到的每一句，这张 PPT 里都有一条；复习时只翻这份，不用再回去翻分P。

今天结束时，你应该能做到

验收标准（做到这句就算过关）

- 用 Power Query 把脏 Excel 一键清洗（可刷新），并做出带条件格式的汇总表；下午投出 ≥8 家。

今天的做法（顺序别颠倒）

- 题库地址：file:///D:/DSH工作区/00-学习中心/60-练习环境/Python-Env/data
- 做完记一句：今天卡在哪、错在哪 → 写进 3-每日打卡。

今天看哪几集、读哪一章

这一天的知识：4 集 • 合计约 64 分钟（边看边在本机敲一遍）

P44	数据透视表-创建数据透视表	约 18 分钟
P45	数据透视表-数据透视表的结构	约 12 分钟
P49	数据透视表-计算字段	约 14 分钟
P26	条件格式	约 20 分钟

教材（已下载到 20-教材与讲义\python-data\，断网也能看）

`https://raw.githubusercontent.com/datawhalechina/joyful-pandas/master/source/Content/ch5.rst`

对照看：pandas 的 pivot_table 和 Excel 透视表是同一件事

视频地址就在下面的「今天做哪些题」网页里，点一下直达分P。

知识点 1/7 • Power Query 的核心是「步骤可重放」，不是「拖一次」

- 一句话：数据 → 获取数据 → 从表格/区域；每一步都记在右侧「应用的步骤」里，改一步全链路重算。
- 代码：
- 数据 → 获取数据 → 来自文件 → 从 Excel 工作簿→ 选中「明细」表 → 转换数据→ 主页 → 删除行 → 删除重复项（选 订单号）
- → 转换 → 格式 → 修整（去空格）
- → 添加列 → 自定义列：= [数量] * [单价]→ 关闭并上载 → 到 表格

★ 关键区别：普通 Excel 操作是「改一次算一次」，Power Query 是「记下步骤，下次刷新自动重跑」。数据每月更新时，前者要重做，后者按一下刷新。

- 面试问「你会不会 Power Query」，本质问的是「你懂不懂可复现」。

知识点 2/7 • 本机 Excel 里就有一份三表结构，正好练合并

- 一句话：sales_dirty.xlsx 有 明细（400 行）/ 商品表（15 行）/ 城市表（9 行），天然适合练 merge。
- 代码：
- 明细 sheet 列：订单号 日期 城市 品类 商品 数量 单价 金额 销售员商品表 sheet 列：商品 品类 成本价城市表 sheet 列：城市 大区 负责人

★ 本机实测：明细 400 行 9 列，金额合计 408,549.00。

- Power Query 里用「合并查询」把 大区 接到明细上，再用透视表按大区汇总。

知识点 3/7 • 合并查询 vs 追加查询：一个是横着接，一个是竖着摞

- 一句话：合并（Merge）= 加列，需要选匹配键；追加（Append）= 加行，要求列名一致。
- 代码：
- 合并：主页 → 合并查询 → 选两表 → 点选 城市 列 → 联接种类「左外部」追加：主页 → 追加查询 → 三表以上 → 选表集合

★ 左外部 = pandas 的 how='left'，只保留第一张表的所有行。

- 选错成「内部」会悄悄丢掉匹配不上的行 —— 和 pandas 完全一样的坑。

知识点 4/7 • 条件格式：让异常自己跳出来

- 一句话：开始 → 条件格式 → 突出显示单元格规则 / 数据条 / 色阶 / 图标集。
- 代码：
- 选中 金额 列 → 条件格式 → 数据条 → 渐变填充（一眼看出谁大）
- 选中 金额 列 → 条件格式 → 突出显示 → 大于 → 2000 → 红填充选中 退款占比 列 → 条件格式 → 色阶 → 红-白-绿（红=高退款）
- 选中 数量 列 → 条件格式 → 图标集 → 三色旗

★ 条件格式是视图**，不改数据 —— 但它会让「异常」在汇报时自动被看见。**

- 做数据看板交付时，这一条比多加一张图更实用。

知识点 5/7 • 透视表四个区域 + 值显示方式

- 一句话：行 / 列 / 值 / 筛选；「值显示方式」能把合计变成占比。
- 代码：
- 插入 → 数据透视表 → 新工作表行：城市　列：品类　值：金额（求和）
- 再拖一次 金额 到值 → 值字段设置 → 值显示方式 → 列汇总的百分比

★ 同一列拖两次到「值」，一次显示金额、一次显示占比 —— 这是最省事的「金额 + 占比」双列做法，不用写计算字段。

知识点 6/7 • 计算字段 vs 计算项：一个加列、一个加行

- 一句话：计算字段 = 新的一列（用现有字段算）；计算项 = 某一列里的新分类。
- 代码：
- 透视表分析 → 字段、项目和集 → 计算字段名称：客单价　公式：= 金额 / 数量字段、项目和集 → 计算项（在「品类」里加一个「数码+家居」）

★ 计算字段引用的是求和后的值**再相除，不是逐行相除再求和 ——两种算法结果不同（前者是加权均价，后者是简单均价）。报数时说清用哪种。**

知识点 7/7 • 刷新与「数据源换了怎么办」

- 一句话：数据 → 全部刷新；源文件路径变了要在「数据 → 查询和连接 → 属性」里改。
- 代码：
- 数据 → 全部刷新（Ctrl+Alt+F5）
- 查询和连接 → 右键查询 → 属性 → 定义 → 源（改路径）

- （今天产出的 xlsx 要满足：改一行原始明细 → 刷新 → 透视表数字跟着变。
- 做不到这一点，说明你做的还是「手工表」，不是「可复现报表」。）

代码速查：今天这几段，要在本机跑通（1/2）

两件事：Excel 作品 1 份 + 投递 ≥8 家。

A. Excel 作品（60 分钟）

基于 sales_dirty.xlsx 做一份「销售分析.xlsx」，必须含：

```
1）Power Query 查询 1：明细 去重 + 修整 + 自定义列（金额校验 = 数量×单价）
```

2) Power Query 查询 2：明细 合并 城市表，带出「大区」

3) 透视表 1：大区 × 品类 金额（值显示方式：列汇总百分比）

4) 透视表 2：销售员 Top10（降序）

5) 条件格式：金额列数据条 + 退款/异常高亮

6) 一个图表：大区金额柱形图（引用透视表）

验收：改原始明细任意一行的数量 → 全部刷新 → 透视表和图表都变。

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（2/2）

B. 投递（下午，≥8 家，含百度 3 条）

```
用 4-投递追踪 sheet 记录：公司 / 岗位 / JD 关键词 / 投递时间 / 渠道。
```

每投一家，在「投递追踪」里写一句「这个 JD 要什么、我哪条经历对得上」。

验收（做到才算过关）：

① 刷新链路能跑通（改一行→刷新→数字变），当场演示给自己看；

② 透视表里大区金额合计 = 408,549.00（和明细金额合计对得上）；

③ 投递记录 ≥8 行，每行都有「JD 要什么 / 我对得上什么」。

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

最容易错的地方

★ 关键区别：普通 Excel 操作是「改一次算一次」，Power Query 是「记下步骤，下次刷新自动重跑」。数据每月更新时，前者要重做，后者按一下刷新。

选错成「内部」会悄悄丢掉匹配不上的行 —— 和 pandas 完全一样的坑。

这些句子都是「差一个字、结果差一截」的地方，考前只翻这一页也值。

合上视频，自测一遍（能打勾才算学会）

- [ ] 我能不看资料写出：左外部 = pandas 的 how='left'，只保留第一张表的所有行。
- [ ] 我能不看资料写出：条件格式是**视图**，不改数据 —— 但它会让「异常」在汇报时自动被看见。
- [ ] 我能不看资料写出：同一列拖两次到「值」，一次显示金额、一次显示占比 —— 这是最省事的「金额 + 占比」双列做法，不用写计算字段。
- [ ] 我能不看资料写出：计算字段引用的是**求和后的值**再相除，不是逐行相除再求和 ——两种算法结果不同（前者是加权均价，后者是简单均价）。报数时说清用哪种。
- [ ] 我能不看题源，把今天用到的每个函数「为什么要有它」说一遍
- [ ] 我能说出今天每个参数/返回值改了会怎样——换个写法结果差在哪

这些句子都是「差一个字、结果差一截」的地方；三个勾以上，今天就算过关。

今天收工前，先知道明天学什么

第 20 天 • 把 W4–W6 的脚本收成「一个能讲的项目」

不用提前学。明天点「① 开始今天的学习」，会换上明天那份 PPT。

收工三件事（5 分钟）

- 今天没跑通的脚本存进 90-我的作业\，命名 2026-11-20-第19天-*.py
- 打卡表「③ 每日打卡」记一笔：今天卡在哪、错在哪（一句话就够）
- 错题明天重做一遍，练习数据在 60-练习环境\Python-Env\data

全阶段 · 第 20 天 · 11/21

把 W4–W6 的脚本收成「一个能讲的项目」

能对着它讲 3 分钟：数据哪来、怎么洗、算出什么、结论是什么。

第 20 天 • 11/21 • 今日知识点回顾

把 W4–W6 的脚本收成「一个能讲的项目」

目标：有一个 5 个文件的目录，能对着它讲 3 分钟：数据哪来、怎么洗、算出什么、结论是什么。

对着 B 站视频看：视频讲到的每一句，这张 PPT 里都有一条；复习时只翻这份，不用再回去翻分P。

今天结束时，你应该能做到

验收标准（做到这句就算过关）

- 有一个 5 个文件的目录，能对着它讲 3 分钟：数据哪来、怎么洗、算出什么、结论是什么。

今天的做法（顺序别颠倒）

- 题库地址：file:///D:/DSH工作区/00-学习中心/60-练习环境/Python-Env/data
- 做完记一句：今天卡在哪、错在哪 → 写进 3-每日打卡。

今天看哪几集、读哪一章

这一天的知识：2 集 • 合计约 30 分钟（边看边在本机敲一遍）

P97	模块和包	约 18 分钟
P93	异常的传递性	约 12 分钟

教材（已下载到 20-教材与讲义\python-data\，断网也能看）

`https://raw.githubusercontent.com/datawhalechina/joyful-pandas/master/source/Content/ch1.rst`

第一章的练习题：用列表推导式改写 for 循环

知识点 1/7 • 项目结构：5 个文件，一个 README

• 一句话：一个脚本（main）、一个模块（清洗）、一个模块（分析）、一个 README、一个输出目录。

• 代码：

• project/README.md ← 3 分钟能讲完的说明src/load.py ← 读数据src/clean.py ← 六条清洗规则 src/analyze.py ← 汇总与图表run.py ← 只做编排out/ ← 所有产物

★ 面试官翻你的 GitHub 时只看 README 和目录结构。一个 800 行的单文件即使能跑，也会被当成「作业」而不是「项目」。

知识点 2/7 • README 的四段：数据 / 方法 / 结果 / 局限

- 一句话：局限那一段最加分 —— 它证明你知道自己的分析边界在哪。
- 代码：

```
# 电商订单分析
## 数据
```

- 1200 行订单（2026-08 ~ 2026-10），含重复单号、缺失金额、三种日期格式。

知识点 2/7 • README 的四段：数据 / 方法 / 结果 / 局限（续）

```
## 方法
```

> • 6 条清洗规则（去重 37 / 金额解析 35 缺失 / 城市填空 56 / 日期统一 145 / 数量置空 25 / 状态规整 6→3），清洗后 1163 行。

```
## 结果
```

知识点 2/7 • README 的四段：数据 / 方法 / 结果 / 局限（续）

- paid 417 单，实付 422,436 元；数码占 45.5%，南京最高 60,532。

```
## 局限
```

- 样本仅 3 个月，无法判断季节性；39 单用户信息缺失，性别维度分析不完整。

★ 「局限」不是自曝其短，是防止被问倒。写了局限，面试官就不会往那儿问；

- 不写，他一定会问。

知识点 3/7 • 异常处理：让脚本在数据变了的时候「说人话」

- 一句话：读不到文件、缺列、类型不对，三种情况都要有明确报错。
- 代码：

知识点 3/7 • 异常处理：让脚本在数据变了的时候「说人话」（续）

```
import sys, pathlib, pandas as pd
def load(path: pathlib.Path) -> pd.DataFrame:
if not path.exists():
sys.exit(f'[错误] 找不到数据文件：{path}')
df = pd.read_csv(path, encoding='utf-8-sig')
need = {'order_id', 'amount', 'city', 'status', 'create_time'}
miss = need - set(df.columns)
if miss:
sys.exit(f'[错误] 缺列：{sorted(miss)}；实际列：{list(df.columns)}')
if len(df) == 0:
sys.exit('[错误] 文件是空的')
return df
```

> **★ sys.exit(字符串) 会打印到 stderr 并以退出码 1 结束 —— 这是「脚本给用户看」最简单的做法。比 traceback 友好得多。**

这一页只讲「异常处理：让脚本在数据变了的时候「说人话」」。讲完就翻页，下一页是另一个知识点。

知识点 4/7 • 参数化：三个入口参数就够

- 一句话：--data 数据目录、--out 输出目录、--top N；不给就用默认。
- 代码：

```
import argparse
p = argparse.ArgumentParser()
p.add_argument('--data', default=r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
p.add_argument('--out', default=None)
p.add_argument('--top', type=int, default=10)
a = p.parse_args()
print(a.data, a.out, a.top)
```

★ 用 argparse 而不是 sys.argv[1]，好处是 --help 自动生成、参数可乱序、

- 少传不报错。30 行以内的脚本也值得用。

知识点 5/7 • 日志：一行时间戳 + 一行结论

- 一句话：不用 logging 模块也能有日志 —— print 到文件，格式统一就够。
- 代码：

```
import datetime as dt
def log(msg, f=None):
line = f'[{dt.datetime.now():%H:%M:%S}] {msg}'
print(line)
if f: f.write(line + '\n')
with open('run.log', 'w', encoding='utf-8') as f:
log('开始清洗', f)
log(f'清洗后 {1163} 行', f)
```

知识点 5/7 • 日志：一行时间戳 + 一行结论（续）

★ 项目里的 print 要能回答「什么时候、做了什么、结果多少」。

- 「print('done')」在面试官眼里等于没有日志。

知识点 6/7 • 自查：这份项目能不能被别人跑起来

- 一句话：换一台电脑、只给数据目录，能不能一条命令出结果。
- 代码：
- python run.py --data D:\somewhere\data --out .\out

- （今天要验证的是「可移植」：把项目目录复制到另一个路径，数据目录指向60-练习环境\Python-Env\data，跑一条命令，out/ 里四个文件都在。）

这一页只讲「自查：这份项目能不能被别人跑起来」。讲完就翻页，下一页是另一个知识点。

知识点 7/7 • 三分钟讲法：背景 → 做了什么 → 结果 → 学到了什么

- 一句话：每段 45 秒，结果段必须有两个具体数字。
- 代码：
- 背景：想练「脏数据到结论」的全流程，选了自己造的电商订单数据。
- 做了什么：1200 行、6 类脏点，写了 6 条清洗规则 + 4 张汇总表 + 3 张图。
- 结果：paid 417 单 42.2 万，数码占 45.5%，南京最高 6 万；
- 漏斗浏览→加购 30%，加购→下单只有 14.8%，卡点在加购后。
- 学到：日期格式不统一时 format='mixed' 也会静默丢行，必须先规整字符串。

★ 「学到」那一段要讲一个具体的坑**，不要讲「学会了 pandas」。**

- 讲坑 = 你真的做过；讲功能名 = 你看了教程。

代码速查：今天这几段，要在本机跑通（1/2）

```
把 W4-W6 的脚本收成一个目录 py-portfolio/order-analysis/：
```

py-portfolio/order-analysis/

README.md

run.py

src/__init__.py

src/load.py

src/clean.py

src/analyze.py

out/　　← 跑完才有

验收（做到才算过关）：

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（2/2）

```
① 复制整个目录到 D:\temp_test\，改 --data 指回练习数据，一条命令出 4 个产物；
```

② README 四段齐全，「局限」段不少于 2 条；

③ 对着 README 讲 3 分钟，计时器计时，不超 3 分 30 秒。

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

最容易错的地方

> 样本仅 3 个月，无法判断季节性；39 单用户信息缺失，性别维度分析不完整。

> 少传不报错。30 行以内的脚本也值得用。

这些句子都是「差一个字、结果差一截」的地方，考前只翻这一页也值。

合上视频，自测一遍（能打勾才算学会）

- [ ] 我能不看资料写出：面试官翻你的 GitHub 时只看 README 和目录结构。一个 800 行的单文件即使能跑，也会被当成「作业」而不是「项目」。
- [ ] 我能不看资料写出：sys.exit(字符串) 会打印到 stderr 并以退出码 1 结束 —— 这是「脚本给用户看」最简单的做法。比 traceback 友好得多。
- [ ] 我能不看资料写出：项目里的 print 要能回答「什么时候、做了什么、结果多少」。
- [ ] 我能不看资料写出：「学到」那一段要讲一个**具体的坑**，不要讲「学会了 pandas」。
- [ ] 我能不看题源，把今天用到的每个函数「为什么要有它」说一遍
- [ ] 我能说出今天每个参数/返回值改了会怎样——换个写法结果差在哪

这些句子都是「差一个字、结果差一截」的地方；三个勾以上，今天就算过关。

今天收工前，先知道明天学什么

第 21 天 • ★简历第一版 + W6 总验收

不用提前学。明天点「① 开始今天的学习」，会换上明天那份 PPT。

收工三件事（5 分钟）

- 今天没跑通的脚本存进 90-我的作业\，命名 2026-11-21-第20天-*.py
- 打卡表「③ 每日打卡」记一笔：今天卡在哪、错在哪（一句话就够）
- 错题明天重做一遍，练习数据在 60-练习环境\Python-Env\data

全阶段 · 第 21 天 · 11/22

★简历第一版 + W6 总验收

一页 PDF 简历，项目段写的是「做了什么 + 数字结果」，投递 8 家的复盘写出来。

第 21 天 • 11/22 • 今日知识点回顾

★简历第一版 + W6 总验收

目标：一页 PDF 简历，项目段写的是「做了什么 + 数字结果」，投递 8 家的复盘写出来。

对着 B 站视频看：视频讲到的每一句，这张 PPT 里都有一条；复习时只翻这份，不用再回去翻分P。

今天结束时，你应该能做到

验收标准（做到这句就算过关）

- 一页 PDF 简历，项目段写的是「做了什么 + 数字结果」，投递 8 家的复盘写出来。

今天的做法（顺序别颠倒）

- 题库地址：file:///D:/DSH工作区/00-学习中心/60-练习环境/Python-Env/data
- 做完记一句：今天卡在哪、错在哪 → 写进 3-每日打卡。

今天看哪几集、读哪一章

这一天的知识：2 集 • 合计约 27 分钟（边看边在本机敲一遍）

P1	业务数据分析师岗位介绍	约 12 分钟
P2	数据分析师的日常工作	约 15 分钟

教材（已下载到 20-教材与讲义\python-data\，断网也能看）

`https://raw.githubusercontent.com/datawhalechina/joyful-pandas/master/source/Content/ch10.rst`

最后一章复习，确认「时序 / 透视 / 分组」三件事都能默写

视频地址就在下面的「今天做哪些题」网页里，点一下直达分P。

知识点 1/7 • 项目段的三行结构：背景 / 动作 / 结果

- 一句话：每行都以动词开头，结果行必须有数字。
- 代码：
- 【电商订单数据分析】 个人项目 · 2026.10–11

  · 针对 1200 行含重复单号、缺失金额、三种日期格式的脏数据，设计 6 条清洗规则，将可用样本从 1200 行收敛到 1163 行并量化每步影响。

  · 用 pandas 完成 groupby / pivot_table / resample 三层汇总，产出 4 张业务表与 3 张图，定位到「加购→下单」转化率仅 14.8% 为漏斗主卡点。

  · 技术栈：Python（pandas / numpy / matplotlib）、Excel（Power Query / 透视表）。

★ 「1200 → 1163」这种收敛数字，比「熟练使用 pandas」有说服力十倍。

- 写不出数字，说明你没真的做过。

知识点 2/7 • 技能栏只写「能验证」的

- 一句话：写了「精通 SQL」，面试官就会问你窗口函数和索引优化。
- 代码：
- 技能（按熟练度排序，只写能当场写的）：
  - Python：pandas 清洗/分组/透视/时序，numpy 向量化，matplotlib 出图
  - SQL：多表 JOIN、窗口函数、CTE、聚合与子查询（MySQL 8.4 实机练习）
  - Excel：Power Query、数据透视表、条件格式、VLOOKUP/INDEX-MATCH
  - 统计：描述统计、假设检验、A/B 实验设计与样本量估算（进行中）

★ 「进行中」三个字很有用 —— 它让面试官知道你在学，而不是装会。

- 但只允许出现在一两项上，多了就成了「什么都不会」。

知识点 3/7 • 简历的排版硬约束

- 一句话：一页、PDF、不要照片、不要花哨模板、字号 10.5–11pt。
- 代码：
- 页边距 1.5 cm；正文 10.5pt；行距 1.15；中文字体 思源黑体/微软雅黑导出：Word → 另存为 PDF（不要截图转 PDF）

★ 截图转 PDF 的文件搜不到文字，HR 系统（ATS）直接判定「无内容」。

- 必须用 Word/WPS 导出真 PDF。

知识点 4/7 • 投递复盘表：8 家的共同点

- 一句话：把 8 个 JD 的关键词列出来，重复出现的词就是你的简历要补的词。
- 代码：
- JD 关键词统计（示例）：
- SQL 8/8Python 6/8 数据可视化 7/8A/B 实验 5/8 业务指标体系 6/8 用户增长 4/8

★ 8 家里出现 5 次以上的词，必须在简历的技能栏和项目段里各出现一次。

- 这就是「按 JD 改简历」的全部方法论，不用玄学。

知识点 5/7 • 投递渠道的记录方式

- 一句话：公司 / 岗位 / 渠道 / 时间 / 状态 / 一句「为什么投」。
- 代码：
- | 公司 | 岗位 | 渠道 | 投递日 | 状态 | JD 要什么 / 我对得上什么 || 百度 | 数据分析师 | 官网 | 11-20 | 已投 | SQL+指标；我做过漏斗和指标体系 || … |

★ 「为什么投」这一列是给自己看的：两周后 HR 打电话，你能立刻想起这家是干什么的、你当时怎么想的。

知识点 6/7 • W6 验收清单：8 条

- 一句话：pandas 三件套 + Excel + 简历，三块都要能当场演示。
- 代码：

```
CHECK_W6 = [
'能不看资料写出 pivot_table 四件套（index/columns/values/aggfunc）',
'能解释 resample 的 ME/MS 与 pct_change 首期为什么是 nan',
'能说清 apply / map / np.select 三者什么时候用哪个',
'能用 Excel Power Query 做出可刷新的一键清洗',
'能做出带条件格式的透视表汇总',
'有一份能一条命令跑通的项目目录',
'有一页 PDF 简历，项目段含 3 个以上数字',
'投递记录 ≥8 家，每家都有「为什么投」',
```

知识点 6/7 • W6 验收清单：8 条（续）

-]

- （8 条里只要有 1 条做不到，就还没到 W6 结束的标准。别往下跳 ——W7 的统计和 A/B 都建立在这些基础上。）

知识点 7/7 • 面试自述：60 秒版本

- 一句话：现在做什么 / 会什么 / 想做什么，三句话。
- 代码：
- 「我目前在做数据分析方向的系统学习，已经完整跑过一轮从脏数据到业务结论的项目：
- 1200 行订单清洗到 1163 行，做出品类、城市、漏斗三层汇总。
- 技术上主要用 Python 的 pandas 和 SQL，Excel 的 Power Query 和透视表也熟。
- 接下来想找数据分析/商业分析的岗位，把清洗和分析这套流程用在真实业务上。」

★ 60 秒版本不要背稿感 —— 把「1200 → 1163」「14.8%」这种数字记住就行，其余用口语。

代码速查：今天这几段，要在本机跑通

三件事：

A. 简历第一版（PDF，一页）

B. 投递复盘表（8 家，含 JD 关键词统计）

C. W6 验收 8 条逐条打勾

验收（做到才算过关）：

① PDF 里能搜到文字（用 Edge 打开 Ctrl+F 搜「pandas」能命中）；

② 项目段有 3 个以上具体数字（1200/1163/422,436/14.8%/60,532 任选）；

③ 60 秒自述能不看稿说完，含 2 个数字。

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

最容易错的地方

_x0000_RUNPANEL_x0000_（8 条里只要有 1 条做不到，就还没到 W6 结束的标准。别往下跳 ——W7 的统计和 A/B 都建立在这些基础上。）

这些句子都是「差一个字、结果差一截」的地方，考前只翻这一页也值。

合上视频，自测一遍（能打勾才算学会）

- [ ] 我能不看资料写出：「1200 → 1163」这种收敛数字，比「熟练使用 pandas」有说服力十倍。
- [ ] 我能不看资料写出：「进行中」三个字很有用 —— 它让面试官知道你在学，而不是装会。
- [ ] 我能不看资料写出：截图转 PDF 的文件搜不到文字，HR 系统（ATS）直接判定「无内容」。
- [ ] 我能不看资料写出：8 家里出现 5 次以上的词，必须在简历的技能栏和项目段里各出现一次。
- [ ] 我能不看资料写出：60 秒版本不要背稿感 —— 把「1200 → 1163」「14.8%」这种数字记住就行，其余用口语。
- [ ] 我能不看题源，把今天用到的每个函数「为什么要有它」说一遍

这些句子都是「差一个字、结果差一截」的地方；三个勾以上，今天就算过关。

今天收工前，先知道明天学什么

第 22 天 • 统计专项：描述统计、抽样与置信区间、假设检验、 P 值三种误用

不用提前学。明天点「① 开始今天的学习」，会换上明天那份 PPT。

收工三件事（5 分钟）

- 今天没跑通的脚本存进 90-我的作业\，命名 2026-11-22-第21天-*.py
- 打卡表「③ 每日打卡」记一笔：今天卡在哪、错在哪（一句话就够）
- 错题明天重做一遍，练习数据在 60-练习环境\Python-Env\data

全阶段 • 第 22 天 • 11/23

统计专项：描述统计、抽样与置信区间、假设检验、P 值三种误用

能判断该用 t 检验还是卡方，算完能正确解读 P 值，并说出三种常见的误用。

第 22 天 • 11/23 • 今日知识点回顾

统计专项：描述统计、抽样与置信区间、假设检验、P 值三种误用

目标：拿到两组数据，能判断该用 t 检验还是卡方，算完能正确解读 P 值，并说出三种常见的误用。

对着 B 站视频看：视频讲到的每一句，这张 PPT 里都有一条；复习时只翻这份，不用再回去翻分P。

今天结束时，你应该能做到

验收标准（做到这句就算过关）

- 拿到两组数据，能判断该用 t 检验还是卡方，算完能正确解读 P 值，并说出三种常见的误用。

今天的做法（顺序别颠倒）

- 题库地址：file:///D:/DSH工作区/00-学习中心/60-练习环境/Python-Env/data
- 做完记一句：今天卡在哪、错在哪 → 写进 3-每日打卡。

今天看哪几集、读哪一章

这一天的知识：5 集 • 合计约 68 分钟（边看边在本机敲一遍）

P1	总体均值与样本均值	约 12 分钟
P5	样本标准差与总体标准差	约 14 分钟
P25	中心极限定理	约 16 分钟
P30	置信区间	约 12 分钟
P36	假设检验与 P 值	约 14 分钟

教材（已下载到 20-教材与讲义\python-data\，断网也能看） https://raw.githubusercontent.com/datawhalechina/joyful-pandas/master/source/Content/ch2.rst

视频地址就在下面的「今天做哪些题」网页里，点一下直达分P。

知识点 1/7 • 描述统计的五个数，比「平均值」有用得多

- 一句话：均值、中位数、标准差、P25/P75、极值 —— 一起看才知道分布长什么样。
- 代码：

这一页只讲「描述统计的五个数，比「平均值」有用得多」。讲完就翻页，下一页是另一个知识点。

知识点 1/7 • 描述统计的五个数，比「平均值」有用得多（续）

```
import pandas as pd, numpy as np, pathlib
DATA = pathlib.Path(r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
df = pd.read_csv(DATA / 'clean_v2.csv', encoding='utf-8-sig')
a = df['amount'].dropna()
print('n', len(a))
print('均值', round(a.mean(), 2), '中位数', round(a.median(), 2))
print('标准差', round(a.std(), 2), '偏度', round(a.skew(), 3))
print(a.describe(percentiles=[.25, .5, .75, .9, .99]).round(2).to_string())
```

★ 本机实测：金额均值 ≈ 1,013，中位数 ≈ 674 —— 均值明显高于中位数，说明右偏（少数大单拉高均值）。只报均值会让业务方误判「客单价一千」，实际上「一半的订单在 674 以下」。

这一页只讲「描述统计的五个数，比「平均值」有用得多」。讲完就翻页，下一页是另一个知识点。

知识点 2/7 • 标准差 ddof：pandas 和 numpy 默认不一样

- 一句话：pandas 的 std() 默认 ddof=1（样本），numpy 的 std() 默认 ddof=0（总体）。
- 代码：

```
arr = a.to_numpy()
print('pandas std', round(a.std(), 4), 'ddof=0', round(a.std(ddof=0), 4))
print('numpy  std', round(arr.std(), 4), 'ddof=1', round(arr.std(ddof=1), 4))
```

★ 样本量大时差别很小（差 √(n/(n-1)) 倍），但两组数对不上时，第一个要查的就是 ddof。这个坑在 A/B 实验算标准误时会被放大。

知识点 3/7 • 标准误与置信区间：手算一遍

- 一句话：SE = s/√n；95% CI = 均值 ± 1.96×SE（大样本）。
- 代码：

```
n = len(a); m = a.mean(); s = a.std(ddof=1)
se = s / np.sqrt(n)
```

知识点 3/7 • 标准误与置信区间：手算一遍（续）

- lo, hi = m - 1.96 * se, m + 1.96 * se

```
print(f'n={n} 均值={m:.2f} SE={se:.2f} 95%CI=[{lo:.2f}, {hi:.2f}]')
```

★ 置信区间的正确解读是「反复抽样 100 次，约 95 次算出的区间会包含真值」，不是**「真值有 95% 概率落在这个区间里」。面试常考这一句。**

知识点 4/7 • t 检验：两组均值是否有差异

- 一句话：scipy.stats.ttest_ind(样本1, 样本2, equal_var=False) —— 默认 Welch，更稳。
- 代码：

```
from scipy import stats
paid = df[df['status'] == 'paid']
beijing = paid[paid['city'] == '北京']['amount'].dropna()
shanghai = paid[paid['city'] == '上海']['amount'].dropna()
t, p = stats.ttest_ind(beijing, shanghai, equal_var=False)
print(f'北京 n={len(beijing)} 均值={beijing.mean():.0f}')
print(f'上海 n={len(shanghai)} 均值={shanghai.mean():.0f}')
print(f't={t:.3f} p={p:.4f}')
```

知识点 4/7 • t 检验：两组均值是否有差异（续）

★ equal_var=True（Student）要求两组方差相等，真实数据很难满足；

• Welch（equal_var=False）是默认推荐。scipy 的 ttest_ind 默认是 equal_var=True，必须显式写 False —— 这是最容易漏的一个参数。

知识点 5/7 • 卡方检验：两个分类变量是否相关

- 一句话：pd.crosstab 出列联表，再 scipy.stats.chi2_contingency。
- 代码：

```
ct = pd.crosstab(paid['city'], paid['category'])
chi2, p, dof, exp = stats.chi2_contingency(ct)
print(f'chi2={chi2:.3f} p={p:.4f} dof={dof}')
print('期望频数最小值', round(exp.min(), 2))
```

★ 卡方的前提是「期望频数 ≥5」，本机 9×4 表里最小期望值如果小于 5，结论就不可靠，要合并类别或改用 Fisher 精确检验。

这一页只讲「卡方检验：两个分类变量是否相关」。讲完就翻页，下一页是另一个知识点。

知识点 6/7 • P 值的三种误用（面试必问）

- 一句话：P 小 ≠ 效应大；P 大 ≠ 没差异；做了 20 次检验总有 1 次 P<0.05。
- 代码：

知识点 6/7 • P 值的三种误用（面试必问）（续）

```
# 误用 1：把 P 当效应量
#   P=0.001 只说明「不太可能是巧合」，不说明差异大。
#   要看效应量：Cohen's d = (m1-m2)/pooled_std
d = (beijing.mean() - shanghai.mean()) / np.sqrt((beijing.var() + shanghai.var()) / 2)
print('Cohen d =', round(d, 3))       # |d|<0.2 可忽略，0.5 中等，0.8 大
# 误用 2：P>0.05 就说「两组一样」
#   正确说法是「没有足够证据认为不同」，不是「证明相同」。
# 误用 3：多重比较不校正
#   做 20 次独立检验，至少一次 P<0.05 的概率 = 1-0.95^20 ≈ 64%
print('20 次检验的假阳性概率', round((1 - 0.95 ** 20) * 100, 1), '%')
```

★ 这三条是数据分析面试里「统计」部分的高频题。答得出「P 值不是效应量」这一句，基本就过关了。

知识点 7/7 • 样本量：想检出多大差异，需要多少人

- 一句话：$n \approx 16 \times \sigma^2 / \Delta^2$（两组均值比较，α=0.05、power=0.8 的经验公式）。
- 代码：

```
sigma = a.std(ddof=1)
for delta in [50, 100, 200]:
n_need = int(np.ceil(16 * sigma ** 2 / delta ** 2))
print(f'想检出 {delta} 元的差异，每组需要 {n_need} 人')
```

- （本机 σ≈1,100：检出 100 元差异每组要约 1,900 人；检出 200 元约 480 人。
- 这就是「为什么小流量实验测不出小改进」的量化答案 ——明天做 A/B 设计时直接用这个公式。）

代码速查：今天这几段，要在本机跑通（1/6）

写 stats_practice.py，五道题都要打印数字：

```
import numpy as np, pandas as pd, pathlib
```

```
from scipy import stats
```

```
DATA = pathlib.Path(r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
```

```
df = pd.read_csv(DATA / 'clean_v2.csv', encoding='utf-8-sig')
```

paid = df[df['status'] == 'paid']

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（2/6）

1) 描述统计：均值/中位数/偏度/分位数

a = df['amount'].dropna()

```
print(a.describe(percentiles=[.25, .5, .75, .9, .99]).round(2).to_string())
```

```
print('偏度', round(a.skew(), 3))
```

2) ddof 对比

```
print('pandas std', round(a.std(), 4), '| numpy std', round(a.to_numpy().std(), 4))
```

3) 95% 置信区间

代码速查：今天这几段，要在本机跑通（3/6）

```
n, m, s = len(a), a.mean(), a.std(ddof=1)
```

se = s / np.sqrt(n)

```
print(f'CI = [{m - 1.96*se:.2f}, {m + 1.96*se:.2f}]')
```

4) 两个城市金额差异：t 检验 + 效应量

g = paid.groupby('city')['amount']

```
top2 = paid['city'].value_counts().head(2).index.tolist()
```

代码速查：今天这几段，要在本机跑通（4/6）

```
x, y = g.get_group(top2[0]).dropna(), g.get_group(top2[1]).dropna()
```

```
t, p = stats.ttest_ind(x, y, equal_var=False)
```

```
d = (x.mean() - y.mean()) / np.sqrt((x.var() + y.var()) / 2)
```

```
print(f'{top2[0]} vs {top2[1]}: t={t:.3f} p={p:.4f} d={d:.3f}')
```

5) 卡方：城市与品类是否相关

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（5/6）

```
ct = pd.crosstab(paid['city'], paid['category'])
```

```
chi2, p2, dof, exp = stats.chi2_contingency(ct)
```

```
print(f'chi2={chi2:.3f} p={p2:.4f} dof={dof} 最小期望={exp.min():.2f}')
```

验收（做到才算过关）：

① 第 1 题能说出「均值 > 中位数」代表右偏，并解释业务含义；

② 第 4 题 p 值和 Cohen d 都报出来，能说清「p 小但 d 小」意味着什么；

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（6/6）

③ 第 5 题能判断卡方是否可用（看最小期望是否 ≥5）。

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

最容易错的地方

★ 样本量大时差别很小（差 √(n/(n-1)) 倍），但两组数对不上时，第一个要查的就是 ddof。这个坑在 A/B 实验算标准误时会被放大。

★ 卡方的前提是「期望频数 ≥5」，本机 9×4 表里最小期望值如果小于 5，结论就不可靠，要合并类别或改用 Fisher 精确检验。

这些句子都是「差一个字、结果差一截」的地方，考前只翻这一页也值。

合上视频，自测一遍（能打勾才算学会）

- [ ] 我能不看资料写出：样本量大时差别很小（差 √(n/(n-1)) 倍），但两组数对不上时，第一个要查的就是 ddof。这个坑在 A/B 实验算标准误时会被放大。

- [ ] 我能不看资料写出：置信区间的正确解读是「反复抽样 100 次，约 95 次算出的区间会包含真值」，**不是**「真值有 95% 概率落在这个区间里」。面试常考这一句。

- [ ] 我能不看资料写出：卡方的前提是「期望频数 ≥5」，本机 9×4 表里最小期望值如果小于 5，结论就不可靠，要合并类别或改用 Fisher 精确检验。

- [ ] 我能不看资料写出：这三条是数据分析面试里「统计」部分的高频题。答得出「P 值不是效应量」这一句，基本就过关了。

- [ ] 我能不看题源，把今天用到的每个函数「为什么要有它」说一遍

- [ ] 我能说出今天每个参数/返回值改了会怎样，换个写法结果差在哪

这些句子都是我自己写、结果要十秒内能想起来，当自己没上过今天就算过关。

今天收工前，先知道明天学什么

第 23 天 • 上午 A/B 实验设计 + 下午指标体系（一天两段，各出 300 字）

不用提前学。明天点「① 开始今天的学习」，会换上明天那份 PPT。

收工三件事（5 分钟）

- 今天没跑通的脚本存进 90-我的作业\，命名 2026-11-23-第22天-*.py
- 打卡表「③ 每日打卡」记一笔：今天卡在哪、错在哪（一句话就够）
- 错题明天重做一遍，练习数据在 60-练习环境\Python-Env\data

全阶段 • 第 23 天 • 11/24

上午 AB 实验设计 + 下午指标体系（一天两段，各出 300 字）

能落地的 A/B 方案（含样本量与判定规则），并对任意 App 说出核心指标体系 + 画主漏斗 + 指 3 个优化点。

第 23 天 • 11/24 • 今日知识点回顾

上午 A/B 实验设计 + 下午指标体系（一天两段，各出 300 字）

目标：写出一份能落地的 A/B 方案（含样本量与判定规则），并对任意 App 说出核心指标体系 + 画主漏斗 + 指 3 个优化点。

对着 B 站视频看：视频讲到的每一句，这张 PPT 里都有一条；复习时只翻这份，不用再回去翻分P。

今天结束时，你应该能做到

验收标准（做到这句就算过关）

- 写出一份能落地的 A/B 方案（含样本量与判定规则），并对任意 App 说出核心指标体系 + 画主漏斗 + 指 3 个优化点。

今天的做法（顺序别颠倒）

- 题库地址：file:///D:/DSH工作区/00-学习中心/60-练习环境/Python-Env/data
- 做完记一句：今天卡在哪、错在哪 → 写进 3-每日打卡。

今天看哪几集、读哪一章

这一天的知识：4 集 • 合计约 50 分钟（边看边在本机敲一遍）

P38	假设检验的基本思想	约 14 分钟
P40	单侧与双侧检验	约 12 分钟
P42	第一类错误与第二类错误	约 12 分钟
P43	两样本均值差的检验	约 12 分钟

教材（已下载到 20-教材与讲义\python-data\，断网也能看）

`https://raw.githubusercontent.com/datawhalechina/joyful-pandas/master/source/Content/ch4.rst`

分组聚合：实验组/对照组的分桶统计就是 groupby

视频地址就在下面的「今天做哪些题」网页里，点一下直达分P。

知识点 1/7 • A/B 的第一步不是算样本量，是「定一个可证伪的假设」

- 一句话：格式固定为「做 X，预期 Y 指标从 A 变到 B，因为 Z 机制」。
- 代码：
- 假设模板：
- 现状：加购到下单转化 14.8% 动作：在购物车页加「限时 30 分钟」倒计时条预期：加购→下单转化从 14.8% 提升到 ≥16.5% 机制：制造紧迫感，减少「先加购再比价」的流失证伪：如果 14 天实验后提升 <0.5pp 且 p>0.05，假设不成立

★ 「提升转化率」不是假设，是愿望。假设必须有具体数字 + 机制 + 证伪条件。

知识点 2/7 • 样本量：用昨天的公式，别凭感觉

- 一句话：$n = 16\sigma^2/\Delta^2$（连续指标）；比例指标用 $n = 16p(1-p)/\Delta^2$。
- 代码：

```
import numpy as np
```

- p0, delta = 0.148, 0.017　　# 基线 14.8%，想提升 1.7pp

知识点 2/7 • 样本量：用昨天的公式，别凭感觉（续）

```
n = 16 * p0 * (1 - p0) / delta ** 2
print(f'每组需要 {int(np.ceil(n))} 人，两组共 {int(np.ceil(n))*2}')
```

★ 本机算：p=0.148、Δ=0.017 → 每组约 1,166 人，两组约 2,332 人。

- 按每天 500 浏览算，需要 5 天。这个「5 天」就是实验周期的最小值。

知识点 3/7 • 分流与实验期：两个容易被忽略的坑

- 一句话：分流要随机且稳定（同一用户始终同组），实验期要覆盖完整周期（含周末）。
- 代码：

```
# 分桶：用 user_id 的哈希取模，保证同一用户稳定落组
import hashlib
def bucket(uid, salt='exp_cart_timer'):
h = hashlib.md5(f'{uid}{salt}'.encode()).hexdigest()
return int(h[:8], 16) % 100        # 0-99
print([bucket(u) < 50 for u in [1001, 1002, 1003]])   # True=实验组
```

★ 用「随机数」分桶会让同一用户刷新页面就换组，实验直接废掉。

- 用哈希（uid + 实验名）才是行业标准做法。加了实验名做 salt，不同实验之间不会互相干扰。

知识点 4/7 • 判定规则：先写死，不许事后改

- 一句话：主指标 p<0.05 且提升方向正确 → 上线；p<0.05 但方向反 → 下线；p≥0.05 → 不做结论。
- 代码：

```
DECIDE = {
'p<0.05 且 Δ>0': '上线（并观察 7 天留存）',
'p<0.05 且 Δ<0': '下线，记录负向原因',
'p>=0.05': '无足够证据，不上线（不是「没差异」）',
```

这一页只讲「判定规则：先写死，不许事后改」。讲完就翻页，下一页是另一个知识点。

知识点 4/7 • 判定规则：先写死，不许事后改（续）

- }

```
for k, v in DECIDE.items():
print(f'{k:20s}{v}')
```

★ 「p≥0.05 就继续加量再测」是典型的数据偷看（peeking），会把假阳性率从 5% 推到 20% 以上。实验期内不许中途判定。

知识点 5/7 • 指标体系：一个 App 的四层结构

- 一句话：北极星 → 一级指标 → 二级指标 → 过程指标，每层不超过 5 个。
- 代码：
- 电商 App 示例：
- 北极星：GMV 一级：DAU / 下单用户数 / 客单价 / 复购率二级：浏览-加购率 / 加购-下单率 / 支付成功率 / 退款率过程：页面停留时长 / 搜索次数 / 收藏数 / 优惠券使用率

★ 「DAU/MAU/留存/ARPU/GMV/转化率」这六个词必须能各自解释一句，并且知道它们之间的乘法关系（GMV = 下单用户 × 客单价 = DAU × 转化率 × 客单价）。

知识点 6/7 • 漏斗：画出来才能指优化点

- 一句话：漏斗的每一层都要有「人数 + 转化率 + 相对上一层的流失」。
- 代码：

```
import pandas as pd, pathlib
DATA = pathlib.Path(r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
e = pd.read_csv(DATA / 'events_dirty.csv', encoding='utf-8-sig')
step = ['view', 'cart', 'order', 'pay']
cnt = e.groupby('event')['user_id'].nunique().reindex(step)
rate = (cnt / cnt.iloc[0] * 100).round(1)
step_rate = (cnt / cnt.shift(1) * 100).round(1)
print(pd.DataFrame({'人数': cnt, '整体转化%': rate, '层间转化%': step_rate}).to_string())
```

知识点 6/7 • 漏斗：画出来才能指优化点（续）

★ 本机实测：500 / 150 / 74 / 28；整体 100% / 30.0% / 14.8% / 5.6%；

• 层间 100% / 30.0% / 49.3% / 37.8%。**层间转化率**才是找卡点的工具 ——整体转化率看不出「哪一步掉得最狠」。

知识点 7/7 • 三个优化点：从漏斗和数据里找，不靠猜

- 一句话：每个优化点都要挂一个数字 + 一个可执行动作。
- 代码：
- 优化点 1：加购→下单 49.3%，比浏览→加购的 30.0% 好但仍是最大流失环节，假设是运费在结算页才显示 → 动作：加购页就展示预估运费。
- 优化点 2：下单→支付 37.8%，支付环节流失超过六成，假设是支付方式少 → 动作：接入更多支付渠道 + 支付失败重试。
- 优化点 3：整体 5.6% 的付费转化，若 DAU 1 万则日付费 560 人，提升 1pp = 每天多 100 单 → 优先做影响面最大的环节。

- （三个优化点必须能排优先级。「影响面 × 可执行性」是最好用的排序口径，不要用「我觉得这个重要」。）

代码速查：今天这几段，要在本机跑通（1/2）

```
A 段（上午，90 分钟）：写 ab_design.md，300 字以上，必须含：
```

① 假设（含现状数字、动作、预期数字、机制、证伪条件）

② 样本量计算（写出公式和代入的数）

③ 分流方案（哈希分桶代码）

④ 主指标 / 护栏指标（护栏指标：退款率、客诉数，防止「提升转化但伤体验」）

⑤ 判定规则表（三种结果分别怎么做）

⑥ 实验周期（由样本量 ÷ 日均流量推出来）

B 段（下午，90 分钟）：写 metrics.md，300 字以上，必须含：

① 一个 App 的四层指标体系（北极星/一级/二级/过程）

② 主漏斗（四步，带人数和层间转化率）

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（2/2）

③ 三个优化点（每个带数字 + 动作 + 优先级理由）

验收（做到才算过关）：

① A 段样本量能算出来并说出「按当前流量需要几天」；

```
② B 段漏斗用 events_dirty.csv 的真实数字（500/150/74/28）；
```

③ 三个优化点能排序，并说出排序依据（影响面 × 可执行性）。

最容易错的地方

★ 「p≥0.05 就继续加量再测」是典型的数据偷看（peeking），会把假阳性率从 5% 推到 20% 以上。实验期内不许中途判定。

这些句子都是「差一个字、结果差一截」的地方，考前只翻这一页也值。

合上视频，自测一遍（能打勾才算学会）

- [ ] 我能不看资料写出：「提升转化率」不是假设，是愿望。假设必须有具体数字 + 机制 + 证伪条件。
- [ ] 我能不看资料写出：用「随机数」分桶会让同一用户刷新页面就换组，实验直接废掉。
- [ ] 我能不看资料写出：「p≥0.05 就继续加量再测」是典型的数据偷看（peeking），会把假阳性率从 5% 推到 20% 以上。实验期内不许中途判定。
- [ ] 我能不看题源，把今天用到的每个函数「为什么要有它」说一遍
- [ ] 我能说出今天每个参数/返回值改了会怎样——换个写法结果差在哪
- [ ] 今天这几段代码我在本机都跑通、并看清了输出

这些句子都是「差一个字、结果差一截」的地方；三个勾以上，今天就算过关。

今天收工前，先知道明天学什么

第 24 天 • Tableau 看板 1/3：连接数据、字段类型、第一张工作表

不用提前学。明天点「① 开始今天的学习」，会换上明天那份 PPT。

收工三件事（5 分钟）

- 今天没跑通的脚本存进 90-我的作业\，命名 2026-11-24-第23天-*.py
- 打卡表「③ 每日打卡」记一笔：今天卡在哪、错在哪（一句话就够）
- 错题明天重做一遍，练习数据在 60-练习环境\Python-Env\data

全阶段 · 第 24 天 · 11/25

Tableau 看板 13：连接数据、字段类型、第一张工作表

Tableau 装上、连上 douyin_notes.csv、做出第一张「各标签播放量」条形图，并理解「维度 vs 度量」。

第 24 天 • 11/25 • 今日知识点回顾

Tableau 看板 1/3：连接数据、字段类型、第一张工作表

目标：Tableau 装上、连上 douyin_notes.csv、做出第一张「各标签播放量」条形图，并理解「维度 vs 度量」。

对着 B 站视频看：视频讲到的每一句，这张 PPT 里都有一条；复习时只翻这份，不用再回去翻分P。

今天结束时，你应该能做到

验收标准（做到这句就算过关）

- Tableau 装上、连上 douyin_notes.csv、做出第一张「各标签播放量」条形图，并理解「维度 vs 度量」。

今天的做法（顺序别颠倒）

- 题库地址：file:///D:/DSH工作区/00-学习中心/60-练习环境/Python-Env/data
- 做完记一句：今天卡在哪、错在哪 → 写进 3-每日打卡。

今天看哪几集、读哪一章

这一天的知识：4 集 • 合计约 55 分钟（边看边在本机敲一遍）

P1	Tableau 产品介绍	约 10 分钟
P4	Tableau 数据字段与数据类型	约 14 分钟
P13	Tableau 行列与视图	约 16 分钟
P17	Tableau 条形图	约 15 分钟

教材（已下载到 20-教材与讲义\python-data\，断网也能看）

`https://raw.githubusercontent.com/datawhalechina/joyful-pandas/master/source/Content/ch5.rst`

对照看：Tableau 的「行列 + 标记卡」≈ pandas 的 pivot_table

视频地址就在下面的「今天做哪些题」网页里，点一下直达分P。

知识点 1/7 • Tableau 本机没装，先装 Public 版（免费）

- 一句话：tableau.com/products/public 下载 Tableau Public（免费，能存到云端公开库）。
- 代码：
- 下载：https://www.tableau.com/products/public/download 安装到 D 盘：D:\Tableau\（不装 C 盘）
- 注意：Public 版的工作簿会存到公开云，**不要**放公司/客户数据；
- 本阶段用自己造的练习数据，完全没问题。

★ 如果不想装，用 Tableau Online 试用或直接跳过、改用 Power BI。

- 但简历上「BI 看板」这一项，有真作品和没有是两回事 —— 建议装。

知识点 2/7 • 维度 vs 度量：Tableau 的第一道门

- 一句话：维度（蓝）= 分类，用来切分；度量（绿）= 数字，用来算。
- 代码：
- douyin_notes.csv 字段分类：
- 维度：note_id / tag / publish_date / title 度量：views / likes / comments / shares / collects 判断规则：能求和的拖「度量」，能分组的拖「维度」。

★ 日期字段会被识别成「日期」类型，可以右键转成「离散（维度）」或「连续（度量）」—— 这一步决定它出现在列上还是行上，是新手最晕的地方。

知识点 3/7 • 连接 douyin_notes.csv：本机实测 200 行

- 一句话：连接 → 文本文件 → 选 CSV → 检查字段类型是否对。
- 代码：
- 数据 → 新建数据源 → 文本文件
- 路径：D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data\douyin_notes.csv 检查：views 是「数字（整数）」不是「字符串」；publish_date 是「日期」

★ 本机实测：200 行，views 合计 3,408,004，中位数 9,888。

- 如果 views 被识别成字符串，拖到行上只会显示「计数」而不是「求和」——这是「数字算不出来」最常见的原因。

知识点 4/7 • 第一张图：各标签播放量条形图

- 一句话：tag 拖列、views 拖行，排序，加标签。
- 代码：
- 列：tag（维度）
- 行：SUM(views)（度量）
- 排序：点工具栏「降序」标签：把 SUM(views) 再拖到「标签」卡格式：右键轴 → 设置格式 → 数字 → 自定义 → #,##0

★ 本机实测各标签播放量：职场 740,894 / 美食 703,311 / 知识 596,569 / 数码 534,917 / 穿搭 430,467 / 旅行 401,846（六项合计 3,408,004）。

这一页只讲「第一张图：各标签播放量条形图」。讲完就翻页，下一页是另一个知识点。

知识点 5/7 • 聚合粒度：Tableau 默认帮你 groupby

- 一句话：拖上去的每个维度都会进 GROUP BY，度量自动按当前粒度聚合。
- 代码：
- 只有 tag：SELECT tag, SUM(views) GROUP BY tag
- 加 publish_date（月）：SELECT tag, MONTH(publish_date), SUM(views)
- GROUP BY tag, MONTH(publish_date)

★ 「为什么我的数字变大了」几乎都是粒度问题：多拖一个维度 = 多一层分组，但求和的总数不变；多拖一个度量**才会翻倍（那是交叉连接）。**

知识点 6/7 • 本机数据的三个现成业务问题

- 一句话：每张图都要能回答一个问题，不然就是装饰。
- 代码：
- Q1 哪个标签播放量最高？ → tag × SUM(views)
- Q2 播放量和点赞率有关系吗？ → 散点图：views vs likes/viewsQ3 发布时间（月）对播放量的影响？ → 折线：MONTH(publish_date) × SUM(views)

★ 本机实测点赞率 5.26%（总 likes / 总 views）。

- 注意「总点赞率」和「每条点赞率再平均」是两个不同的数 ——前者加权，后者等权。报数时要说清用哪个。

知识点 7/7 • 先保存工作簿，再开始做图

- 一句话：文件 → 另存为 → 打包工作簿（.twbx），把数据一起带走。
- 代码：
- 保存：文件 → 另存为 → Tableau 打包工作簿 (*.twbx)
- 路径：D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\tableau\

- （.twbx 会把 CSV 打包进去，换电脑也能打开；.twb 只是引用路径，数据一挪就红叉。交付/进简历一律用 .twbx。）

代码速查：今天这几段，要在本机跑通

```
A. 装 Tableau Public（或 Power BI Desktop），装到 D 盘。
```

B. 连上 douyin_notes.csv，做 2 张工作表：

① 各标签播放量（条形，降序，带数值标签）

② 月度播放趋势（折线，publish_date 转月）

C. 存成 douyin_v1.twbx。

验收（做到才算过关）：

```
① 各标签播放量六个数 = 740,894 / 703,311 / 596,569 / 534,917 / 430,467 / 401,846;
```

② 能说出「维度 vs 度量」的区别，并举本数据里各两个例子；

③ .twbx 文件存在且能重新打开。

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

最容易错的地方

注意：Public 版的工作簿会存到公开云，**不要**放公司/客户数据；

★ 日期字段会被识别成「日期」类型，可以右键转成「离散（维度）」或「连续（度量）」—— 这一步决定它出现在列上还是行上，是新手最晕的地方。

如果 views 被识别成字符串，拖到行上只会显示「计数」而不是「求和」——这是「数字算不出来」最常见的原因。

注意「总点赞率」和「每条点赞率再平均」是两个不同的数 ——前者加权，后者等权。报数时要说清用哪个。

这些句子都是「差一个字、结果差一截」的地方，考前只翻这一页也值。

合上视频，自测一遍（能打勾才算学会）

- [ ] 我能不看资料写出：如果不想装，用 Tableau Online 试用或直接跳过、改用 Power BI。
- [ ] 我能不看资料写出：日期字段会被识别成「日期」类型，可以右键转成「离散（维度）」或「连续（度量）」—— 这一步决定它出现在列上还是行上，是新手最晕的地方。
- [ ] 我能不看资料写出：本机实测点赞率 5.26%（总 likes / 总 views）。
- [ ] 我能不看资料说出：如果 views 被识别成字符串，拖到行上只会显示「计数」而不是「求和」——这是「数字算不出来」最常见的原因。
- [ ] 我能不看资料说出：注意「总点赞率」和「每条点赞率再平均」是两个不同的数 ——前者加权，后者等权。报数时要说清用哪个。
- [ ] 我能不看题源，把今天用到的每个函数「为什么要有它」说一遍

这些句子都是「差一个字、结果差一截」的地方；三个勾以上，今天就算过关。

今天收工前，先知道明天学什么

第 25 天 • Tableau 看板 2/3：计算字段、筛选器、参数、仪表板拼装

不用提前学。明天点「① 开始今天的学习」，会换上明天那份 PPT。

收工三件事（5 分钟）

- 今天没跑通的脚本存进 90-我的作业\，命名 2026-11-25-第24天-*.py
- 打卡表「③ 每日打卡」记一笔：今天卡在哪、错在哪（一句话就够）
- 错题明天重做一遍，练习数据在 60-练习环境\Python-Env\data

全阶段 • 第 25 天 • 11/26

Tableau 看板 23：计算字段、筛选器、参数、仪表板拼装

做出带交互筛选的仪表板第一屏（KPI 卡片 + 趋势 + 结构图），筛选器能联动。

第 25 天 • 11/26 • 今日知识点回顾

Tableau 看板 2/3：计算字段、筛选器、参数、仪表板拼装

目标：做出带交互筛选的仪表板第一屏（KPI 卡片 + 趋势 + 结构图），筛选器能联动。

对着 B 站视频看：视频讲到的每一句，这张 PPT 里都有一条；复习时只翻这份，不用再回去翻分P。

今天结束时，你应该能做到

验收标准（做到这句就算过关）

- 做出带交互筛选的仪表板第一屏（KPI 卡片 + 趋势 + 结构图），筛选器能联动。

今天的做法（顺序别颠倒）

- 题库地址：file:///D:/DSH工作区/00-学习中心/60-练习环境/Python-Env/data
- 做完记一句：今天卡在哪、错在哪 → 写进 3-每日打卡。

今天看哪几集、读哪一章

这一天的知识：4 集 • 合计约 60 分钟（边看边在本机敲一遍）

P14	Tableau 视图与筛选器	约 16 分钟
P15	Tableau 标记卡	约 12 分钟
P22	Tableau 仪表板	约 18 分钟
P30	Tableau 计算字段	约 14 分钟

教材（已下载到 20-教材与讲义\python-data\，断网也能看）

`https://raw.githubusercontent.com/datawhalechina/joyful-pandas/master/source/Content/ch2.rst`

对照：计算字段 ≈ pandas 里新建一列

视频地址就在下面的「今天做哪些题」网页里，点一下直达分P。

知识点 1/7 • 计算字段：先把「率」算出来，别在图里除

- 一句话：分析 → 创建计算字段 → 点赞率 = SUM([likes]) / SUM([views])。
- 代码：
- 计算字段 1：点赞率 = SUM([likes]) / SUM([views])
- 计算字段 2：评论率 = SUM([comments]) / SUM([views])
- 计算字段 3：互动率 = (SUM([likes]) + SUM([comments]) + SUM([shares]) + SUM([collects])) / SUM([views])
- 数字格式：右键 → 默认属性 → 数字格式 → 百分比 → 2 位小数

★ 必须用 SUM/SUM 而不是 AVG(likes/views) —— 后者是「每条视频点赞率的平均」，会被小播放量的视频拉偏。加权 vs 等权，报数前想清楚。

知识点 2/7 • 参数：让用户自己选 TopN

- 一句话：右键数据窗格 → 创建参数 → 整数，范围 3–20；再用它做筛选。
- 代码：
- 创建参数：TopN（整数，当前值 10，范围 3-20，步长 1）
- 计算字段：TopN 标记 =RANK_UNIQUE(SUM([views]), 'desc') <= [TopN] 把「TopN 标记」拖到筛选器 → 选 True → 右键筛选器 → 显示参数

★ RANK_UNIQUE 保证不并列；用 RANK 遇到并列会一次显示很多条。

- 参数 + 计算字段是 Tableau 里唯一的「动态筛选」做法。

知识点 3/7 • 筛选器作用域：选错就白做

- 一句话：右键筛选器 → 应用于工作表 / 使用此数据源的所有工作表 / 仅此工作表。
- 代码：
- 仪表板里的筛选器，默认只作用于「它所在的那张工作表」。
- 要让筛选器联动所有图：右键筛选器 → 应用到工作表 → 选定工作表 → 全选

★ 「筛选器不动别的图」是 Tableau 新手第一大坑。默认作用域是「本工作表」，不是「整个仪表板」。

知识点 4/7 • KPI 卡片：一个数字 + 一行说明

- 一句话：用「文本」标记卡，把度量拖到文本上，就是一张 KPI 卡。
- 代码：
- 新建工作表 → 标记卡选「文本」文本：SUM([views])
- 标签：把计算字段「总播放量」拖到文本格式：字号 28，居中；副标题用「说明」卡

★ KPI 卡不要放超过 4 个，且每个都要有「对比」（环比/目标值），单独一个数字没有决策价值。

知识点 5/7 • 仪表板拼装：先定尺寸，再放图

- 一句话：仪表板 → 大小 → 固定 1200×800（或自动）；先放容器再放图。
- 代码：
- 仪表板 → 大小 → 固定大小 1200 × 800 左侧「对象」→ 水平容器 → 拖入 4 张 KPI 卡下方：垂直容器 → 折线图（左）+ 条形图（右）
- 标题：双击顶部 → 「抖音内容分析 · 总览」

★ 固定尺寸比「自动」好排，而且导出图片/PDF 时不会变形。

- 进简历的截图必须是固定尺寸导出的，不能靠屏幕截图。

知识点 6/7 • 三屏结构：总览 / 结构 / 明细

- 一句话：第一屏看整体，第二屏看拆解，第三屏看单条明细。
- 代码：
- 第 1 屏 总览：4 个 KPI（总播放/总点赞/点赞率/发布条数） + 月度趋势 + 标签结构第 2 屏 结构：标签 × 指标热力/条形矩阵 + TopN 参数控制第 3 屏 明细：单条视频表格（标题/日期/播放/点赞率），可按标签筛选

★ 三屏是「作品」和「练习」的分界线。一屏只能证明你会拖字段，三屏才证明你会组织信息。

知识点 7/7 • 配色：一套色，不超过 5 种

- 一句话：主色 #0F2E35、辅色 #2E7D8A、强调 #C0392B，其余用灰阶。
- 代码：
- 格式 → 颜色 → 编辑颜色 → 自定义主 #0F2E35 辅 #2E7D8A 浅 #9CC9D0 强调 #C0392B 灰 #8A9BA0 顺序色阶：浅蓝 → 深青（不要用红绿，色盲不友好）

- （配色统一后，三屏放一起才像一个作品。红绿对比在色盲眼里是同一个颜色，汇报时如果有人看不出差异，图就白做了。）

代码速查：今天这几段，要在本机跑通

A. 在 douyin_v1.twbx 基础上做 3 张新工作表：

```
① KPI 卡（总播放 3,408,004 / 总点赞 / 点赞率 5.26% / 发布条数 200）
```

② 月度播放趋势（折线，带环比标注）

③ TopN 视频（参数控制，默认 10）

B. 拼成第 1 屏仪表板（1200×800），筛选器联动全部图。

验收（做到才算过关）：

```
① 四个 KPI 数字与 pandas 算的一致（3,408,004 / 5.26% / 200）；
```

② TopN 参数从 10 改到 5，图里的条数跟着变；

③ 筛选器改标签后，趋势图和 KPI 卡同时变（不是只有一张变）。

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

最容易错的地方

★ 「筛选器不动别的图」是 Tableau 新手第一大坑。默认作用域是「本工作表」，不是「整个仪表板」。

这些句子都是「差一个字、结果差一截」的地方，考前只翻这一页也值。

合上视频，自测一遍（能打勾才算学会）

- [ ] 我能不看资料写出：RANK_UNIQUE 保证不并列；用 RANK 遇到并列会一次显示很多条。
- [ ] 我能不看资料写出：「筛选器不动别的图」是 Tableau 新手第一大坑。默认作用域是「本工作表」，不是「整个仪表板」。
- [ ] 我能不看资料写出：KPI 卡不要放超过 4 个，且每个都要有「对比」（环比/目标值），单独一个数字没有决策价值。
- [ ] 我能不看资料写出：固定尺寸比「自动」好排，而且导出图片/PDF 时不会变形。
- [ ] 我能不看资料写出：三屏是「作品」和「练习」的分界线。一屏只能证明你会拖字段，三屏才证明你会组织信息。
- [ ] 我能不看题源，把今天用到的每个函数「为什么要有它」说一遍

这些句子都是「差一个字、结果差一截」的地方；三个勾以上，今天就算过关。

今天收工前，先知道明天学什么

第 26 天 • Tableau 看板 3/3：第二三屏 + 导出 3 张截图进简历

不用提前学。明天点「① 开始今天的学习」，会换上明天那份 PPT。

收工三件事（5 分钟）

- 今天没跑通的脚本存进 90-我的作业\，命名 2026-11-26-第25天-*.py
- 打卡表「③ 每日打卡」记一笔：今天卡在哪、错在哪（一句话就够）
- 错题明天重做一遍，练习数据在 60-练习环境\Python-Env\data

全阶段 · 第 26 天 · 11/27

Tableau 看板 33：第二三屏 + 导出 3 张截图进简历

能放进简历和作品集。

第 26 天 • 11/27 • 今日知识点回顾

Tableau 看板 3/3：第二三屏 + 导出 3 张截图进简历

目标：三屏仪表板完成，导出 3 张 1600px 宽的 PNG，能放进简历和作品集。

对着 B 站视频看：视频讲到的每一句，这张 PPT 里都有一条；复习时只翻这份，不用再回去翻分P。

今天结束时，你应该能做到

验收标准（做到这句就算过关）

- 三屏仪表板完成，导出 3 张 1600px 宽的 PNG，能放进简历和作品集。

今天的做法（顺序别颠倒）

- 题库地址：file:///D:/DSH工作区/00-学习中心/60-练习环境/Python-Env/data
- 做完记一句：今天卡在哪、错在哪 → 写进 3-每日打卡。

今天看哪几集、读哪一章

这一天的知识：3 集 • 合计约 44 分钟（边看边在本机敲一遍）

P23	Tableau 故事	约 12 分钟
P24	Tableau 分享与导出	约 12 分钟
P32	Tableau 基本表与高亮表	约 20 分钟

教材（已下载到 20-教材与讲义\python-data\，断网也能看）

`https://raw.githubusercontent.com/datawhalechina/joyful-pandas/master/source/Content/ch4.rst`

第二屏的「标签 × 指标矩阵」就是 groupby + agg

视频地址就在下面的「今天做哪些题」网页里，点一下直达分P。

知识点 1/7 • 第二屏：标签 × 指标 矩阵，用高亮表

- 一句话：行放 tag、列放度量（用「度量名称」+「度量值」两个胶囊）。
- 代码：
- 列：度量名称（筛选只留 views / likes / 评论率）
- 行：tag 标记卡：方形标签：度量值（自动格式化）
- 颜色：度量值（浅蓝→深青）

★ 「度量名称 / 度量值」是 Tableau 的透视神器 —— 一次把多个指标排成列。

- 不熟悉的话，就做 3 张单独的表再拼，效果一样。

知识点 2/7 • 第三屏：明细表，别放太多列

- 一句话：6 列以内（日期/标题/播放/点赞/点赞率/标签），按播放降序。
- 代码：
- 行：note_id（或 title）
- 列：publish_date / views / likes / 点赞率 / tag 排序：views 降序筛选器：tag（作用于本屏所有图）

★ 明细屏的作用是「让老板能自己找一条看看」，不是放全量数据。

- 超过 10 列就没人看了。

知识点 3/7 • 故事（Story）：把三屏串成一条线

- 一句话：新建故事 → 每屏一个故事点 → 每个点配一句标题。
- 代码：
- 新建 → 故事故事点 1：「总览：200 条内容，340 万播放，点赞率 5.26%」故事点 2：「结构：职场类贡献 21.7%，是最强内容带」故事点 3：「明细：播放 Top10 里 6 条是职场类」

★ 故事点是「一句话结论 + 一张图」。汇报时按顺序点过去，就是一份不会跑题的 3 分钟讲解。

知识点 4/7 • 导出：图片 / PDF / 打包工作簿

- 一句话：仪表板 → 导出 → 图像（选 1600px 宽）；或导出 PDF。
- 代码：
- 导出图像：仪表板 → 导出 → 图像 → 宽度 1600 → 下载导出 PDF：文件 → 打印为 PDF 打包：文件 → 另存为 → 打包工作簿 (.twbx)

★ 屏幕截图（Win+Shift+S）分辨率不够，贴进简历会糊。

- 必须用 Tableau 自带导出，它按仪表板尺寸渲染。

知识点 5/7 • 截图进简历的三个硬要求

- 一句话：能看清标题、能看清至少一组数字、不出现 Tableau 水印以外的杂乱元素。
- 代码：
- 简历里的图：宽 800px 左右，高度按比例文件名：douyin_dashboard_1.png / _2 / _3 存放：D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\tableau\shots\

★ 简历上放 1 张全景图 + 1 句话（「3 屏交互看板，含 TopN 参数与联动筛选」）

- 就够了。三张全塞进一页简历会挤掉项目描述。

知识点 6/7 • 作品集的两种形式

- 一句话：PDF 三页（每屏一页 + 说明）或 Tableau Public 链接。
- 代码：
- PDF 作品集：shots/*.png → Word 三页 → 导出 PDFTableau Public：服务器 → Tableau Public → 保存到 Public → 复制链接

★ Public 链接在简历里很有用（HR 能点开看交互），但数据会公开 ——只用自己造的练习数据，绝不放公司数据。

知识点 7/7 • 自查：这份看板能不能「当场演示」

- 一句话：断网能开、数据不红叉、3 分钟内能讲完三屏。
- 代码：

```
DEMO_CHECK = [
'双击 .twbx 能直接打开（数据已打包，不红叉）',
'三屏切换流畅，无空白页',
'筛选器联动正常（改一个，三屏都变）',
'每个 KPI 都能说出「和什么比」',
'3 分钟内能讲完（计时器计时）',
```

知识点 7/7 • 自查：这份看板能不能「当场演示」（续）

-]

- （面试现场网络可能不通，演示一律用本地 .twbx，不要依赖 Public 链接。
- 链接只作为「回去可以看」的补充。）

代码速查：今天这几段，要在本机跑通

A. 完成三屏仪表板 + 故事。

B. 导出 3 张 1600px PNG 到 tableau\shots\。

```
C. 写 dashboard_readme.md（200 字）：数据是什么、三屏各回答什么问题、用了哪些交互。
```

验收（做到才算过关）：

① 三张 PNG 都能看清标题和数字，宽度 ≥1600px；

② .twbx 在另一台电脑（或换个路径）能打开不红叉；

③ 3 分钟讲解计时，不超 3 分 30 秒，且三屏各有一句带数字的结论。

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

合上视频，自测一遍（能打勾才算学会）

- [ ] 我能不看资料写出：「度量名称 / 度量值」是 Tableau 的透视神器 —— 一次把多个指标排成列。
- [ ] 我能不看资料写出：明细屏的作用是「让老板能自己找一条看看」，不是放全量数据。
- [ ] 我能不看资料写出：故事点是「一句话结论 + 一张图」。汇报时按顺序点过去，就是一份不会跑题的 3 分钟讲解。
- [ ] 我能不看资料写出：屏幕截图（Win+Shift+S）分辨率不够，贴进简历会糊。
- [ ] 我能不看资料写出：简历上放 1 张全景图 + 1 句话（「3 屏交互看板，含 TopN 参数与联动筛选」）
- [ ] 我能不看资料写出：Public 链接在简历里很有用（HR 能点开看交互），但数据会公开 ——只用自己造的练习数据，绝不放公司数据。

这些句子都是「差一个字、结果差一截」的地方；三个勾以上，今天就算过关。

今天收工前，先知道明天学什么

第 27 天 • 投递冲刺：把简历、项目、看板三件套对齐 JD

不用提前学。明天点「① 开始今天的学习」，会换上明天那份 PPT。

收工三件事（5 分钟）

- 今天没跑通的脚本存进 90-我的作业\，命名 2026-11-27-第26天-*.py
- 打卡表「③ 每日打卡」记一笔：今天卡在哪、错在哪（一句话就够）
- 错题明天重做一遍，练习数据在 60-练习环境\Python-Env\data

全阶段 • 第 27 天 • 11/28

投递冲刺：把简历、项目、看板三件套对齐 JD

投出 ≥8 家，每家都做了「JD 关键词 → 我的证据」映射；累计投递 ≥30 家。

第 27 天 • 11/28 • 今日知识点回顾

投递冲刺：把简历、项目、看板三件套对齐 JD

目标：投出 ≥8 家，每家都做了「JD 关键词 → 我的证据」映射；累计投递 ≥30 家。

对着 B 站视频看：视频讲到的每一句，这张 PPT 里都有一条；复习时只翻这份，不用再回去翻分P。

今天结束时，你应该能做到

验收标准（做到这句就算过关）

- 投出 ≥8 家，每家都做了「JD 关键词 → 我的证据」映射；累计投递 ≥30 家。

今天的做法（顺序别颠倒）

- 题库地址：file:///D:/DSH工作区/00-学习中心/60-练习环境/Python-Env/data
- 做完记一句：今天卡在哪、错在哪 → 写进 3-每日打卡。

今天看哪几集、读哪一章

这一天的知识：2 集 • 合计约 32 分钟（边看边在本机敲一遍）

P3	数据分析师的职业路径	约 14 分钟
P5	数据分析面试常见问题	约 18 分钟

教材（已下载到 20-教材与讲义\python-data\，断网也能看）

`https://raw.githubusercontent.com/datawhalechina/joyful-pandas/master/source/Content/参考答案.rst`

挑 5 道题默写，检验「能不能当场写」

视频地址就在下面的「今天做哪些题」网页里，点一下直达分P。

知识点 1/7 • JD 拆解：三类词，三种应对

- 一句话：硬技能词（要对得上证据）、业务词（要能聊）、软性词（要能举例）。
- 代码：
- JD 示例拆解：
- 「熟练使用 SQL 进行数据提取」 → 硬技能：我的 SQL 23 天 + 窗口函数练习「熟悉电商/内容业务指标体系」 → 业务：我做过漏斗和四层指标体系「有较强的跨部门沟通能力」 → 软性：需要准备一个具体例子

★ 硬技能词必须 100% 覆盖，业务词覆盖 70%，软性词准备 2 个例子就够。

- 时间有限时按这个优先级补。

知识点 2/7 • 证据映射表：一行一个 JD 要求

- 一句话：左边抄 JD 原话，右边写「我哪件事能证明」，写不出就标红去补。
- 代码：
- | JD 要求 | 我的证据 | 状态 || SQL 熟练 | 23 天 SQL 专项，41 题作战手册，窗口函数/CTE 实机练 | ✅ || Python 数据处理 | 1200 行脏数据清洗项目，6 条规则可复现 | ✅ || BI 工具 | Tableau 三屏看板，TopN 参数 + 联动筛选 | ✅ || A/B 实验 | 写过完整方案（样本量 2,332 人，判定规则表） | ⚠ 无真实实验 |

★ 标 ⚠ 的项要在面试时主动说「我做过设计，但没有真实流量跑过」——主动交代比被问出来好。

知识点 3/7 • 投递节奏：一天 8 家怎么排

- 一句话：上午 4 家（官网/内推，需要填表），下午 4 家（招聘 App，一键投）。
- 代码：
- 09:00-11:00 官网/内推 4 家（每家 30 分钟：读 JD + 改简历关键词 + 填表）
- 14:00-16:00 BOSS/拉勾 4 家（每家 20 分钟：打招呼语 + 发简历）
- 16:00-17:00 记录到投递追踪表 + 标 ⚠ 项

★ 官网投递慢但回复率高，App 投递快但容易被埋。两种都要有。

知识点 4/7 • 打招呼语：三句话，不要模板

- 一句话：我是谁 + 我做过什么（带数字） + 我为什么投这个岗。
- 代码：
- 「您好，我在找数据分析岗。做过一个 1200 行脏数据到业务结论的完整项目（清洗到 1163 行，产出品类/城市/漏斗三层汇总），SQL 和 pandas 都能独立写。
- 看到贵司这个岗要求指标体系搭建，我最近刚做完一套四层指标 + 主漏斗分析，方便的话想投个简历。」

★ 「您好，我对贵司岗位很感兴趣，附件是我的简历」—— 这种打招呼语回复率极低，因为它没有提供任何信息。

知识点 5/7 • 面试前 24 小时的三件事

- 一句话：查公司业务、准备 2 个项目故事、准备 3 个反问。
- 代码：
- ① 查业务：公司靠什么赚钱、核心指标是什么、最近有什么新闻② 项目故事：STAR 结构各准备 1 个（清洗项目 + 看板项目）
- ③ 反问 3 个：团队现在最痛的指标是什么 / 我这个岗前三个月要交付什么 / 数据团队和业务团队怎么协作

★ 反问环节是加分项。「你们加班多吗」这类问题留到 offer 阶段再问。

知识点 6/7 • 被拒之后做什么

- 一句话：记录拒的原因（如果能拿到），一周后复盘一次，别当天情绪化改简历。
- 代码：
- 投递追踪表加两列：结果（待回复/面试/拒/无回复）、复盘（一句话）
- 每周日花 20 分钟看一遍：哪类岗位回复率高、哪个渠道有效

★ 30 家投出去没有回复，通常不是简历问题，是「岗位匹配度」或「渠道」问题。

- 复盘看数据，不要凭感觉改。

知识点 7/7 • W7 验收清单

- 一句话：统计、A/B、指标、看板、投递，五块都要有产出物。
- 代码：

```
CHECK_W7 = [
'能说清 P 值的三种误用，并解释 Cohen d',
'有一份完整的 A/B 方案（含样本量计算与判定规则）',
'能对任意 App 说出四层指标体系并画主漏斗',
'有三个优化点且能排序（影响面 × 可执行性）',
'有一份三屏 Tableau 看板 + 3 张截图',
'累计投递 ≥30 家，每家都有证据映射',
'60 秒自述 + 3 分钟项目讲解都能不看稿说完',
```

知识点 7/7 • W7 验收清单（续）

-]

- （7 条里「累计投递 ≥30」是最硬的指标 —— 它不能靠突击，只能靠每天 45 分钟。
- 如果现在是 8 家，说明前面几周的投递没跟上，今天要多投。）

代码速查：今天这几段，要在本机跑通

```
A. 投递 ≥8 家（含官网/内推 4 家 + App 4 家），每家做证据映射。
```

```
B. 更新投递追踪表：累计行数 ≥30，每行有「JD 要什么 / 我对得上什么」。
```

```
C. 准备 3 个反问 + 2 个 STAR 项目故事（写下来，各 150 字）。
```

验收（做到才算过关）：

① 投递追踪表累计 ≥30 行；

② 8 家的证据映射里，标 ⚠ 的项不超过 2 个；

③ 两个 STAR 故事能各讲 90 秒，含至少 2 个数字。

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

合上视频，自测一遍（能打勾才算学会）

- [ ] 我能不看资料写出：硬技能词必须 100% 覆盖，业务词覆盖 70%，软性词准备 2 个例子就够。
- [ ] 我能不看资料写出：标 ⚠ 的项要在面试时主动说「我做过设计，但没有真实流量跑过」——主动交代比被问出来好。
- [ ] 我能不看资料写出：官网投递慢但回复率高，App 投递快但容易被埋。两种都要有。
- [ ] 我能不看资料写出：「您好，我对贵司岗位很感兴趣，附件是我的简历」—— 这种打招呼语回复率极低，因为它没有提供任何信息。
- [ ] 我能不看资料写出：反问环节是加分项。「你们加班多吗」这类问题留到 offer 阶段再问。
- [ ] 我能不看资料写出：30 家投出去没有回复，通常不是简历问题，是「岗位匹配度」或「渠道」问题。

这些句子都是「差一个字、结果差一截」的地方；三个勾以上，今天就算过关。

今天收工前，先知道明天学什么

第 28 天 • ★W7 总验收 + 三周复盘：从 SQL 到 Python 到看板

不用提前学。明天点「① 开始今天的学习」，会换上明天那份 PPT。

收工三件事（5 分钟）

- 今天没跑通的脚本存进 90-我的作业\，命名 2026-11-28-第27天-*.py
- 打卡表「③ 每日打卡」记一笔：今天卡在哪、错在哪（一句话就够）
- 错题明天重做一遍，练习数据在 60-练习环境\Python-Env\data

全阶段 • 第 28 天 • 11/29

★W7 总验收 + 三周复盘：从 SQL 到 Python 到看板

用一份 5 分钟的完整演示串起「数据清洗 → 分析 → 看板 → 结论」，并把 28 天的产出物清单列清楚。

第 28 天 • 11/29 • 今日知识点回顾

★W7 总验收 + 三周复盘：从 SQL 到 Python 到看板

目标：用一份 5 分钟的完整演示串起「数据清洗 → 分析 → 看板 → 结论」，并把 28 天的产出物清单列清楚。

对着 B 站视频看：视频讲到的每一句，这张 PPT 里都有一条；复习时只翻这份，不用再回去翻分P。

今天结束时，你应该能做到

验收标准（做到这句就算过关）

- 用一份 5 分钟的完整演示串起「数据清洗 → 分析 → 看板 → 结论」，并把 28 天的产出物清单列清楚。

今天的做法（顺序别颠倒）

- 题库地址：file:///D:/DSH工作区/00-学习中心/60-练习环境/Python-Env/data
- 做完记一句：今天卡在哪、错在哪 → 写进 3-每日打卡。

今天看哪几集、读哪一章

这一天的知识：3 集 • 合计约 40 分钟（边看边在本机敲一遍）

P36	假设检验与 P 值	约 14 分钟
P42	第一类错误与第二类错误	约 12 分钟
P46	Tableau 表计算（YTD）	约 14 分钟

教材（已下载到 20-教材与讲义\python-data\，断网也能看）

`https://raw.githubusercontent.com/datawhalechina/joyful-pandas/master/source/Content/参考答案.rst`

全书答案，最后过一遍，把错的题标出来重做

视频地址就在下面的「今天做哪些题」网页里，点一下直达分P。

知识点 1/7 • 5 分钟演示的结构：4 段，每段 75 秒

- 一句话：问题 → 数据与清洗 → 分析结论 → 看板演示。
- 代码：
- 0:00-1:15 问题：电商订单转化漏斗，想知道卡在哪一步1:15-2:30 数据与清洗：1200 行、6 类脏点、6 条规则、收敛到 1163 行2:30-3:45 分析结论：paid 417 单 42.2 万；加购→下单 49.3%，卡点在这里3:45-5:00 看板演示：三屏，改一个筛选器让它联动

★ 演示顺序不能反 —— 先讲清洗会让人觉得「你在讲工具」，先讲问题才会让人觉得「你在解决问题」。

这一页只讲「5 分钟演示的结构：4 段，每段 75 秒」。讲完就翻页，下一页是另一个知识点。

知识点 2/7 • 产出物清单：28 天到底留下了什么

- 一句话：脚本、表、图、文档、看板，五类各数一遍。
- 代码：

```
ARTIFACTS = {
'脚本': ['numpy_probe.py', 'clean_orders_v2.py', 'filter_practice.py',
'groupby_practice.py', 'merge_practice.py', 'pivot_practice.py',
'timeseries_practice.py', 'apply_practice.py', 'viz_three.py',
'run.py + src/*.py（项目）'],
'数据产物': ['clean_v2.csv', 'clean_log.json', 'summary_*.csv',
'monthly.csv', 'pivot_out/*.csv', 'ts_out/*.csv'],
'图': ['fig1_month.png', 'fig2_city.png', 'fig3_funnel.png'],
'文档': ['README.md', 'ab_design.md', 'metrics.md', 'dashboard_readme.md'],
'看板': ['douyin_v1.twbx（三屏）', 'shots/*.png（3 张）'],
```

知识点 2/7 • 产出物清单：28 天到底留下了什么（续）

- }

```
for k, v in ARTIFACTS.items():
print(f'{k} ({len(v)} 项) : ', '、'.join(v))
```

★ 这份清单就是简历「项目经历」的素材库。写不出清单，说明很多东西只在脑子里过了一遍，没有落成文件。

知识点 3/7 • 三周能力对照：SQL / Python / BI 各到什么程度

- 一句话：每项用「能不能当场做」来判定，不用「熟不熟」。
- 代码：

```
ABILITY = {
'SQL': ['多表 JOIN 不查资料', '窗口函数写排名/累计', 'CTE 拆复杂查询',
'能解释执行计划里的 type=ALL'],
'Python': ['pandas 清洗六规则', 'groupby/pivot_table/resample',
'能写 100 行以上脚本不查语法', '能用 argparse 做命令行工具'],
'BI': ['三屏仪表板', '计算字段与参数', '筛选器作用域', '导出高分辨率图'],
'统计': ['描述统计与偏度', 't 检验与卡方', 'P 值三种误用', '样本量计算'],
```

知识点 3/7 • 三周能力对照：SQL / Python / BI 各到什么程度（续）

- }

```
for k, v in ABILITY.items():
print(f'{k}: {len(v)} 项 →', ' / '.join(v))
```

★ 每一项都要能「当场写出来」。看着眼熟但写不出，等于不会。

知识点 4/7 • 复盘：这 28 天哪一步最费时间

- 一句话：把每天的打卡记录翻一遍，找出「计划 3 小时、实际 6 小时」的那几天。
- 代码：
- 复盘问题：
- ① 哪一类任务总是超时？（视频太长 / 练习太难 / 环境问题）
- ② 哪些知识点是「看懂了但写不出」？
- ③ 投递为什么没跟上？（时间被学习吃掉 / 不敢投 / 简历没准备好）

★ 超时不可怕，可怕的是不知道为什么超时。找出一个反复出现的模式，下个月就能省下几十小时。

知识点 5/7 • 下个月的三件事

- 一句话：只写三件，多了做不完。
- 代码：

```
NEXT = [
'① 投递到 60 家，把回复率当指标优化（每周复盘渠道）',
'② 补一个「真实数据」项目（Kaggle 或公开数据集，不再是自造数据）',
'③ 面试题库刷 50 题（SQL 手写 20 + 统计 15 + 业务 15）',
```

-]

★ 「自造数据」是这一阶段最大的短板 —— 面试官一眼能看出来。

- 下个月必须换成公开真实数据集（Kaggle / 天池 / 国家统计局）。

这一页只讲「下个月的三件事」。讲完就翻页，下一页是另一个知识点。

知识点 6/7 • 把作品整理成一个入口

- 一句话：一个 README 索引，把所有产物的路径列清楚。
- 代码：

```
# 数据分析作品集（2026.10–11）
## 1. 电商订单分析（Python）
```

· 代码：py-portfolio/order-analysis/
· 结论：1200→1163 行清洗，paid 42.2 万，加购→下单 49.3%

```
## 2. 抖音内容分析（Tableau）
```

这一页只讲「把作品整理成一个入口」。讲完就翻页，下一页是另一个知识点。

知识点 6/7 • 把作品整理成一个入口（续）

· 看板：tableau/douyin_v1.twbx（三屏）
· 结论：200 条 340 万播放，职场类占 21.7%

```
## 3. A/B 实验设计（文档）
```

· 方案：docs/ab_design.md（样本量 2,332 人）
· 指标：docs/metrics.md（四层指标体系 + 主漏斗）

★ 一个 README 索引 = 一个作品集。面试官打开一个链接就能看全，不用你一个个发文件。

知识点 7/7 • 最后一件事：把打卡表填完

- 一句话：28 天的打卡记录是「我真的做了」的唯一证据，空着就等于没做。
- 代码：
- 打开 3-每日打卡（求职冲刺计划表）.xlsx → 每日打卡 sheet11/02-11/29 这 28 行，每行填：学了什么 / 卡在哪 / 用时

- （打卡表目前只有 10-10 第 1 天填了。这不是形式主义 ——面试问「你这两个月做了什么」时，有记录的人能说出细节，没记录的人只能说「我在学习」。）

代码速查：今天这几段，要在本机跑通

A. 5 分钟完整演示（录屏或对着人讲），计时。

B. 产出物清单（ARTIFACTS 那份，按实际文件填）。

C. 三周能力对照表，逐项标「能当场做 / 眼熟 / 不会」。

D. 下个月三件事，写进 4-每周该做什么。

```
E. 补填 11/02–11/29 的打卡记录（能补多少补多少，至少写「学了什么」）。
```

验收（做到才算过关，这是 W4–W7 的总验收）：

① 5 分钟演示能讲完，含 ≥6 个具体数字；

② 产出物清单里五类都有实物（脚本/数据/图/文档/看板）；

③ 能力对照表里「能当场做」的项 ≥70%；

④ 打卡表 28 天至少有一句记录。

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

合上视频，自测一遍（能打勾才算学会）

- [ ] 我能不看资料写出：演示顺序不能反 —— 先讲清洗会让人觉得「你在讲工具」，先讲问题才会让人觉得「你在解决问题」。
- [ ] 我能不看资料写出：这份清单就是简历「项目经历」的素材库。写不出清单，说明很多东西只在脑子里过了一遍，没有落成文件。
- [ ] 我能不看资料写出：每一项都要能「当场写出来」。看着眼熟但写不出，等于不会。
- [ ] 我能不看资料写出：超时不可怕，可怕的是不知道为什么超时。找出一个反复出现的模式，下个月就能省下几十小时。
- [ ] 我能不看资料写出：「自造数据」是这一阶段最大的短板 —— 面试官一眼能看出来。
- [ ] 我能不看资料写出：一个 README 索引 = 一个作品集。面试官打开一个链接就能看全，不用你一个个发文件。

这些句子都是「差一个字、结果差一截」的地方；三个勾以上，今天就算过关。